

TripMaestro – Computer Science A2 Project

Contents:

Table of Contents

Analysis.....	3
Stakeholder	3
Research Plan	3
Primary Research.....	3
Secondary Research.....	5
Proposed Solution	7
Essential Features.....	7
SMART Success Criteria.....	8
Why a computational solution?	8
Hardware & Software Requirements	9
Solution Limitations.....	10
Design	12
Justification for Programming Approach	12
Justification for Programming Language	13
Abstracted Solution	15
Structure Diagrams.....	15
Algorithms.....	16
GUI Designs	21
Data dictionary, key variables and methods.....	25
Database Structure	30
Test Plan.....	31
Iterative Test Plan.....	31
Post Development Test Plan	36
Development	38
Account system	38
User login	38
User registration.....	42
Destination recommendations.....	49
Filter system	50
Pagination	52

API Data Retrieval.....	57
Holiday creation	73
Calendar table	75
new_holiday page.....	78
Calendar page(s).....	85
new_activity page.....	96
Adding and deleting to the calendar	97
Current activity details.....	101
Saving the calendar	110
Loading a calendar.....	112
Testing calendar functionality	113
Budget management	119
Testing	125
Testing for function	125
Testing usability features	153
Evaluation.....	161
Evaluating usability features	162
Maintaining the solution.....	168
Limitations of the solution	169
Final Solution Code	170

Analysis

My stakeholder is a online blog company who is looking to have a central point to where they can advise people about how to plan a holiday.

They currently have a problem where information about holidays is currently scattered across their different blog posts, and they want to be able to direct people looking for advice to a website to answer their questions, as well as add new information to the website as the listed holiday destinations develop in terms of attractions as well as price.

The stakeholder wants the solution to have an interactive, drag-and-drop way to show the allocation of time during a holiday. It needs to be easy for the user to work on multiple holidays at the same time as well as for the user to be able to login to access their current projects at any given time.

The stakeholder would require me to act in the interest of the users of their blog so they feel fulfilled by the website and what it offers in terms of holiday planning in hope of retaining them as a user. These blog viewers are often identified as desktop users looking to get comprehensive advice about their holiday, so would feel fulfilled by the planner that I am working on which can help bring all of this information together.

Research Plan

I intend to conduct my research with a questionnaire. This is because there are some things that can be gauged well with a questionnaire such as the intended vibrance of the website's colours, which does not really require the deeper thought that is synonymous with an interview. Moreover, the questionnaire can also help to clarify the priority of the different things that the stakeholder wants displayed to the user after they have identified their holiday destination.

Questionnaire Analysis

The screenshot shows a questionnaire titled "TripMaestro Development Form". It contains two questions:

1. How vibrant do you want the website's colours? 5 - Very colourful, 1 - Darker colours

The response options are 1, 2, 3, 4, and 5. Option 5 is selected.

2. Prioritise these in terms of how they should appear to the user when they decide their holiday destination.

The response options are:

- Activities
- Prices
- Types of residence available
- Scenery
- Dietary requirements
- Luggage

3. What do you want the login functionality to achieve?

I want the user to be able to login in order to continue working on a previous holiday they had started planning previously. I also want them to be able to work on multiple at the same time, as well as for the accounts to have a memory system so it understands that specific user's priority/preferences when deciding different characteristics in their holidays.

4. What devices do you see the website being run on, or need compatibility for?

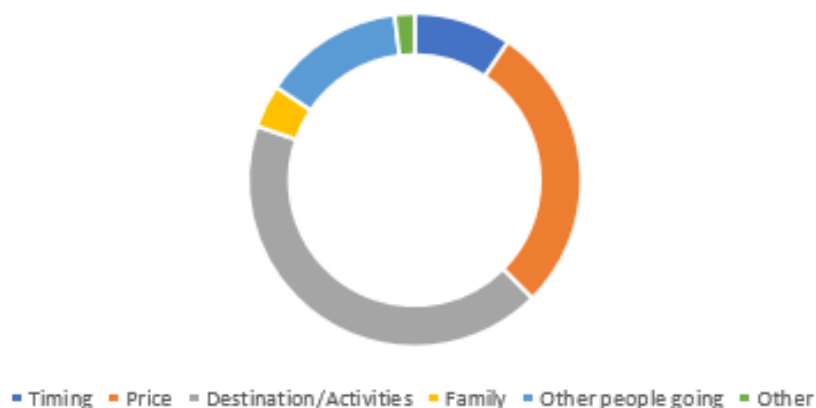
It is imperative that this works for desktops as well as laptops, and can assume the use of a trackpad or mouse in the use of the website. There is little reason for the website to need to run on a phone as I think it can't really provide enough information at a time for the user to actually understand what's happening in their holiday.

5. Is there anything you explicitly don't want in the website?

The website needs to not have any biases towards certain holidays based on pre-existing information or information that is pulled from other websites. It also needs some basic system to check information coming from other sources to make sure an all-inclusive holiday isn't advertised as £5 when obviously that isn't true.

Some key conclusions to draw from the questionnaire is that the stakeholder doesn't want the website to have too much information, but enough to make informed decisions about what to expect in that destination. This is demonstrated in how it needs to understand preferences of holidays for individual users, but not overdo this where it needs to have no excessive biases through information it may receive from 3rd party sources. There also was a later request to give some questions for the blog readers to identify their preferences about holiday planning.

What do you consider first when planning a holiday? Sampled from 623 blog respondents



This desire for information about the holiday is shared by the blog respondents as well as the shareholder, but also highlights a need for price to be easily accessible on the website.

How do you currently plan holidays? (sampled from 590 blog respondents)



This is another pie chart taken from the blog respondents, and develops the idea that the final solution shouldn't be over-ambitious and should be kept simple, as if it is too difficult to use the website, people who hadn't considered online planning before will be kept away from it. This demographic is shown to be major through the large number of respondents that said they had previously used offline word processors, or pen and paper.

Existing Solutions

Mostar: 528 properties found

Sort by: Top picks for solo travellers

Commission paid and other benefits may affect an accommodation's ranking. [Find out more.](#)

Your results include some shared accommodations, such as dormitory beds. [Show private rooms only](#)

Hotel Pasha ★★★★★
Mostar [Show on map](#) 200 yards from centre
Superior Twin Room
2 single beds
Breakfast included
✓ Free cancellation
✓ No prepayment needed – pay at the property
Only 3 rooms left at this price on our site
Fabulous 8.8 (1,598 reviews)
2 nights, 1 adult: £142
Includes taxes and charges
[See availability](#)

Lovely Bosnian Home ★★★☆☆
Mostar [Show on map](#) 200 yards from centre
Bed in 4-Bed Dormitory Room
Bed in dormitory
Superb 9.0 (259 reviews)
Location 9.6
2 nights, 1 adult: £33

This shows Booking.com's solution to showing available places to stay at a destination, but offers little in terms of recommending personalised destinations and suggesting activities, both creating a major difference compared to TripMaestro.

Summer Holiday 21 June – 21 July

Cost in GBP (£) £0.00 30/30 Nights planned

Destinations **Day by day** Show notes

DATE	DESTINATION	SLEEPING	DISCOVER
Fri 21 Jun 2024 Day 1	Mostar Start of your adventure!	+	+
Sat 22 Jun 2024 Day 2	Mostar Spend the day in Mostar	+	+
Sun 23 Jun 2024 Day 3	Mostar Spend the day in Mostar	+	+
Mon 24 Jun 2024 Day 4	Mostar Spend the day in Mostar	+	+
Tue 25 Jun 2024 Day 5	Mostar Spend the day in Mostar	+	+
Wed 26 Jun 2024 Day 6	Mostar Spend the day in Mostar	+	+
Thu 27 Jun 2024 Day 7	Mostar Spend the day in Mostar	+	+
Fri 28 Jun 2024	Mostar		

Map: Add destinations by clicking on the map. Locations shown include Ljubljana, Posušje, Ploče, Sarajevo, and Mostar.

This is stippl.io's way of showing the different components of a holiday, organising it into a map and a list of the days and the activities planned for those days. The list can be argued to not be very appealing to the user, and can show the importance of having a drag and drop functionality in TripMaestro's solution.

London (LON) Anywhere Thu 25/7 Thu 1/8

Stops

- ☒ Any stops
- ☐ Up to one stop
- ☐ Direct flights only

Budget £1,500+

Duration 12+ hours

Type of trip

- ☒ Anything
- ☐ City
- ☐ Beach
- ☐ Family

Map: Shows flight prices to various European destinations. Price tags include £31, £84, £92, £178, £206, £221, £201, £142, £144, £178, £118, £261, £164, £81, £155, £95, £178, £206, £92, £84, £31, £142, £144, £178, £118, £261, £164, £81, £155, £95, £178, £206, £92, £84, £31.

Kayak.co.uk utilises a map to show a variety of destinations and the respective price for travel to that destination. This could be too restrictive where the user may not see the availability of the destination as the website may choose to not show that place's price. It also offers little insight to what's available at the destination, as well as the price of different costs incurred during a holiday.

Proposed Solution

Features to implement or not implement

- Interactive map to identify destinations:
This can impact the stakeholder where the implementation of this deviates from the initial requirements to have a focus on the practical benefits of the trip such as advertising price as well as activities, which often has to be disregarded when displaying information on a map as seen in the screenshot from kayak.co.uk on page 5. This will not be implemented in the program as the time spent on it doesn't seem proportional to the benefit of the user seeing everything in one place, when it isn't too much harder to redirect the user to a Google Maps site that can also do the same thing.
- A review system:
This can impact the stakeholder where there is less reliance on what the blogger has posted as the website's users will naturally consider the reviews that are attached to the different activities. Moreover, this can increase the complexity of using the program now that there is more information attached to each destination or airline. It does provide an advantage though for the user, but this deviates from the stakeholder's initial requirements where there is to be a place for questions and answers, whereas a review system deviates from this and can make the user feel detached from the information they want. Consequently, this review system will not be implemented, but there will be links available to other sources' reviews, where likely there would be more reviews anyways.
- Recommendation system when suggesting holidays
This idea stems from Booking.com's way of selecting accommodation for the holiday as seen on page 4 where the search results are tailored to what the user has been identified as having preference to. It helps to meet the stakeholder's requirement of maintaining an interactive program as it helps to establish a connection between the user and what they are planning. This idea will be implemented and would likely require a database or external source which contains all of the options for the user to select and how they can be filtered.

Essential features to implement

1. Ability to add unique notes for each part of a trip to help those who originally used word processors to plan their holidays
2. Ability for users to set preferences for the components of their holiday so the user doesn't have to waste time looking at things they would never normally consider (as was the case for Booking.com on page 4)
3. Ability to drag and drop different parts of the holiday in terms of time as well as overall budget to plan it and make it interactive unlike stippl.io's list on page 5
4. Ability to login to access the holidays they had previously started planning
5. Ability to check validity of the sources for holiday information
6. The program needs to be appealing and not be cluttered
7. Program needs to check that different components of the holiday so that a flight and an activity do not overlap

Success Criteria:

These are criteria I want to work towards during the development of the program, that if met, will indicate the success of the project.

1. The website has drop down menus that points the user to a list of destinations, and clicking on one of those destinations takes you to another external website to find out more. An external website is used as they will be able to provide specialist information from people who have visited that destination and given reviews, unlike the main website where, when it is launched, no reviews would be on there.
2. The main website can take inputs that indicate preferences about holidays, with a set of buttons ranging from “never” to “very likely” about their tendencies to get involved with different types of activities such as hiking, exploring historical landmarks and visiting scenic areas. This can help provide a more complete user experience as the website can tailor what is displayed based on their previous responses, helping them to find the destination they want to go.
3. When the user first opens the website, there is a login prompt for text boxes for a username and password. There is also another option below the textboxes to register, and these are saved in a database for later use. This enables holiday plans to be stored for later use and modification, so the user is in no pressure to finish planning the holiday in one go.
4. When the user decides on their destination, they are prompted with different categories of activities to select from. When opening these categories, the user is given a prompt to select how much time should be allocated for that. This time is then represented on a block on a calendar for the user to drag for when they want it. The calendar is meant to save those activities, and display the details of those activities when prompted.
5. The website has a textbox below each day that has been planned for the user to add their own specific notes about things that are not included in the original set of activities. For example, a large group may use this to indicate who is participating in certain activities and who is staying back in the accommodation.
6. When the user clicks on an activity it gives the user a form to fill in regarding that activity. This asks for various information that can be useful later, such as a time duration, when it starts/finishes, whether food is available there, what transport will be used, as well as contact information for that place if it is necessary. This is helpful as it can make the user feel that the website is more useful than just a list of activities written on paper, because it can provide information that the user may not immediately need but might want to know later in the trip.

Why a computational solution?

Abstraction is the removal of unnecessary details to simplify the creation of a solution to a problem. This can be implemented here where details about what to pack can be ignored from the final solution. However, there is an immediate disadvantage by implementing this as the user has less of a complete experience where they are essentially left on their own about this aspect of holiday planning. On the other hand, it can be advantageous as it keeps the user feel personal responsibility about their holiday, and also avoids the problem during its implementation where you have to account for every person and the unique requirements they have. This makes it easier to achieve a good final solution as the benefit

to the stakeholder doesn't seem proportionate to the time it would take to account for what people may want to take on holiday.

Decomposition is the breaking down of a problem into smaller, more manageable chunks in order to make it easier to create a solution to the problem. This can be applied here where the problem can have smaller sub-problems identified such as the logging in system, the linking of the website to a database, the grabbing of the holiday data from APIs, as well as taking that data and putting it into drag-and-drop form. This can be a problem to use should the website be very integrated with each other, but the website is designed to plan a holiday, so easily can be broken down to the different aspects of this. For example, picking a holiday destination doesn't really involve the same processes as allocating time to activities. There is obviously some overlap, where both may use the preferences of the user to produce an output, but it is easier to tackle these as two separate tasks so there isn't too much happening at once. This helps the programmer not get overwhelmed in the creation of the program, as well as when the website is being used. This helps to reduce the system requirements for people using the website and is especially important for the stakeholder who would want the website to be accessible to as many of its viewers as possible.

Thinking ahead is where a solution to a problem is provided with knowledge of what future aspects of the problem may include. This can be on a smaller scale such as adding an empty variable at the start of the program, that may only be used at the end to store the total of some variables. This also applies to this website, where the website can be designed in a way so that it already has thought about future possibilities as well as logical proceedings for the user such as providing a link for information as well as bookings for an activity that the user has expressed desire for. This is advantageous to the user where it requires less thinking for themselves in terms of what to do next, rather the website can help them through the process, compared to an existing pen and paper solution which would see them literally have a blank paper, and may overlook some aspects of the holiday that the website wouldn't. Thinking ahead is also beneficial in programming the website as it can reduce the need to go back to previous aspects of the website that was developed so it stays compatible with what is made after. An example of this could be in the implementation of the login system, where the ability to think ahead can point the programmer to the need to work out what database will be used, and what fields would be necessary for each user later in the program.

Hardware and Software Requirements:

Hardware needed for programmer:

- CPU to run the computer that will be used, a quad-core i5 or quad-core Athlon X4 or later should be used. It is clear that quad-core processor would be necessary for multi-tasking the different programs that likely would be open simultaneously. For example, the programmer may need to have an IDE open to write code for the website, as well as an actual web browser to test their changes
- 8GB of RAM seems necessary in the development of the website as this would complement the different applications that would need to be open simultaneously, as well as different images that the programmer may have open when deciding what goes in the website

- GPU with DirectX 10 or later to enable smooth operation of Windows
- Keyboard and mouse to interact with the operating system and IDE
- Monitor to see the code and the website

Software needed for programmer:

- IDE to provide a platform for coding
- Windows 10 or later for a modern multitasking operating system
- A web browser to see the website that has been made
- At least 128GB of disk space in order to store the Windows operating system, the website's contents, as well as the storage needed to store the IDE

Hardware needed for stakeholder:

- Monitor, to see the website
- Keyboard, to add notes and type in details of different aspects of the holiday
- Mouse, to interact and click on buttons to navigate the website
- CPU, to run the website, should be a Core 2 Duo E6300 or better for Intel processors, or a Phenom II or later for AMD processors. These processors have multiple cores to allow parallel processing to occur when the website is being used, meaning it can show things to the user as well as grab data from elsewhere on the Internet at a greater speed. It also allows a smooth experience in terms of dragging and dropping items on the day planner.
- GPU, to provide a display output from the computer so the website can be seen
- RAM, 2GB is recommended by Google for operation of Google Chrome, the most common web browser, and is reasonable for opening this website which would have demands that reflect a common website

Software needed for stakeholder:

- A web browser such as Google Chrome to see the website
- Windows 7 or later for the operating system to enable the running of apps

Solution Limitations

- The time available for development of the website is a major limiting factor in what it can do in the long term. This prevents time from being allocated to details such as providing information on what to pack, as well as accessibility options like alternative text for images that may be displayed. Moreover, it is likely to not be very suitable for mobile use, with larger icons and the website being formatted in a way where it is designed to use the full space of a PC monitor. This likely wouldn't be a major problem as it would be difficult to plan a holiday on a phone anyways where there is only so much information that can be provided on the screen at a time.
- The website will be limited in the ability for the user to book directly from the website, where that would require synchronisation with the different providers of hotels, flights etc. This would need a lot more time to implement and may make the user feel that using this website is a hassle in comparison to others like Booking.com where everything can be paid for and booked on the website. This can be a problem for the stakeholder if their vision for the blog + website combined is to be the only

place you need to be for holiday planning, but if it is just meant to be complementary to other websites it is no problem.

- The website has a limitation in the information it can provide, as I cannot provide all of the information that people may want about a destination, partly due to time but also due to me not specialising in providing that information. A competing website could hire people to fill in the information that customers may want, whereas I would be limited to however much I know and would be forced to link other websites in order for customers to find out what they want to know. This can be fixed by working with the stakeholder and the bloggers who worked on the main blogs to provide information in a way that is compatible with the holiday planner I'm developing, although this requires input from the stakeholder, who may not be invested enough into the planner to commit resources to making it a complete and competitive website.

Design

Possible Programming Approaches:

Procedural programming:

This is where code is organised into procedures or functions, where the overall logic of the program is broken down into smaller subroutines that can be called at different times. These encourage modularity, but are often attached very closely to specific tasks, so aren't reusable to many contexts.

Advantages:

- It is simpler to understand and write, with a linear flow of execution. This makes it ideal for smaller tasks and projects, as well as for debugging.
- It can be more memory-efficient as it doesn't require the additional structures that object-oriented programming (OOP) relies on such as classes or objects. This can improve the performance of the program and make it usable on more devices.

Disadvantages:

- It is difficult to scale up a procedural program, as functions can be interdependent and hard to maintain.
- The program may rely on global variables, and bugs may arise from this where changes in one part of the program can affect other parts unexpectedly.
- Functions aren't really flexible enough to be reused across different contexts without significant modification.

Object-oriented programming (OOP):

This is a paradigm organising the design of software around data/objects, as opposed to logic contained in functions. Classes are used as blueprints to create objects. Inheritance is a key feature to OOP, and allows classes to inherit properties and methods from another class, allowing for code to be reused. This implicitly creates a hierarchy between classes where some further down the hierarchy add more specific and niche attributes.

Advantages:

- Classes and objects promote modular code, making it easier to reuse and extend existing code for slightly different needs.
- OOP's structure helps to manage and scale applications as they grow in complexity, where the abstraction of more complex details means the developer has less complexity to deal with at a time.

Disadvantages:

- The structure of classes and objects can be more complex than is needed for smaller projects.
- There is a risk of over-engineering where developers might create unnecessary classes or inheritance, where the use of OOP is counter-productive, especially if it is developed by someone unfamiliar with the paradigm.

With the scale of the project, it seems more suitable to use object-oriented programming, where the time involved to become familiar with the paradigm will be worth it. This can be attributed to how most parts of the program will be similar where they take requirements for the holiday and add it to the user's final plan for their holiday. This kind of reusability of code in OOP is especially important where the stakeholder specifically required the ability to work on multiple holidays at once.

Possible Programming Languages:

Kotlin

Kotlin is a language that runs on the Java Virtual Machine, with heavy integration with Java.

Advantages:

- The interoperability it has with Java makes backend development easier for databases and communication with APIs.
- Kotlin is very useful for developing Android applications with its easy integration into Android Studio.

Disadvantages:

- The ecosystem is simply younger than JavaScript so there may be fewer resources for a niche problem when using Kotlin.
- It can be more difficult to learn, where it is more complex with more features, and was made in a way that just covers over Java's shortcomings.

JavaScript (JS)

Advantages:

- JavaScript is very useful for programming a client-side interactions, with easy support for form submissions and dynamic content updates.
- There is a large community to support developers at all levels, and is just generally easy to learn for beginners.
- It can be used for front-end and back-end development, meaning 2 languages don't have to be learned for front and back-end.

Disadvantages:

- JavaScript is an interpreted language so may not have the same level of performance on the back-end compared to Kotlin.
- I don't have real experience in programming in JS so the development process would take longer than using Python.
- It doesn't support the full feature set of object-oriented programming with limited implementation of inheritance.

Python

Advantages:

- I am very familiar with Python and will have to spend less time learning the fundamentals of programming in Python.
- It supports the full range of features of object-oriented programming including inheritance which I intend to use.
- Python supports a wide range of libraries that can assist in implementing the features of my program.

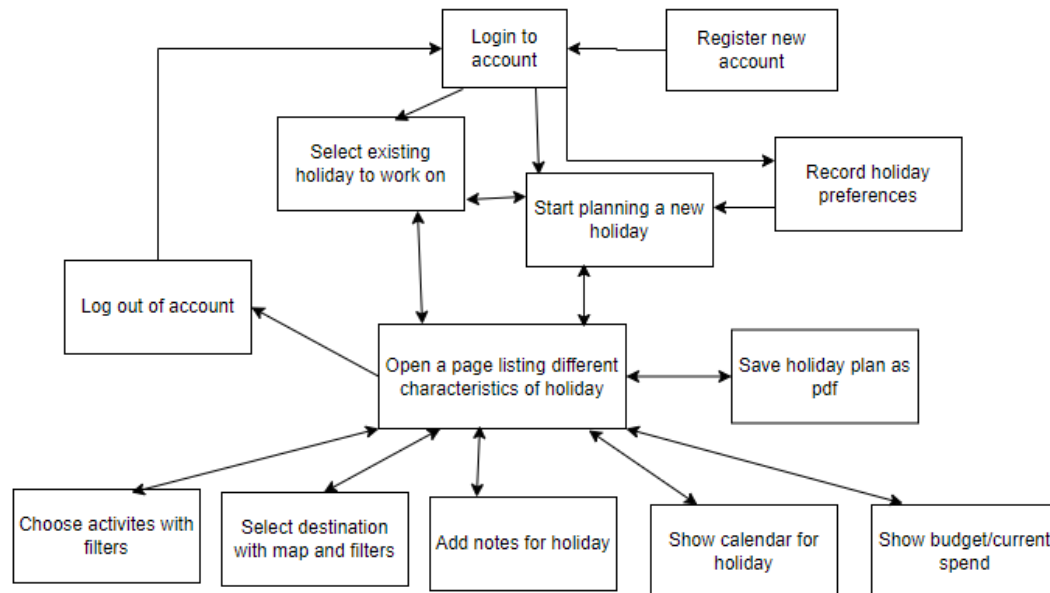
Disadvantages:

- Python doesn't have simple support for multithreading so can result in worse performance on lower-end hardware, where those CPUs have lower clock speeds and/or IPCs (instructions per clock).
- Python can also be seen as a slower language due to its nature of being an interpreted language meaning it can run slower than other compiled languages such as JavaScript.

I've chosen to use Python for programming the backend of this project as the stakeholder wants a focus for desktop users on the website, meaning the advantages that Kotlin has for Android aren't really useful for this project. Moreover, the original requirements from the stakeholder had little mention of performance requirements, where it was more important to have the website as a comprehensive guide to holiday planning which Python helps with through its wide support of databases. To add to this, as a beginner in web development, I don't want to stray too far from what I currently know so would rather stick to Python compared to JavaScript as I would have to learn less, and be able to spend more time on polishing the website so it offers the best solution for the stakeholder. Python also enables me to use Flask which is a back-end tool which is appropriate for lightweight applications

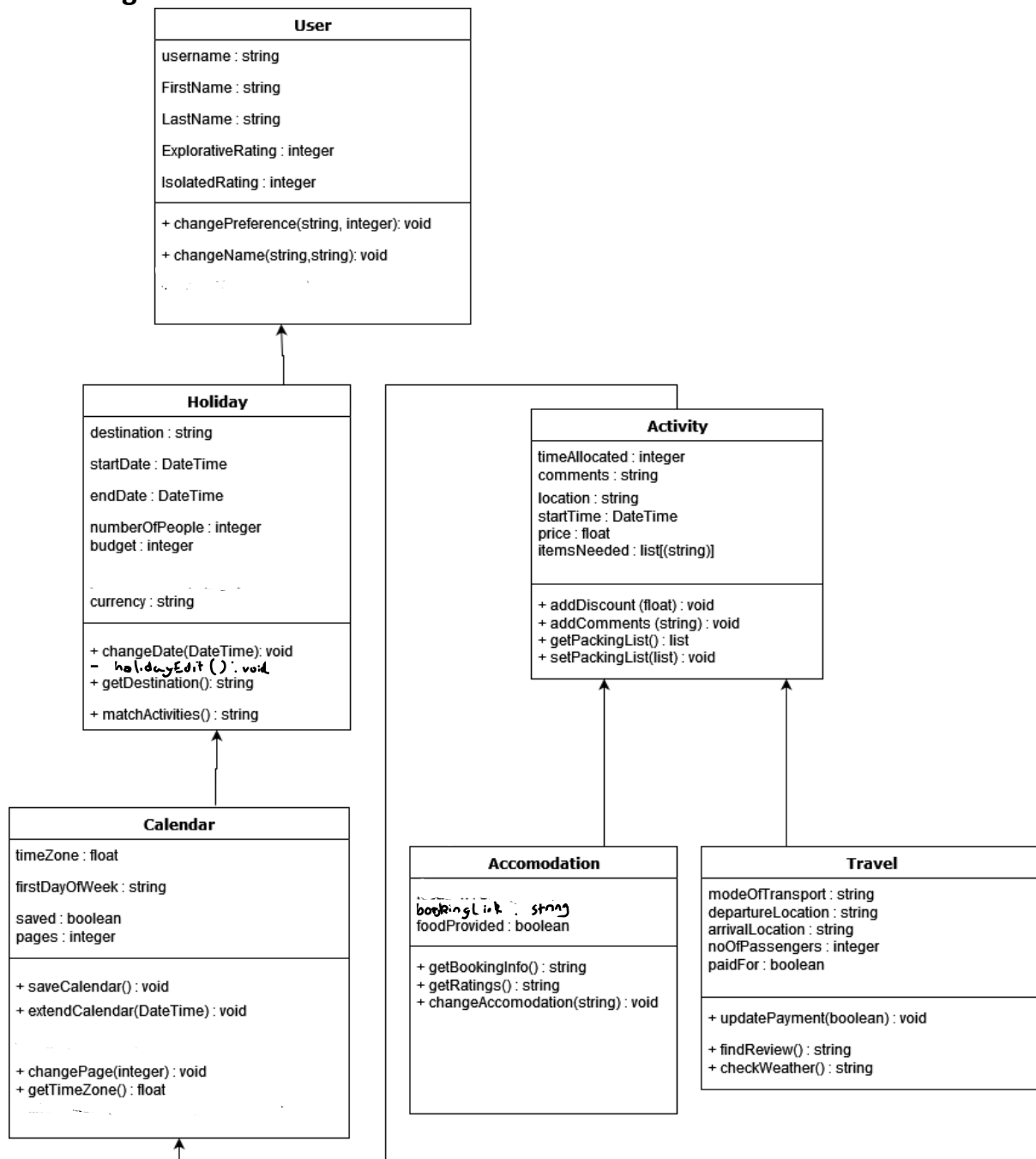
compared to Node.js which can be seen as more difficult to learn with its use of asynchronous programming and similarity to JavaScript.

Abstracted Solution



This shows the top-level structure diagram for this project, where bidirectional arrows indicate that the user can move between them as they wish, such as if the user clicked on the wrong holiday to edit, they can go back to reselect the holiday. This compares to the one-directional arrows that indicate that the user cannot go back once they move onto a later stage in the program. For example, the user cannot go back to editing their holidays if they have logged out.

This shows my use of decomposition when tackling the stakeholder's problem where the original idea of planning a holiday was initially separated into a log-in section, that made an initial layer behind the actual functionality of the program. Moreover, the order of what the user would do is considered here where the stakeholder wanted an appealing user interface that is easy to navigate, meaning that all of the buttons to plan a holiday shouldn't be on one screen. This led to the process being broken down into stages for the user as well as a programmer. These stages include creating a holiday, deciding on its destination, opening a calendar, and being able to take a step back to reconsider budget. This also allows an easier development where the more modular approach allows for easier debugging and maintainability where different parts of the website can stay operational if the part for the budget may have to temporarily undergo maintenance.

Class diagram:

The arrows indicate classes that inherit from another.

The user class represents what is created when someone logs in for the first time. It contains the user's name as well as preferences for future use when suggesting activities.

The holiday class contains the main details of the holiday that can be used to provide boundaries for the rest of the program such as the starting and ending dates. It also allows for a budget that can be followed for the rest of the program. This also contains a method where activities can be suggested based on the preferences as defined by the attributes previously stated.

The calendar class starts to build on the planning nature of the website, where it has a lot of time-based attributes to help the user establish some chronology in their holiday. The

methods it has such as `addPages()` helps to build on what is displayed to the user, where the calendar has pages and the program is able to determine whether everything would fit on one page or not. The `extendCalendar()` method also exists in order to check the new dates to see how many pages the holiday calendar should have based on the start and end dates, and the start of the week.

The activity class establishes a generic template for what the calendar can be filled with, where currently the accommodation and travel classes exist as examples for this. They each have their own attributes, but mainly their details can be found through their methods such as `getRatings()` and `findReview()` which would provide a link for the user to find more information.

Also, all locations that are used would be provided by What3Words and its API.

Pseudocode for initialising a holiday:

```
from datetime import datetime
```

```
Class Holiday inherits User
```

```
    private destination
```

```
    private startDate
```

```
    private endDate
```

```
    private numberOfPeople
```

```
    private budget
```

```
    private currency
```

```
    public procedure new()
```

```
        destination = userInput
```

```
        holidayEdit()
```

```
    endprocedure
```

```
    private procedure holidayEdit()
```

```
        startDate = input("Please enter start date in form of DD/MM/YYYY")
```

```
        endDate = input("Please enter end date in form of DD/MM/YYYY")
```

```
        numberOfPeople = integer input from a dropdown menu
```

```
        budget = input("Please enter your budget in GBP (will be converted later)")
```

```
        currency = input("Please enter the 3 letter letter currency code that you will require.
```

```
(If you don't know, please look at this website https://www.iban.com/currency-codes)")
```

```
        currencyCodes = ["AED","AFN","ALL","AMD",....."YER","ZAR","ZMW","ZWL"]
```

```
        if startDate < datetime.today() then
```

```
            startDate = datetime.today()
```

```
        endif
```

```
        if endDate < startDate then
```

```
            endDate = startDate + 1
```

```
        endif
```

```
        if budget < 0 then
```

```
            holidayEdit()
```

```
        endif
```

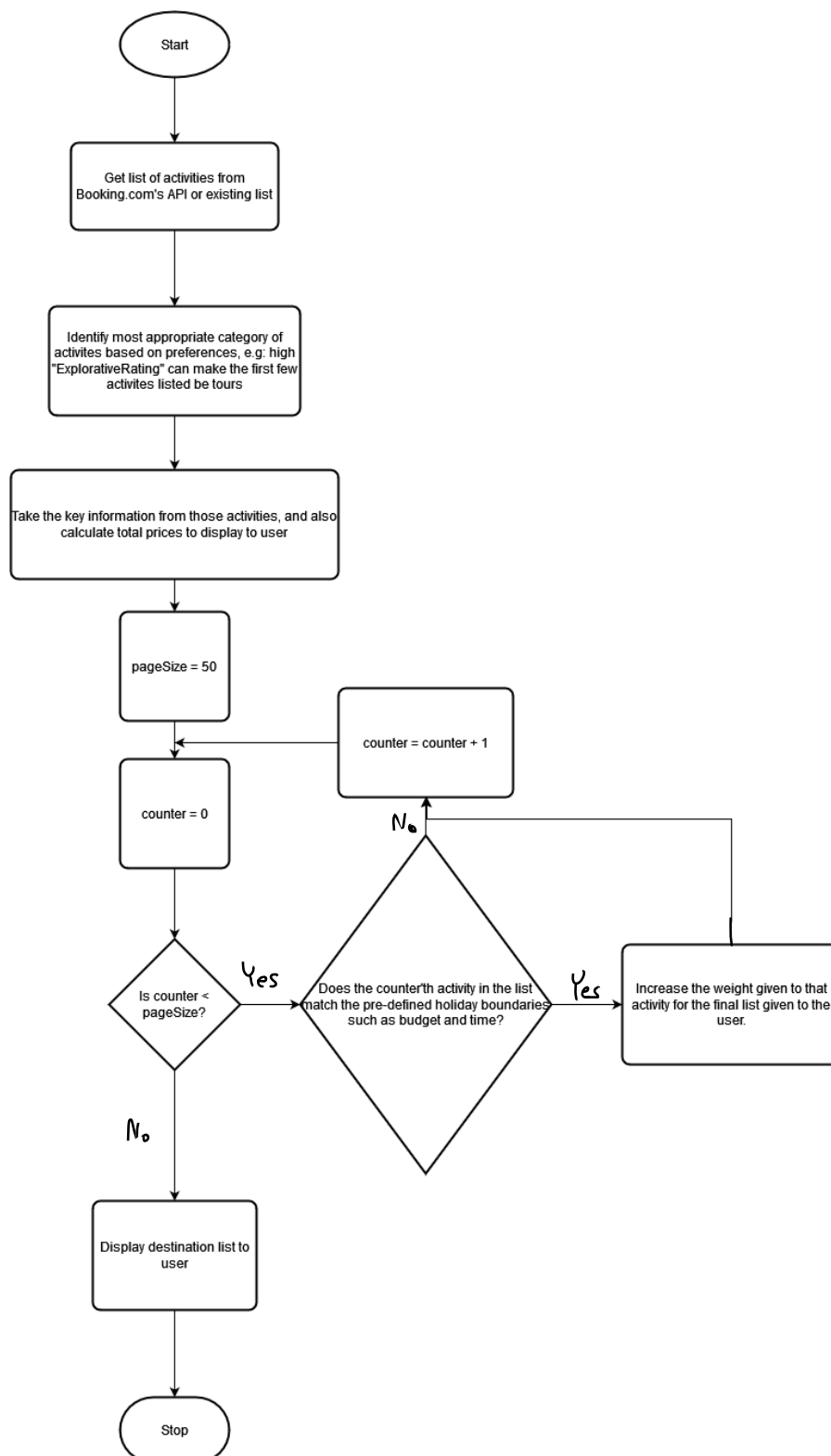
```
        if currency not in currencyCodes then
```

```
            currency = "GBP"
```

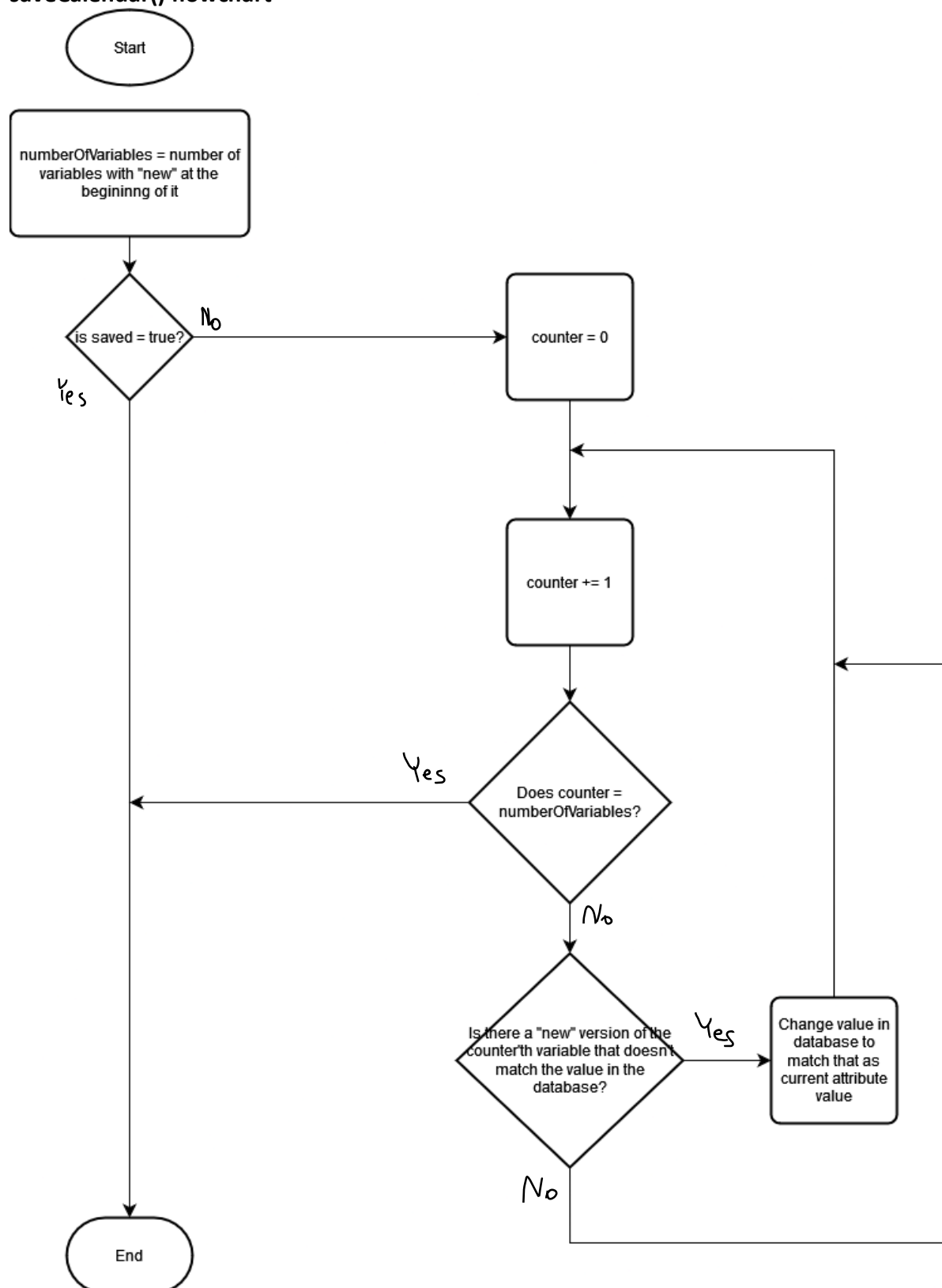
```
        endif
```

```
    endprocedure
```

```
endclass
```

matchActivities() flowchart:

This method allows the program to search for recommended activities for whenever the user requires a list of them, and uses a weight system based on the user's preferences to then put them in an order of appropriateness for the user.

saveCalendar() flowchart

The `saveCalendar()` method makes sure that all existing attributes match the values that are set for the user in the database. This changes the "saved" attribute back to true to allow the user to confirm their changes, but also allows restoration of changes if they hadn't been saved yet.

extendCalendar() pseudocode

```
from datetime import datetime
public procedure extendCalendar(newStartDate,newEndDate)
    if newStartDate < newEndDate then
        firstNoOfCalendar = newStartDate.isoweekday()
        if firstNoOfCalendar > firstDayOfWeek then
            pages = (newEndDate – newStartDate) DIV 7 + 1
        else
            pages = (newEndDate – newStartDate) DIV 7
        endDate = newEndDate
        startDate = newStartDate
        saved = false
    else
        break
    endif
endprocedure
```

This procedure first checks that the new dates are valid entries by making sure the newEndDate is after the newStartDate, and then uses the isoweekday() method from the datetime library which converts a date into an integer from 1 to 7 that is assigned based on the day of the week it falls on. This allows to see if part of a week will be shaved off (e.g a 7 column week timetable but the first 3 days are empty), and to add an extra page for the program to be able to access to account for the excess.

getBookingInfo() pseudocode:

```
public function getBookingInfo()
    return (location, bookingLink, foodProvided)
endfunction
```

This is a simple get function, utilising encapsulation to ensure that the booking information isn't accidentally modified, and can be given to the user or another child class when required.

checkWeather() pseudocode:

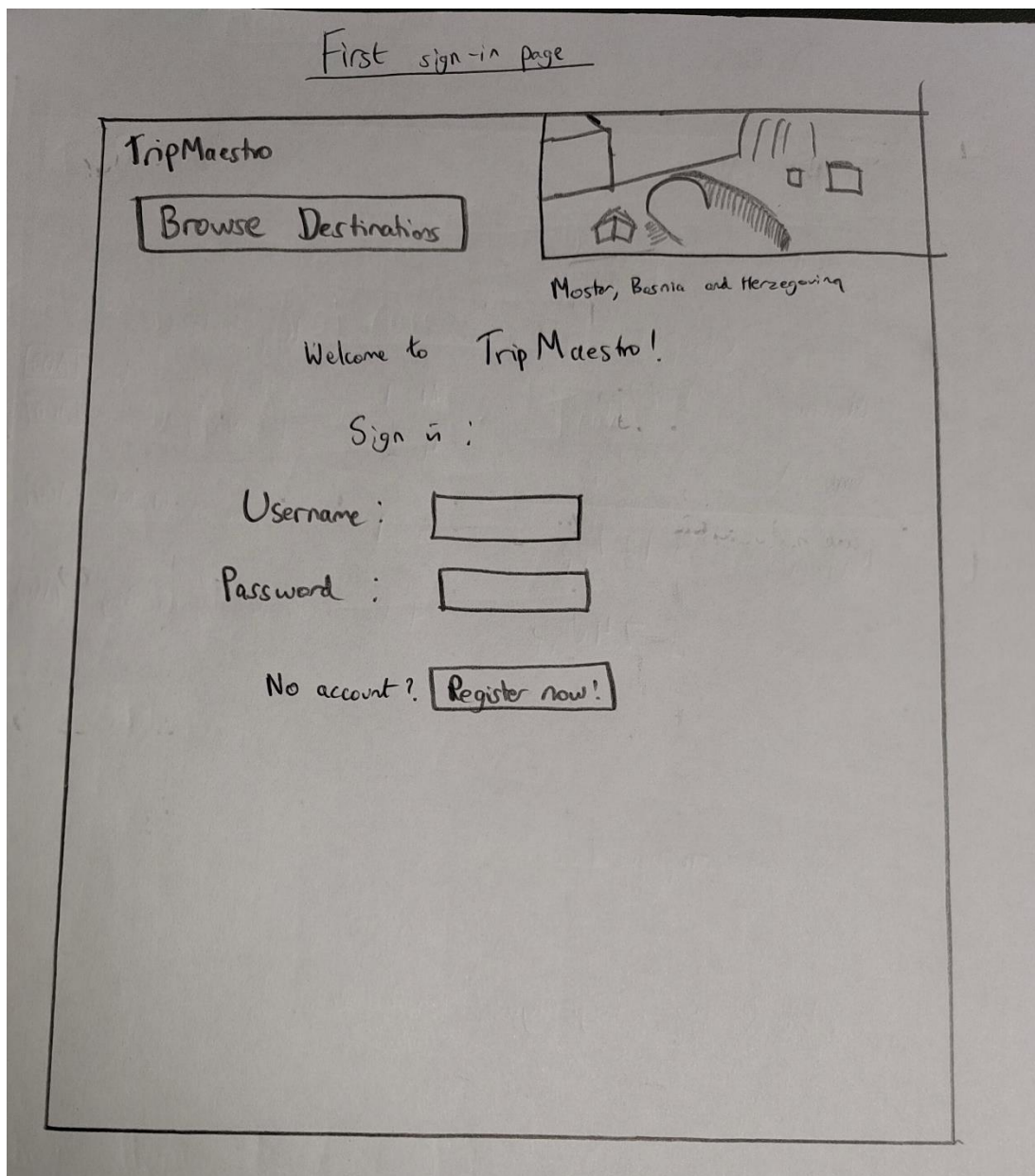
```
import requests
apiKey is generated uniquely for each user
public function checkWeather()
    url = "openweathermap.org/data/{0}&id={1}/metric".format(destination, apiKey)
    response = request(url)
    if response[0] = true then
        data = response[1]
        temperature = data['main']['temp']
        feels_like = data['main']['feels_like']
        humidity = data['main']['humidity']
        weather_description = data['weather'][0]['description']
        wind_speed = data['wind']['speed']
    else
        return "No weather update available"
    endif
```

```
finalString = "Temperature = {0}, it feels like {1} .....".format(temperature, feels_like ..)  
return finalString
```

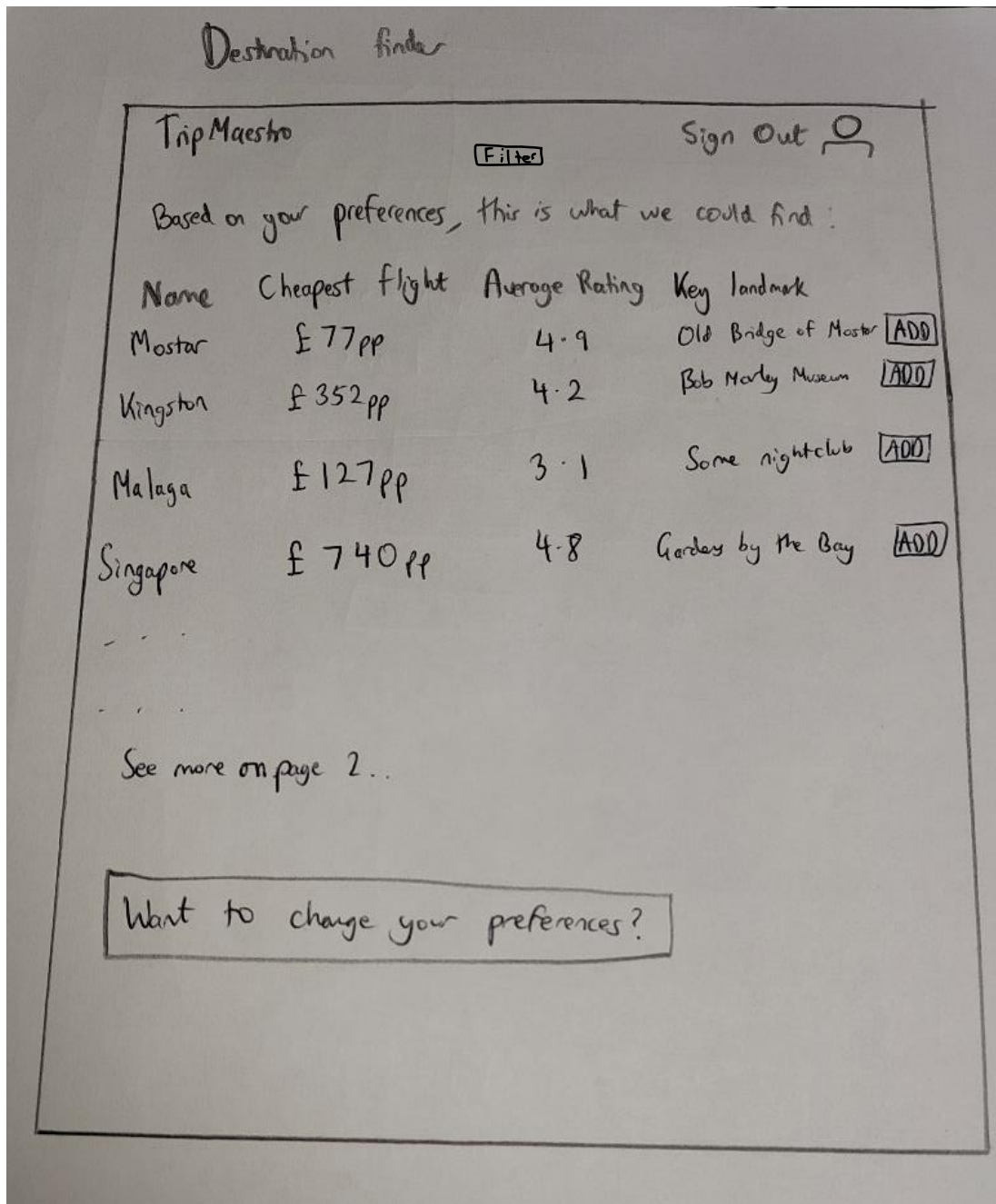
```
endfunction
```

This function takes an OpenWeatherMap API Key (which is generated when an account on the website is created) and requests the weather for the destination city at a given time. It returns a string that is appropriate to be displayed to the user containing key information that is taken from the 2D array that is taken from the API call.

GUI Designs:



This is an illustration of the GUI for the initial sign in page. It allows the user to enter their username and password, but also provides a link to making an account if they don't have one already.



This shows the list of destinations that is displayed to the user, which can adjust based on the holiday preferences that each user chooses when they make the account. It also has a list of key details for each destination, as well as a button to add that destination as the core of the holiday on the far right. A button is also offered to sign out, which would also represent the location of other settings that the user can modify. A filter button is also included in the top right to meet one of the essential features previously outlined where the user can be more specific on the requirements of the destination.

Activity editor

Trip Maestro

Sign out 

Go back to holiday list

Add activity:

Type:

Name:

Time allocated: hours minutes

Date: / / (DD, MM, YYYY)

Location:

Link to booking: N/A or

Notes:

Save

This is an illustration of the web page where the details of an activity can be added to a holiday. These generally match the attributes required from the "Activity" class, and also includes an area for notes (a resizable text box), meeting one of the essential features that was proposed earlier. It also holds enough data for it to be able to be dropped directly into the calendar, with its time, label, and basic information such as location and booking link for the user to open when they are actually on holiday. This also can have more details that have been ignored here such as details about food and other more niche considerations as would be necessary in order to meet the success criteria.

Holiday ~~list~~ calendar:

Trip Maestro
Sign out 

First day of the week: Monday (Default) ▾

Time zone: BST (UTC +0) ▾

Suggested Activity to add: Travel

	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
07:00							
07:30		Travel					
08:00		Jet ski					
08:30	Flight						
...							
21:00	Rest						
21:30							
22:00							
22:30							

Show 11pm → 7am ☐

This shows the base GUI for the holiday calendar. It is mainly composed of a table in the middle as a regular calendar with time slots for each day, but also has dropdown menus above to make sure time zones are synchronised throughout all menus to what the user requires. This also heavily relies on what was inputted previously with each activity where the start and end times are displayed as boxes that cross over multiple times if they take longer than the 30 minute intervals that are shown in the calendar.

Data dictionary

Attribute	Data type	Class it originates from	Description	Example
username	string	user	This is the username that is used for users to login. Its meant to be something easy to remember for the user to login, and would need to be a minimum of 3 characters and a max of 24 to keep it sensible. It would also only need letters and numbers to make sure it is easy to remember.	username = "BigJohn1"
firstName	string	user	The user's first name that can be used to give a name to the creator of different holidays when exporting as a pdf. It can help the recipient of such pdfs to understand who made them and what purpose they have. It can only be made of letters and dashes.	firstName = "Babar"
lastName	string	user	Same as last name, just for completeness	lastName = "Azam"
isLoggedIn	boolean	user	It is a variable that represents whether the user is logged in at any given time. This allows the program to restrict certain actions based on if they are logged in. For example, the user would be prevented from making a new holiday if they aren't logged in and this variable is false.	global isLoggedIn isLoggedIn = true
explorativeRating	integer	user	This is a rating that originates from the preferences that the user indicated when the account was created. This variable can only be integers from 1 to 5, which is determined based on the quantifier that the user picks when being asked questions about their holiday preferences.	explorativeRating = 3

isolatedRating	integer	user	Same as explorativeRating	isolatedRating = 5
destination	string	holiday	This is an attribute that indicates the destination that the holiday currently being worked on is for.	destination = "Islamabad"
startDate	datetime	holiday	This is a date attribute for the start of the holiday. It is inputted by the user in either a string format which would need to be converted, or more often a calendar that the user can click around, where the validation is done already through how it is inputted.	startDate = "2024-12-25T10:00"
endDate	datetime	holiday	Same as startDate but represents the end of the holiday	endDate = "2024-12-31T19:00"
newStartDate	datetime	holiday	This variable is what is used as a temporary store for the new starting date of the holiday whilst it undergoes validation. It also is stored here before the actual value in the database is replaced to confirm a save in the holiday.	newStartDate = "2025-10-19T10:00"
newEndDate	datetime	holiday	same as newEndDate for the end of the holiday	newEndDate = "2025-10-31T19:00"
numberOfPeople	integer	holiday	This attribute is defined when a holiday object is created, and is used to create limits for how many tickets the user is able to buy for whatever activity they use.	numberOfPeople = 2
budget	float	holiday	This attribute is used to meet one of the essential features of the website with its requirement for budget management. This allows the user to enter any number as 1135.50 should still be an allowed input instead of just allowing integers. However it does have to be rounded to two decimal places as that is the most decimals that modern currency extends	budget = 1125.5

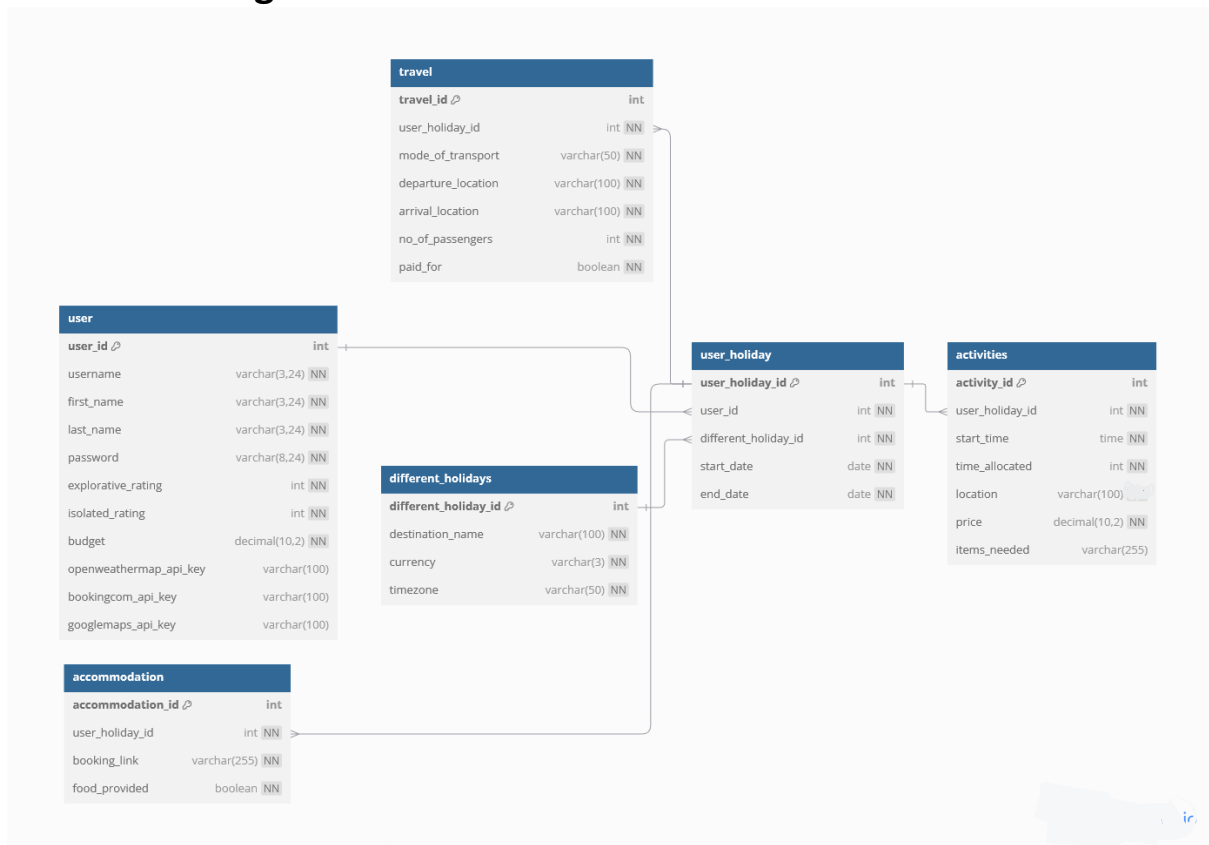
			towards, and allows for pounds and pence for example.	
currency	string	holiday	This is a 3 letter ISO 4217 currency code, which is unique for each currency in the world. The user can only enter a three character long string, and is auto-capitalised when the user's input is stored. This allows for prices to be converted so the user is aware of what they are spending and in what currency, helping to build on the key feature of the budget management.	currency = "EUR"
timeZone	string	calendar	This string represents the UTC offset that they want the calendar to be viewed in. It could be automatically determined from the destination, but is taken as a manual dropdown selection as you can't guarantee if the user wants the calendar to be viewed in the current time zone of the user, or their destination. This is automatically validated through how there is only a set number of options the user can select from through the dropdown menu so an invalid input cannot be entered.	timeZone = "+1"
firstDayOfWeek	integer	calendar	This is selected through a dropdown menu where the user picks what they want the first day of the week to display as on the calendar. This helps the user to fit more onto one page of the calendar, and also accounts for other calendars who may start their week on Sunday.	firstDayOfWeek = 7
saved	boolean	calendar	This is a Boolean attribute that indicates whether the changes made to the holiday are	saved = false

			synchronised with what is currently in the database. This flag will determine whether the “Save and Exit” button is usable on the calendar’s options, and also makes sure the user putting in one bad input isn’t made permanent and they can rectify it by noticing the mistake before hitting save.	
pages	integer	calendar	This is an integer calculated based on how many pages would be in the calendar, based on firstDayOfWeek, the startDate and endDate.	pages = 3
timeAllocated	int	activity	This is a value that shows how many minutes an activity will take. This is to be used in the calendar to show how much space the activity will take on the calendar. It has to be greater than 0, but can be as long as the rest of the holiday lasts for.	timeAllocated = 30
comments	string	activity	This is an attribute that holds the user-made comments that they may want to write about an activity. This meets one of the success criteria of the program where the user still has some freedom to add details to an activity that there isn’t a pre-existing gap-fill for.	comments = “trains will only be available after 3pm”
location	string	activity	This is a location that can be retrieved by the user to keep what they need in one place. It is stored in the format of What3Words which helps to uniquely identify the location, and can be passed into the Google Maps API for directions. The user can have the choice to enter an address which the website will convert using what3words for storage.	location = “career.dive.regard”

startTime	datetime	activity	This attribute represents the start of the holiday in Python in the datetime data type.	startTime = "2024-10-29T15:30"
price	float	activity	This attribute is a floating point number rounded to 2 decimal places representing the costs related to an activity. This is also meant to take into account other costs rather than just an immediate booking, such as having food after doing the activity.	price = 37.5
itemsNeeded	string	activity	This is an entirely user determined string where they are free to write what they want here, and can use it as a reminder for when the user actually carries out the holiday.	itemsNeeded = "food bag, coat if it rains, phone charger"
bookingLink	string	accommodation	This is a link that the user attaches to the activity and can provide the user with a pdf for booking details such as a QR code for a ticket to somewhere.	bookingLink = "https://outlook.office.com/mail/inbox/id/AAQkAGQ1YmEyMjJiL"
foodProvided	boolean	accommodation	This is a Boolean attribute that indicates whether the activity described has food come with it, such as a tour taking you to a restaurant along the way, or if you will have to make your own food arrangements.	foodProvided = false
modeOfTransport	string	travel	This is a string attribute for each instance of the "travel" class. The user manually types this in and is kept for the user to check over when they are revisiting their activities.	modeOfTransport = "bus"
departureLocation	string	travel	This is in the same format as the original "location" attribute, but is used specifically for the travel class for the user to specifically know where their journey starts from.	departureLocation = "cook.round.tube"
arrivalLocation	string	travel	Same as departureLocation but for where the travel will take them	arrivalLocation = "storm.cardboard.ohio"

noOfPassengers	integer	travel	This attribute indicates how many people will be on that mode of transport at a given time. It has to be less than numberOfPeople but a minimum of 1.	noOfPassengers = 4
paidFor	boolean	travel	This boolean attribute indicates whether money has to be kept to pay for the travel when you go on it, or if has already been paid for. This helps keep it accounted for in the budget where the user will know what still needs to be paid.	paidFor = true

Database design



This diagram represents the database that will be implemented for the website. The user's data is stored in the "user" table with the upper and lower limits of 3 and 24 characters implemented for all of the names. The password will not be encrypted as isn't directly required by the stakeholder. The first 8 fields in "user" is required to be "not null" as they are required for each account. "user_id" is used as a foreign key in the "user_holiday" table, which is used to assign each holiday to a user. The primary key of this table

“user_holiday_id” is used as foreign keys in the other three tables which each have their own fields that describe the different things about the key (that instance of accommodation). This is normalised to the 3rd normal form (3NF) as it meets all of the conditions for 1NF, 2NF and then 3NF. For example, there are no repeated fields, each table has a primary key, and each field is atomicised. The location address was difficult to atomicise, but relying on what3words has made this easy as the website doesn’t have to separate the address into postcode, house number and street road. It also meets the conditions for 2NF where there are no partial key dependencies as every field exists to describe the primary key of that table. There are also no transitive dependencies where there is no reliance of non-key fields on other non-key fields, meaning the database also meets the conditions for 3NF.

Iterative Test Plan

What stage of development is this in	Test purpose	What data can be used to test	Type of test data	Expected outcome
Sign up system (username,password)	User verification	“a” “b”	Invalid	User is told to re-enter username and password as both are meant to be at least 3 characters.
Sign up system (username,password)	User verification	“abcd” “abcd”	Normal	This is between 3 and 24 characters so should be accepted as a suitable name and password.
Sign up system (username,password)	User verification	“1234” “3456”	Normal	This test data is made entirely of numbers, but should still be accepted as a string as it is between 3 and 24 characters.
Sign up system (username,password)	User verification	“” “”	Invalid	User is told to re-enter both as they cannot be empty.
Preference recording	Profile completion	“likely” “very likely”	Normal	These would each be 1 of the 5 options there are for each statement, so all possible inputs would be normal. This is just to test that the user is allowed to continue from this point, which would be the expected outcome.
Holiday creation	Holiday validation	6/10/21 8/10/21	Invalid	This tests dates for a holiday that have

				already happened. The website shouldn't allow the user to carry on with these values, instead making the dates default to today's date and the day after for the start and end dates respectively.
Holiday creation	Holiday validation	25/10/24 (today's date) 7/11/24	Boundary	This tests a boundary date where the user should technically be allowed to plan a holiday for the same day. It is expected that this is allowed and the user is allowed to continue.
Holiday creation	Holiday validation	11/12/24 9/11/24	Invalid	This tests that the website doesn't accept the start date being after the end date, and the website should change the values on the website so that they default back to today's date and the day after that.
Holiday creation	Holiday validation	22/11/24 22/11/24	Invalid	This tests the start and end dates of the holiday being the same. This could be considered as reasonable, but with the stakeholder wanting to see its userbase be primarily of those looking for comprehensive advice, it would not seem reasonable for that to be restricted to one day. This means that the expected outcome is for the start date to remain on the selected date, and for the end date to go to the next day.

Activity editing (multiple tests for each input in a form)	Suitable characteristics described	Activity selected on drop down menu 125.50 for price The Shard, London for location 29/11/24 15:00 for startTime 30 for time allocated	Normal	This tests data that should be accepted by the program (so long as the start time is during the allocated holiday period. This input should allow the user to continue with creating their holiday.
Activity editing (multiple tests for each input in a form)	Suitable characteristics described	Activity selected on drop down menu 0 for price (null) for location startDate (whatever user has it set to) as startTime endDate – startDate as time allocated	Boundary	These inputs represent the boundary for each attribute that they are all tied to. The price is allowed to be 0 but no less, location is allowed to be an empty field so that has been designated as the boundary. The boundary for the starting time of the activity would be the start of the holiday so that is what would be tested there. The lower limit would be 0, but for a boundary result that should still work, the upper limit can be tested here which is the entire length of the holiday (endDate – startDate). These all combined should return an acceptable output where these are all technically suitable for what is required for the website.
Activity editing (multiple tests for each input in a form)	Suitable characteristics described	Activity selected on drop down menu 10 for price	Invalid	The start date would have to be in a date time format, so a string like "John" would not be allowed. This input should be rejected, and

		“The Shard, London” for location startTime as “John” 30 as time allocated		the website should correct it to the default time, which is the start of the holiday if the user doesn’t correct this before the activity is saved.
Activity editing (multiple tests for each input in a form)	Suitable characteristics described	Accommodation selected on drop down menu 10 for price “The Shard, London” for location startTime as 27/11/24 21:00 30 as time allocated (null) as bookingLink “yes” for foodProvided	Invalid	All inputs are valid apart from the one for foodProvided as that is a boolean attribute so needs to be true or false instead of the current “yes”
Activity editing (multiple tests for each input in a form)	Suitable characteristics described	Travel selected on drop down menu 10 for price “The Shard, London” for location startTime as 27/11/24 21:00 30 as time allocated “bus” as modeOfTransport departureLocation as “The Shard, London” arrivalLocation as “The Shard, London” 3 as noOfPassengers true as paidFor	Normal	All inputs are valid so the website should continue to allow the user to put this travel activity onto their holiday calendar.
Activity editing (multiple tests for each input in a form)	Suitable characteristics described	Travel selected on drop down menu 10 for price	Invalid (Empty)	All inputs are valid apart from the mode of transport not having an input. This shouldn’t be

		“The Shard, London” for location startTime as 27/11/24 21:00 30 as time allocated (null) as modeOfTransport departureLocation as “The Shard, London” arrivalLocation as “The Shard, London” 3 as noOfPassengers true as paidFor		allowed as else this activity doesn't represent anything, so the user should be prompted to fill in their form again to correct themselves.
Calendar editing	Test page movement	Click on the next page (given holiday isn't longer than a week, and starting date is on the same day as the firstDayOfWeek attribute)	Invalid	The calendar shouldn't move to the next week as the holiday doesn't extend to that next week. It should not have any response.
Calendar editing	Test page movement	Click on the next page (and holiday is longer than a few weeks)	Normal	Changing the page in the calendar should open up a new week with time slots to be filled in/view.
Calendar editing	Check night time slots	Click on the box marked “Show 11pm to 7am time slots”	Normal	It should extend the calendar vertically to show the time slots that were previously hidden between 11pm and 7am where users are less likely to do planned activities during then.
Budget manager	Check synchronisation of budget	Change the budget for one of the activities so that the budget is still ≥ 0	Normal	This should change the page with the budget manager so that it now displays a reduced budget, with a visual display to show a chunk

				being taken up by this activity.
Budget manager	Check validity of budget	Change the budget for one of the activities so that the budget is < 0	Invalid	This would mean the user has a negative budget left which is obviously not suitable, so the budget page should raise red flags for user to correct in the budget menu.

Post development test plan

This would take into account the website as a whole to test the program's execution in its entirety. All of the drop down menu inputs have been ignored as they can only give suitable values and the user cannot provide something that wouldn't work in the program.

Testing section	Test Purpose	Test Data	Expected Outcome
Sign up system	Test that the username and password boundaries work	Valid – usernames between 3 and 24 characters are accepted so long as password is also between 8 and 24 characters Invalid – usernames are not between 3 and 24 characters, or password are not between 8 and 24 characters	The website accepts the valid inputs and the user continues the sign up process onto the selection of preferences. The invalid inputs should not move the user on, instead just raising a red flag prompting the user to re-enter their username and password.
Sign in system	Test that only already-registered accounts are accepted	Valid – usernames and passwords that match the one in the database Invalid – login credentials are incorrect	The website allows the user to proceed with valid inputs, with all of the holidays kept from previous work on the website available. An invalid input should reject the user from proceeding with the website until they enter correct information, or are

			only using the sections of the website that do not require a log in such as browsing destinations.
Destination list	Test that the list of destinations is appropriate based on the user's preferences	Entering a preference for a holiday that is easy to follow compared to one that is based on discovery	The test with an emphasis on discovery should display locations with different cultures and lots of niche places to discover, whereas the test with preferences inputted of tight plans should display cities with a common tourist path such as London.
Interactive map	Test functionality of map displayed next to the website in the activity menu	Invalid – no location entered so map cannot display anything Valid – location entered	The valid data should see the map automatically mark that location on it, and zoom into that area at an appropriate length so the user can see the area around it.
Holiday individuality	Test that changes made to one website doesn't affect another	Make two holidays with identical characteristics such as destination and the time that it will start. Then edit one thing minor such as the time of one activity.	When you go back to select your holiday, it should still have 2 holidays that can be selected from, and selecting on either should show that they were in the state that they were left in, and that nothing was overwritten.
Boundary time zone	Test that an extreme time zone still organises dates properly	Create a holiday with UTC -12 offset or UTC +14 and plan travel to there and activities after.	The correct result is that the date of travel is before the time of landing, and that it doesn't jump

			or go back a day where the website doesn't understand the chronology of events. This is important as otherwise it wouldn't be possible to plan anything due to the times not being in a sensible order.
--	--	--	---

It can be argued that this website doesn't need to rely on post-development testing as much as the iterative testing as most of the work done by the code contributes to the modification of the user_holiday table, meaning that so long as data there is modified and retrieved correctly (which can be tested in the iterative stage), it is very likely to be fully operational when the website is tested in its entirety. However, that shouldn't understate the importance of post development testing as there are still many things that can go wrong and likely haven't been considered before putting all of the individually made classes together.

Development

The first part of the success criteria I wanted to develop was the login system along with the basis of the user preferences. I began with the HTML for the form for username and password, as well as the general CSS formatting that can be used across the webpages.

```
<html>
<head>
  <link rel="stylesheet" href="styles.css">
  <title>Login - TripMaestro</title>
</head>
<body>
  <h1>Welcome to TripMaestro</h1>
  <form action="/login" method="post">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>
    <br>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>
    <br>
    <button type="submit">Login</button>
  </form>
  <p>Don't have an account? <a href="/register">Sign up</a></p>
</body>
</html>
```

```
1  * {
2    margin: 0;
3    padding: 0;
4    box-sizing: border-box;
5    font-family: Arial, sans-serif;
6  }
7
8  body {
9    background-color: #f5f5f5;
10   color: #333;
11   line-height: 1.6;
12   display: flex;
13   justify-content: center;
14   align-items: center;
15   min-height: 100vh;
16   margin: 0;
17 }
18
19 h1 {
20   color: #444;
21   margin-bottom: 20px;
22   font-size: 2rem;
23   text-align: center;
24 }
25
26 form {
27   background: #fff;
28   padding: 20px;
29   border-radius: 8px;
30   box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
31   width: 100%;
32   max-width: 400px;
33 }
34
35 label {
36   display: block;
37   margin-bottom: 8px;
38   font-weight: bold;
39 }
```

```
39 }
40
41 input {
42   width: 100%;
43   padding: 10px;
44   margin-bottom: 15px;
45   border: 1px solid #ddd;
46   border-radius: 4px;
47   font-size: 1rem;
48 }
49
50 button {
51   width: 100%;
52   padding: 10px;
53   background-color: #007bff;
54   color: #fff;
55   border: none;
56   border-radius: 4px;
57   cursor: pointer;
58   font-size: 1rem;
59 }
60
61 button:hover {
62   background-color: #0056b3;
63 }
64
65 p {
66   margin-top: 15px;
67   text-align: center;
68 }
69
70 a {
71   color: #007bff;
72   text-decoration: none;
73 }
74
75 a:hover {
76   text-decoration: underline;
77 }
78
```

This creates a base style format for all the pages to use, and makes a form posting data for a main python file to access.

```
<html>
<head>
  <link rel="stylesheet" href="styles.css">
  <title>Register - TripMaestro</title>
</head>
<body>
  <h1>Create Your TripMaestro Account</h1>
  <form action="/register" method="post">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>
    <br>
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
    <br>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>
    <br>
    <label for="confirm_password">Confirm Password:</label>
    <input type="password" id="confirm_password" name="confirm_password" required>
    <br>
    <button type="submit">Register</button>
  </form>
  <p>Already have an account? <a href="/login">Login here</a></p>
</body>
</html>
```

This HTML gives a page for users to register.

I then went into SQLite Browser to make my database for the users, which includes usernames, passwords, ids, as well as ratings that indicate preferences, and that hasn't been defined as NOT NULL as this will be updated later and will enable it to still work in the meanwhile. The following SQL was used:

```
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL,
  explorativeRating INTEGER DEFAULT 0 CHECK(explorativeRating BETWEEN 1
AND 10),
  isolatedRating INTEGER DEFAULT 0 CHECK(isolatedRating BETWEEN 1 AND
10),
  uniqueCuisineRating INTEGER DEFAULT 0 CHECK(uniqueCuisineRating
BETWEEN 1 AND 10)
);
```

How the database is created here does a lot of the validation for the program, such as the username being unique to avoid being able to replace another user. Moreover, username and password being not null covers problems that could arise in a future HTML form where they are left blank. The ratings being defined as being between 1 and 10 also makes it much easier to change the value during the main body of code as this doesn't have to be checked every time as being between 1 and 10.

```

1  from flask import Flask, render_template, request, redirect, url_for, flash, session
2  import sqlite3
3
4  app = Flask(__name__)
5  app.secret_key = 'TemporaryFlaskKey123'
6
7  @app.route('/') #Run for a blank URL, e.g: www.tripmaestro.com/
8  def baseRoute():
9      return render_template('login.html') #Returns user to login page
10
11 @app.route('/login', methods=['GET', 'POST']) #Run for what would be www.tripmaestro.com/login
12 #These specify what HTTP methods this route can accept, GET meaning it can request data, and POST meaning it can send data for processing
13
14 def login():
15     if request.method == 'POST': #Checks that a form has actually been submitted where data has been sent, rather than just changing the URL
16         username = request.form['username'] #Defines 'username' with what was inputted in with the label of 'username' in the form
17         password = request.form['password'] #Same as above but for the variable 'password' and the label of 'password' in the HTML
18         connection = sqlite3.connect('users.db') #Creates an instance of the database
19         cursor = connection.cursor() #Creates a cursor that the program can use to interact with the database
20         cursor.execute(f"SELECT * FROM users WHERE username = '{username}'") #Gets all of the data from a matching username in the database
21         user = cursor.fetchone() #This takes a record from the database and assigns it to user.
22         connection.close() #Ends connection between program and database
23
24         if user and user[2] == password: #Checks if user exists and inputted password matches what is in the database
25             session['user_id'] = user[0] #Takes ID from database and enters it into the session to make future requests easier
26             session['username'] = user[1] #Does same thing with username as some things are easier to request with username
27             flash('Login successful!', 'success') #Gives pop-up to the user saying successful login
28             return redirect(url_for('dashboard')) #Redirects user to their dashboard (www.tripmaestro.com/dashboard) now logged in
29         else:
30             flash('Invalid username or password. Please try again.') #Password doesn't match, that is indicated to the user with a pop-up
31
32     return render_template('login.html') #Makes a default point for if the URL was typed in manually for www.../login or another error
33

```

The first part of my app.py imports Flask and its various modules, which individually enable the program's ability to open a new webpage, get data inputted into a webpage and provide pop-ups to give messages to the user.

I then have (in lines 4 and 5) created an instance of the Flask application, and created a secret key to sign session cookies as a step to safety.

Line 7 to 9 establish a base route for the website to resort to as a default page, that being the original login page.

Those mentioned above are not particularly unique to my website and consequently are very difficult to unit test.

The login function checks to see if the username and password matches what is in the database. If it matches, it creates a session for the user with their relevant info, and takes them to the dashboard now logged in.

I have tested that the webpage works displaying login.html, however a small issue arose where the styling didn't display on the page. This turned out to be an issue with how I referenced the CSS file, where Flask expected it in a 'static' folder, and for the link to be written like: `<link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">`

To test this part of the code, I first had to create a record in the database of a hypothetical user. That was achieved with the following SQL:

INSERT INTO users (id, username, password, explorativeRating, isolatedRating, uniqueCuisineRating)

VALUES (1, 'testname123', 'testpassword123', 3, 5, 4);

id	username	password	explorativeRating	isolatedRating	uniqueCuisineRating
Filter	Filter	Filter	Filter	Filter	Filter
1	testname123	testpassword123	3	5	4

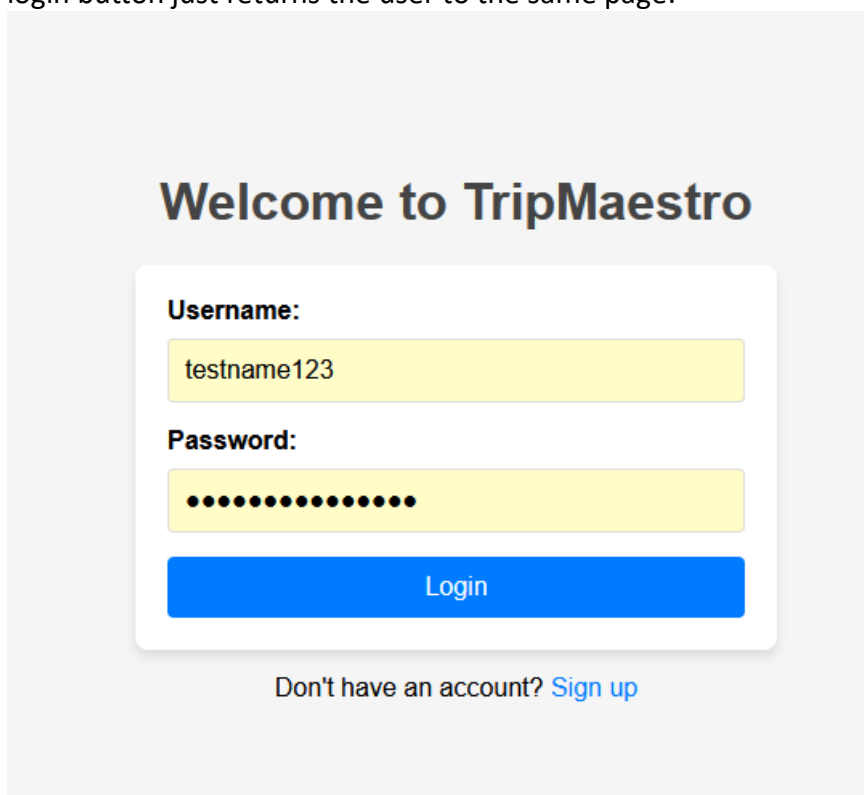
I also had to create a blank dashboard.html file that will open if the login is successful.

```
<html>
<body>
  <h1>dashboard test</h1>
</body>
</html>
```

To make this accessible through the Python code, this also had to have a route created for it with the following:

```
@app.route('/dashboard', methods=['GET', 'POST'])
def dashboard():
    return render_template('dashboard.html')
```

This test was unsuccessful as when typing in the test username and password, pressing the login button just returns the user to the same page:



That was because I didn't save the database file, because after it was it showed me the test page that I expected.

dashboard test

Above image taken from web page, outlined in red

These form boxes don't particularly require validation, as users are able to write any form of characters for their username and password. However, this poses the problem of SQL injection, which means input sanitisation is required as otherwise the data in the database can be compromised. This can cause a major problem for the stakeholders where they do not follow the Data Protection Act if measures are not taken to protect the data of users who use the website.

This was fixed by modifying line 20 in app.py in order to query using a parameter rather than string interpolation:

```
14 def login():
15     if request.method == 'POST': #Checks that a form has actually been submitted where data has been sent, rather than just changing the URL
16         username = request.form['username'] #Defines 'username' with what was inputted in with the label of 'username' in the form
17         password = request.form['password'] #Same as above but for the variable 'password' and the label of 'password' in the HTML
18         connection = sqlite3.connect('users.db') #Creates an instance of the database
19         cursor = connection.cursor() #Creates a cursor that the program can use to interact with the database
20         cursor.execute("SELECT * FROM users WHERE username = ?", (username,)) #Gets all of the data from a matching username in the database
21         user = cursor.fetchone() #This takes a record from the database and assigns it to user
22         connection.close() #Ends connection between program and database
```

The test executed earlier was still successful, but now was safe from SQL injections.

I then began working on a registration system for the user to be able to create an account and used the following HTML to create the base webpage.

```
1 <html>
2 <head>
3     <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
4     <title>Register - TripMaestro</title>
5 </head>
6 <body>
7     <h1>Create Your TripMaestro Account</h1>
8     <form action="/register" method="post">
9         <label for="username">Username:</label>
10        <input type="text" id="username" name="username" required>
11        <br>
12        <label for="password">Password:</label>
13        <input type="password" id="password" name="password" required>
14        <br>
15        <label for="confirm_password">Confirm Password:</label>
16        <input type="password" id="confirm_password" name="confirm_password" required>
17        <br>
18        <button type="submit">Register</button>
19    </form>
20    <p>Already have an account? <a href="/login">Login here</a></p>
21 </body>
22 </html>
```

I then made the following Python code to handle the registration process, with similar processes to the login system regarding the handling of the database connection. However, registration required the "INSERT" command as new entries are being added to the database.

```
34 # Register route
35 @app.route('/register', methods=['GET', 'POST']) #Enables GET and POST methods for Python to send and receive data from the webpage
36 def register():
37     if request.method == 'POST': #Checks that data has been received
38         username = request.form['username'] #Defines username with what was entered in 'username' on the webpage
39         password = request.form['password'] #Defines password with what was entered in 'password' on the webpage
40         confirm_password = request.form['confirm_password'] #Defines confirm_password with what was entered as the 2nd password
41
42         if password != confirm_password: #Checks that the passwords are the same
43             flash('Passwords do not match!') #Sends a pop-up to the user saying passwords do not match
44             return redirect(url_for('register')) #Sends the user back to the registration page to try again
45
46         connection = sqlite3.connect('users.db') #Same database connection as login system
47         cursor = connection.cursor()
48         cursor.execute(f"INSERT INTO users (username, password) VALUES ({username}, {password})") #Uses SQL to add new user to database
49         connection.commit()
50         connection.close()
51         flash('Registration successful! Please log in.', 'success') #Informs user registration was successful
52         return redirect(url_for('login')) #Takes user to login
53
54     return render_template('register.html') #If webpage opened directly (with no form filled in), user redirected to that webpage
```

I tested this part of the code by first checking the hyperlink on the login page works. This directed me to the registration page as expected, and then I filled in the form and clicked register. To ensure I tested this unit individually, I checked that this worked by opening the database directly to see if a new record was made. If I tested using the original login page, then there would be more things that could go wrong and harder to identify a bug.

OperationalError

sqlite3.OperationalError: database is locked

Traceback (most recent call last)

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in __call__

My first test of this showed an "OperationalError" where the database was locked. This was due to me keeping the database open in an SQLite window in the background so I closed the database through that and tried again.

My next attempt, after filling in the registration form directed me to the login page as expected, and checking the database file showed me this.

	id	username	password	explorativeRating	isolatedRating	uniqueCuisineRating
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	testname123	testpassword123	3	5	4

This result was unexpected as this record was entered much earlier in the development of the program and not in the form used in the test.

I hypothesised that this was because my Python code didn't make a complete record filling in the other fields of the database, rather just adding a username and password each time. I tested this by changing line 48 of the above code to test some placeholder values:

```

45 connection = sqlite.connect('users.db') #Same database connection as login system
46 cursor = connection.cursor()
47 cursor.execute(f"INSERT INTO users (username, password, explorativeRating, isolatedRating, uniqueCuisineRating) VALUES ('{username}', '{password}', (5), (5), (5))") #Uses SQL to add new user to database
48 connection.commit()
49 connection.close()

```

When I tested this the database updated appropriately as shown in the image below:

	id	username	password	explorativeRating	isolatedRating	uniqueCuisineRating
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	testname123	testpassword123	3	5	4
2	2	abc123	abc123	5	5	5

To make this change permanent across the program, I had to first edit the HTML to take in inputs for the ratings during the registration process.

A dropdown menu is to be used, which in itself validates the user input as there are only as many possible inputs as there are options in the dropdown.

The following is the updated register.html that includes a dropdown selection menu for the user to select their own ratings:

```

1 <html>
2 <head>
3   <title>Register for TripMaestro</title>
4 </head>
5 <body>
6   <h1>Register</h1>
7   <form action="{url_for('register')}" method="POST">
8     <label for="username">Username:</label>
9     <input type="text" id="username" name="username" required>
10
11     <label for="password">Password:</label>
12     <input type="password" id="password" name="password" required>
13
14     <label for="confirm_password">Confirm Password:</label>
15     <input type="password" id="confirm_password" name="confirm_password" required>
16
17     <label for="explorativeRating">How much do you prioritise exploring the area in your holiday?</label>
18     <select id="explorativeRating" name="explorativeRating" required>
19       <option value="1">1</option>
20       <option value="2">2</option>
21       <option value="3">3</option>
22       <option value="4">4</option>
23       <option value="5">5</option>
24       <option value="6">6</option>
25       <option value="7">7</option>
26       <option value="8">8</option>
27       <option value="9">9</option>
28       <option value="10">10</option>
29     </select>
30
31     <label for="isolatedRating">How much do you prioritise a peaceful, isolated area?</label>
32     <select id="isolatedRating" name="isolatedRating" required>
33       <option value="1">1</option>
34       <option value="2">2</option>

```

```
31 <label for="isolatedRating">How much do you prioritise a peaceful, isolated area?</label>
32 <select id="isolatedRating" name="isolatedRating" required>
33   <option value="1">1</option>
34   <option value="2">2</option>
35   <option value="3">3</option>
36   <option value="4">4</option>
37   <option value="5">5</option>
38   <option value="6">6</option>
39   <option value="7">7</option>
40   <option value="8">8</option>
41   <option value="9">9</option>
42   <option value="10">10</option>
43 </select>
44
45 <label for="uniqueCuisineRating">How much do you value a unique cuisine?</label>
46 <select id="uniqueCuisineRating" name="uniqueCuisineRating" required>
47   <option value="1">1</option>
48   <option value="2">2</option>
49   <option value="3">3</option>
50   <option value="4">4</option>
51   <option value="5">5</option>
52   <option value="6">6</option>
53   <option value="7">7</option>
54   <option value="8">8</option>
55   <option value="9">9</option>
56   <option value="10">10</option>
57 </select>
58
59 <button type="submit">Register</button>
60 </form>
61 </body>
```

When loading the registration page, the dropdowns and questions appeared as expected, although I added line breaks between the dropdowns and the next questions to help the aesthetic.

The image displays two versions of a web registration form. The left version shows the form as it appears when loaded, with fields for Username, Password, Confirm Password, and three rating questions. The right version shows the form with a dropdown menu open for the first question, 'How much do you prioritise exploring the area in your holiday?'. The dropdown menu lists options from 1 to 10. The 'Register' button is visible at the bottom of the form in both versions.

Register

Username:
testname123

Password:
.....

Confirm Password:

How much do you prioritise exploring the area in your holiday?
1 ▾

How much do you prioritise a peaceful, isolated area?
1 ▾

How much do you value a unique cuisine?
1 ▾

Register

Username:
testname123

Password:
.....

Confirm Password:

How much do you prioritise exploring the area in your holiday?
1 ▾

How much do you prioritise a peaceful, isolated area?
1 ▾

How much do you value a unique cuisine?
1 ▾

Register

To make the Python accommodate for this, I added new variables that took its value from what was added in the HTML. I also replaced the placeholder value of “5” from page 44 with the actual values.

```
username = request.form['username'] #Defines username with what was entered in 'username' on the webpage
password = request.form['password'] #Defines password with what was entered in 'password' on the webpage
confirm_password = request.form['confirm_password'] #Defines confirm_password with what was entered as the 2nd password
explorative_rating = request.form['explorativeRating'] #Defines explorative_rating with value from webpage
isolated_rating = request.form['isolatedRating'] #Defines isolated_rating with value from webpage
unique_cuisine_rating = request.form['uniqueCuisineRating'] #Defines unique_cuisine_rating with value from webpage

if password != confirm_password: #Checks that the passwords are the same
    flash('Passwords do not match!') #Sends a pop-up to the user saying passwords do not match
    return redirect(url_for('register')) #Sends the user back to the registration page to try again
connection = sqlite3.connect('users.db') #Same database connection as login system
cursor = connection.cursor()
cursor.execute(f"INSERT INTO users (username, password, explorativeRating, isolatedRating, uniqueCuisineRating) VALUES ('{username}', '{password}', {explorative_rating}, {isolated_rating}, {unique_cuisine_rating})")
connection.commit()
connection.close()
flash('Registration successful! Please log in.', 'success') #Informs user registration was successful
return redirect(url_for('login')) #Takes user to login
```

When testing this, the database updated to add the new record (I manually deleted the old ones for clarity) as expected.

	id	username	password	explorativeRating	isolatedRating	uniqueCuisineRating
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	aaa123	aaapass123	10	2	3

To further test the existing functionality, only the username and password can have a range of values as the ratings can only be selected from a dropdown menu.

When testing empty values for username and password during registration, no error occurs and the user's cursor is put into the username textbox.

Special characters when tested are accepted in the form, but this is reasonable as passwords should have them to be secure. They are also reasonable for usernames as there is no real disadvantage to having them, and has the benefit of being usable in different character scripts such as Cyrillic which can help keep the user base of the website broad at the benefit of the stakeholder who wants to maximise traffic through the website.

When testing a repeated value for username (which is meant to be unique) an error appears with an “sqlite3.IntegrityError” where the unique condition defined in the original database isn't met. This can be a large problem for the stakeholder as two users may want the same username and they would have no way of knowing that theirs isn't available if they only see the below error. The error is shown below:

IntegrityError

```
sqlite3.IntegrityError: UNIQUE constraint failed: users.username
```

Traceback (most recent call last)

```
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in __call__
```

```
    return self.wsgi_app(environ, start_response)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1514, in wsgi_app
```

```
    response = self.handle_exception(e)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

To remedy this, a try and except can be used that corresponds to this error that prompts the user that the username is unavailable, and gives them an opportunity to try again.

```

48 try:
49     connection = sqlite3.connect('users.db') #Same database connection as login system
50     cursor = connection.cursor()
51     cursor.execute(f"INSERT INTO users (username, password, explorativeRating, isolatedRating, uniqueCuisineRating) VALUES ({username}, {password}, {explorativeRating}, {isolatedRating}, {uniqueCuisineRating})")
52     connection.commit()
53     connection.close()
54     flash('Registration successful! Please log in.', 'success') #Informs user registration was successful
55     return redirect(url_for('login')) #Takes user to login
56 except sqlite3.IntegrityError: #Considers the error of a duplicate username
57     flash('Username already exists. Please choose a different one.')

```

When trying this again, the page is reloaded, but there is no pop-up to indicate to the user that the username is unavailable. This arises from how the flash message isn't referred to in the HTML file so doesn't know how to deal with the message. A solution to this is to add a container for pop-ups using the Flask function `get_flashed_messages()` and its implementation in `register.html` is shown below:

```

6 <body>
7     <h1>Register</h1>
8
9     <div>
10         {% for message in get_flashed_messages() %}
11             <div>{{ message }}</div>
12         {% endfor %}
13     </div>
14
15     <form action="{{ url_for('register') }}" method="POST">
16         <label for="username">Username:</label>
17         <input type="text" id="username" name="username" required>
18

```

Now when the username is identified as already existing in the database, the "Username already exists ..." message appears above the form to indicate to the user their issue.

The screenshot shows a web page titled "Register". At the top, there is a message: "Username already exists. Please choose a different one." Below this message is a form with three input fields: "Username:", "Password:", and "Confirm Password:". The "Username:" field is currently empty. The form is styled with a light gray background and rounded corners.

However, when I try and fill in the form after the first error message, the following error is displayed:

OperationalError

sqlite3.OperationalError: database is locked

Traceback (most recent call last)

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in __call__

```
    return self.wsgi_app(environ, start_response)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1514, in wsgi_app

```
    response = self.handle_exception(e)
```

This indicates to me that the database isn't adequately closed before the initial error occurred and required an additional 'finally' condition on the initial try and except to ensure the connection is closed before it is opened again for the next attempt of the user to register.

```
47     try:
48         connection = sqlite3.connect('users.db') #Same database connection as login system
49         cursor = connection.cursor()
50         cursor.execute(f"INSERT INTO users (username, password, explorativeRating, isolatedRating, uniqueCuisineRating) VALUES ('{u
51         connection.commit()
52         connection.close()
53         flash('Registration successful! Please log in.', 'success') #Informs user registration was successful
54         return redirect(url_for('login')) #Takes user to login
55     except sqlite3.IntegrityError: #Considers the error of a duplicate username
56         flash('Username already exists. Please choose a different one.') #Gives a pop-up to the user indicating an error
57     finally:
58         connection.close()
59
60     return render_template('register.html') #If webpage opened directly (with no form filled in), user redirected to that webpage
```

This makes the registration process repeatable for the user where lines 57 and 58 have been added.

This completes points 2 and 3 of my success criteria on page 8, although they have no immediate functionality until the code for the actual holidays is made. The implementation of point 2 in particular isn't exactly matched, where my code contains numbers for the user to rate themselves with. This can be seen as more beneficial as it would require less thought for a user to give a number rating for a certain characteristics of theirs as opposed to an answer of how likely they would do something. This means that it is quicker for the user to register, and given that the whole website can only be accessed after registering, it is important to keep the time taken to register down to keep users on the site and for them to explore everything else TripMaestro offers.

The next success criteria to be worked on is the first point, and is quoted as:

1. The website has drop down menus that points the user to a list of destinations, and clicking on one of those destinations takes you to another external website to find out more. An external website is used as they will be able to provide specialist information from people who have visited that destination and given reviews, unlike the main website where, when it is launched, no reviews would be on there.

This can be placed on the dashboard where this is meant to be the main attraction of the webpage, to be able to explore destinations to complement the original website. However, this page is also meant to provide the user-specific functionality, which would include a dropdown for holidays that the user has previously made/started, although this functionality will be covered in the section for its respective criteria.

To first work on this, the HTML for the dashboard needs to be developed, and has been made in mind of the design thought of on page 21.

```

1 <html>
2 <head>
3   <title>TripMaestro Dashboard</title>
4   <link rel="stylesheet" href="{{ url_for('static', filename='dashboard.css') }}">
5 </head>
6 <body>
7   <header class="dashboard-header">
8     <div class="header-left">
9       <h1>TripMaestro</h1>
10    </div>
11    <div class="header-right">
12      <button class="btn-signout">Sign Out</button>
13      <button class="btn-profile">Profile</button>
14    </div>
15  </header>
16
17  <main class="dashboard-main">
18    <p>Based on your preferences, here's what we found:</p>
19    <button class="btn-filter" onclick="toggleFilterPanel()">Filter</button>
20
21    <div class="filter-panel" id="filterPanel" style="display: none;">
22      <h2>Filter Options</h2>
23      <p>Use these filters to refine your destination suggestions:</p>
24      <div class="filter-option">
25        <label for="distance">Distance Away</label>
26        <select name="distance" id="distance">
27          <option value="any">Any</option>
28          <option value="close">Close (under 500 miles)</option>
29          <option value="medium">Medium (500-1500 miles)</option>
30          <option value="far">Far (over 1500 miles)</option>
31        </select>
32      </div>
33      <div class="filter-option">
34        <label for="rating">Average Rating</label>
35        <select name="rating" id="rating">
36          <option value="any">Any</option>
37          <option value="3plus">3.0+</option>
38          <option value="4plus">4.0+</option>
39          <option value="5">5.0 only</option>
40        </select>
41      </div>
42      <div class="filter-option">
43        <label for="other">Other Criteria</label>
44        <select name="other" id="other">
45          <option value="any">Any</option>
46          <option value="family-friendly">Family-Friendly</option>
47          <option value="budget-friendly">Budget-Friendly</option>
48          <option value="luxury">Luxury</option>
49        </select>
50      </div>
51      <!-- More filters go here -->
52      <button class="btn-apply-filters">Apply Filters</button>
53    </div>
54
55    <table class="destination-table">
56      <thead>
57        <tr>
58          <th>Name</th>
59          <th>Cheapest Flight</th>
60          <th>Average Rating</th>
61          <th>Key Landmark</th>

```

```

62     <th></th>
63   </tr>
64 </thead>
65 <tbody>
66   <tr>
67     <td>Mostar</td>
68     <td>£77 pp</td>
69     <td>4.9</td>
70     <td>Old Bridge of Mostar</td>
71     <td><button class="btn-add">Add</button></td>
72   </tr>
73   <tr>
74     <td>Kingston</td>
75     <td>£352 pp</td>
76     <td>4.2</td>
77     <td>Bob Marley Museum</td>
78     <td><button class="btn-add">Add</button></td>
79   </tr>
80   <tr>
81     <td>Malaga</td>
82     <td>£127 pp</td>
83     <td>3.1</td>
84     <td>Gibralfaro Castle</td>
85     <td><button class="btn-add">Add</button></td>
86   </tr>
87   <tr>
88     <td>Singapore</td>
89     <td>£740 pp</td>
90     <td>4.8</td>
91     <td>Gardens by the Bay</td>
92     <td><button class="btn-add">Add</button></td>
93   </tr>
94 </tbody>
95 </table>
96
97 <!-- More rows go here when show more pressed (hidden initially) -->
98 <div id="extendedDestinations" style="display: none;">
99   <!-- Future rows here -->
100 </div>
101
102 <button class="btn-show-more" onclick="showMoreDestinations()">Show More</button>
103 </main>

```

Below is the JavaScript used on this page to allow the visibility of the filter page to be toggled. toggleFilterPanel() is what is triggered when the 'Filter' button is clicked, and checks if the filter panel is displayed. If it is, then the panel is closed, otherwise it is opened, acting as a switch between them.

```

104
105 <script>
106   function toggleFilterPanel() { //Toggles visibility of the filter panel
107     const filterPanel = document.getElementById('filterPanel'); //Get the filter panel element
108     if (filterPanel.style.display === 'none' || filterPanel.style.display === '') {
109       filterPanel.style.display = 'block'; // Show the filter panel
110     } else {
111       filterPanel.style.display = 'none'; // Hide the filter panel
112     }
113   }

```

Below also is the JavaScript that adds more destinations to the end of the table. This is just a placeholder for the actual destinations that would be taken from the API, and is displayed as a 'block' below the table.

```

114
115 // Function to show more destinations
116 function showMoreDestinations() {
117   const extendedDiv = document.getElementById('extendedDestinations'); //Gets extended destinations div
118   extendedDiv.style.display = 'block'; // Makes new destinations visible
119   //Below is placeholder html for contents of new destinations
120   extendedDiv.innerHTML = `
121     <table class="destination-table" style="margin-top: 20px;">
122       <tbody>
123         <tr>
124           <td>a</td>
125           <td>a</td>
126           <td>a</td>
127           <td>a</td>

```

```
128         <td><button class="btn-add">Add</button></td>
129     </tr>
130     <tr>
131         <td>b</td>
132         <td>b</td>
133         <td>b</td>
134         <td>b</td>
135         <td><button class="btn-add">Add</button></td>
136     </tr>
137 </tbody>
138 </table>
139 `;
140 }
141 </script>
142 </body>
143 </html>
144
```

To test its functionality, I loaded the website's dashboard to see the contents of the page.

TripMaestro

Based on your preferences, here's what we found:

Filter

Name	Cheapest Flight	Average Rating	Key Landmark	
Mostar	£77 pp	4.9	Old Bridge of Mostar	Add
Kingston	£352 pp	4.2	Bob Marley Museum	Add
Malaga	£127 pp	3.1	Gibraltar Castle	Add
Singapore	£740 pp	4.8	Gardens by the Bay	Add

[Show More](#)

This displayed as expected, and when pressing the filter button, the filter appeared just above the table as expected with its three options.

TripMaestro

Based on your preferences, here's what we found:

Filter

Filter Options

Use these filters to refine your destination suggestions:

Distance Away

Average Rating

Other Criteria

[Apply Filters](#)

Name	Cheapest Flight	Average Rating	Key Landmark	
Mostar	£77 pp	4.9	Old Bridge of Mostar	Add
Kingston	£352 pp	4.2	Bob Marley Museum	Add
Malaga	£127 pp	3.1	Gibraltar Castle	Add
Singapore	£740 pp	4.8	Gardens by the Bay	Add

Pressing the filter button again took away the panel as expected.

Then testing the 'Show More' button revealed the two more placeholder rows as expected, although the table isn't aligned.

Based on your preferences, here's what we found:

Filter

Name	Cheapest Flight	Average Rating	Key Landmark	
Mostar	£77 pp	4.9	Old Bridge of Mostar	Add
Kingston	£352 pp	4.2	Bob Marley Museum	Add
Malaga	£127 pp	3.1	Gibralfaro Castle	Add
Singapore	£740 pp	4.8	Gardens by the Bay	Add

a	a	a	a	Add
b	b	b	b	Add

Show More

This suggests that it would be a better solution to implement JavaScript to actually merge the tables rather than make two separate ones vertically on the page. To achieve this, two tables can be made, with one containing the next destinations to be displayed. Then, a function `mergeTables()` can be made to move the rows of the extra table into the original table.

After experimenting with that, I decided it wouldn't be worth it to pursue that, where there is no tangible benefit to the stakeholder to having everything on one page. This would however be much harder for me to implement than a system involving multiple pages to scroll through, so I decided to pivot back to a page system using Python.

```

67 @app.route('/dashboard', methods=['GET', 'POST'])
68 def destinations():
69     all_destinations = [
70         # 10 items just for example
71         {"name": "Mostar", "flight": "£77 pp", "rating": 4.9, "landmark": "Old Bridge"},
72         {"name": "Kingston", "flight": "£352 pp", "rating": 4.2, "landmark": "Bob Marley Museum"},
73         {"name": "Malaga", "flight": "£127 pp", "rating": 3.1, "landmark": "Gibraltar Castle"},
74         {"name": "Singapore", "flight": "£740 pp", "rating": 4.8, "landmark": "Gardens by the Bay"},
75         {"name": "Paris", "flight": "£99 pp", "rating": 4.3, "landmark": "Eiffel Tower"},
76         {"name": "Tokyo", "flight": "£620 pp", "rating": 4.7, "landmark": "Shibuya Crossing"},
77         {"name": "London", "flight": "£80 pp", "rating": 4.5, "landmark": "Big Ben"},
78         {"name": "Rome", "flight": "£120 pp", "rating": 4.6, "landmark": "Colosseum"},
79         {"name": "Berlin", "flight": "£65 pp", "rating": 4.1, "landmark": "Brandenburg Gate"},
80         {"name": "New York", "flight": "£350 pp", "rating": 4.4, "landmark": "Statue of Liberty"},
81     ]
82
83     page = request.args.get('page', 1, type=int) #Gets page parameter from URL, defaults to 1
84     per_page = 4 #Constant used to determine how many destinations are shown on each page
85     start_index = (page - 1) * per_page #Finds first destination to be shown on a page
86     end_index = start_index + per_page #Finds last destination to be shown on a page
87     page_items = all_destinations[start_index:end_index] #Slices list to see which destinations are to be displayed
88     if end_index < len(all_destinations):
89         next_page = page + 1
90     if page > 1:
91         prev_page = page - 1
92
93     return render_template("destinations.html",
94                           page_items=page_items,
95                           page=page,
96                           next_page=next_page,
97                           prev_page=prev_page)
98

```

I made this function in Python to manage the page system, and put in 10 placeholder destinations. 4 are taken from the list at a time, and that slice of 4 items in the list are entered as a parameter in 'page_items'.

To accommodate for this, I also modified the table in the HTML to use Jinja to take the parameters from 'render_template' and display them. I also added buttons that redirect the user to the next and previous page, and that also meant I was able to remove the 'showMoreDestinations()' function that I had previously made in JavaScript.

```

54
55 <table class="destination-table">
56     <thead>
57         <tr>
58             <th>Name</th>
59             <th>Cheapest Flight</th>
60             <th>Average Rating</th>
61             <th>Key Landmark</th>
62             <th></th>
63         </tr>
64     </thead>
65     <tbody>
66         {% for dest in page_items %}
67         <tr>
68             <td>{{ dest.name }}</td>
69             <td>{{ dest.flight }}</td>
70             <td>{{ dest.rating }}</td>
71             <td>{{ dest.landmark }}</td>
72             <td><button class="btn-add">Add</button></td>
73         </tr>
74         {% endfor %}
75     </tbody>
76 </table>
77
78 {% if prev_page %}
79 <a class="btn-page" href="{{ url_for('destinations', page=prev_page) }}">Previous Page</a>
80 {% endif %}
81
82 {% if next_page %}
83 <a class="btn-page" href="{{ url_for('destinations', page=next_page) }}">Next Page</a>
84 {% endif %}
85

```

During my first test of this, I saw the following error:

UnboundLocalError

UnboundLocalError: cannot access local variable 'prev_page' where it is not associated with a value

Traceback (most recent call last)

```
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 917, in full_dispatch_request
```

This prompted me to realise that lines 89 and 91 are the only lines in which `next_page` and `prev_page` are referenced before they are called upon in lines 96 and 97 (in `app.py`). To fix this, I assigned both variables to just take the value of `page`, as if it would need to be changed that would be handled by the original if statements:

```
83     prev_page = page
84     next_page = page
85     page = request.args.get('page', 1, type=int) #Gets page parameter from URL, defaults to 1
```

This gave me the following error:

UnboundLocalError

UnboundLocalError: cannot access local variable 'page' where it is not associated with a value

Traceback (most recent call last)

```
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

I suspected that this would arise from the order in which I have used `page`, where I have tried to use it before it was actually defined from the URL.

TemplateNotFound

jinja2.exceptions.TemplateNotFound: destinations.html

Traceback (most recent call last)

```
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
    ~~~~~^~~~~~

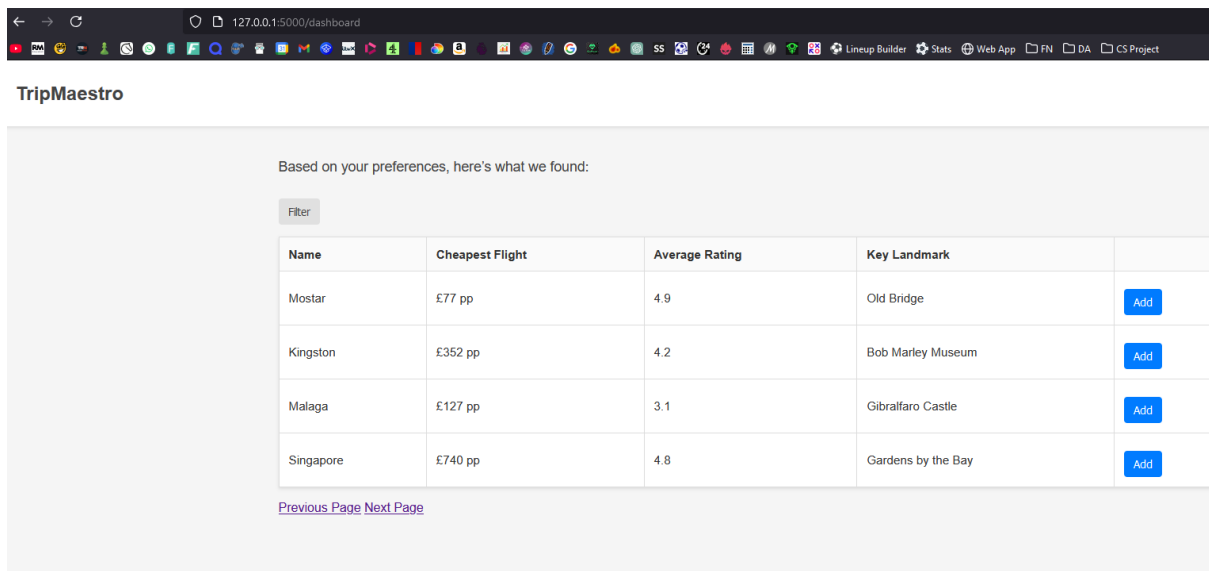
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
    ~~~~~^~~~~~

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
    ~~~~~^~~~~~

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
    ~~~^~~~~~
```

Further testing prompted this error, and made me realise that I during this stage I had kept on referring to it as destinations.html when I wanted this to be part of dashboard.html, so I changed every instance of destinations.html with dashboard.html to ensure it was being referred to at the right moment.

Now, testing a direct link to the dashboard gave me a successful output, where I was taken to the first page of the destinations list.



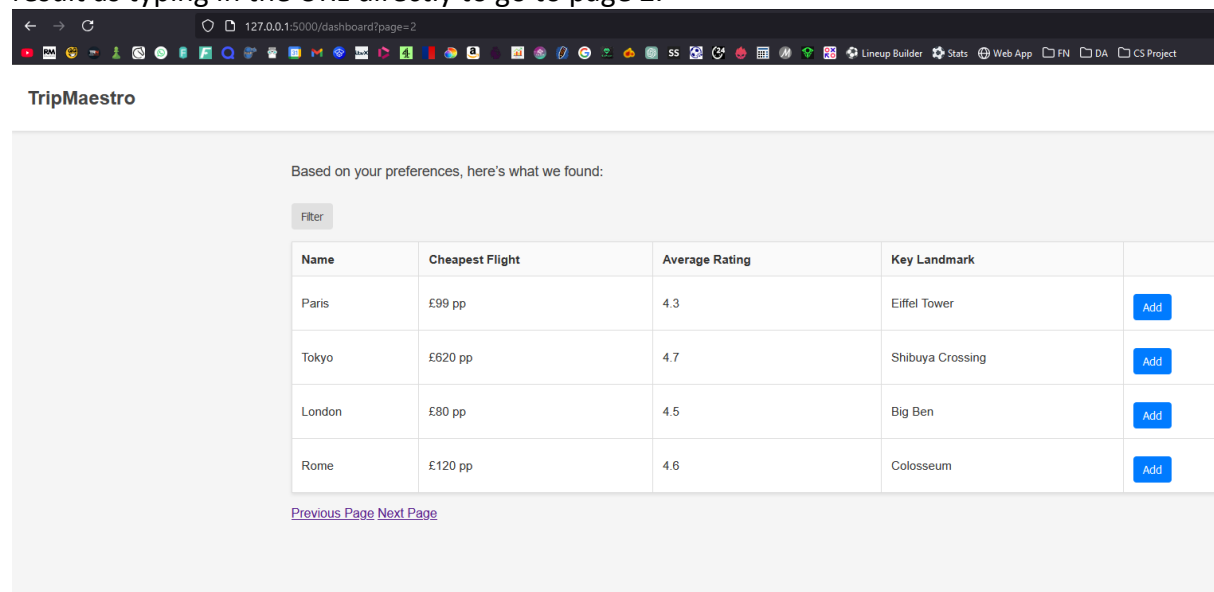
Based on your preferences, here's what we found:

Filter

Name	Cheapest Flight	Average Rating	Key Landmark	
Mostar	£77 pp	4.9	Old Bridge	Add
Kingston	£352 pp	4.2	Bob Marley Museum	Add
Malaga	£127 pp	3.1	Gibralfaro Castle	Add
Singapore	£740 pp	4.8	Gardens by the Bay	Add

[Previous Page](#) [Next Page](#)

I then tested the 'Next Page' and 'Previous Page' buttons which worked, with the same result as typing in the URL directly to go to page 2.



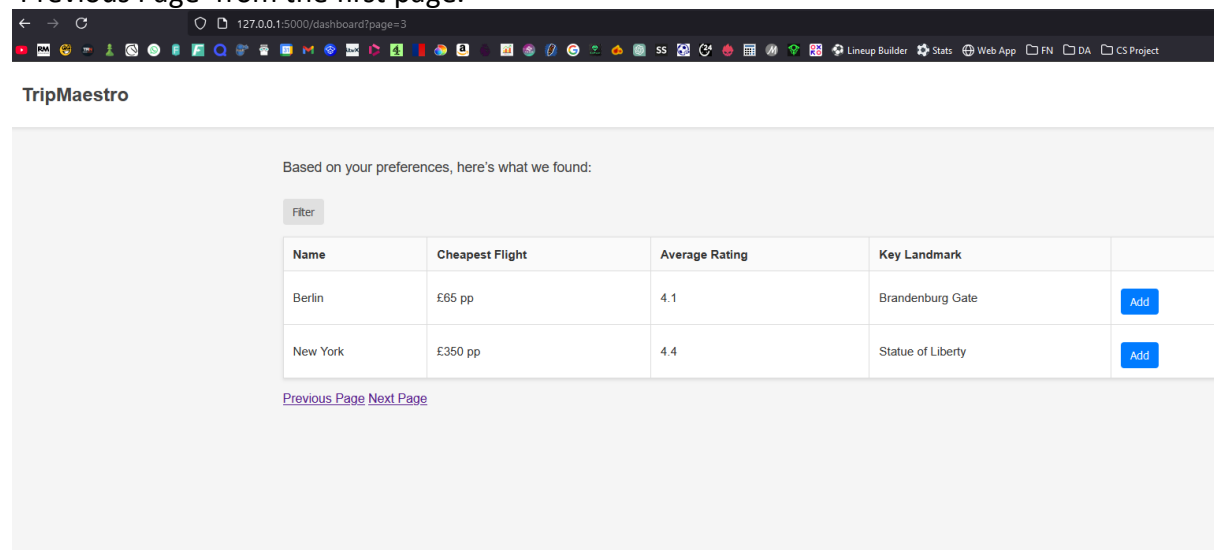
Based on your preferences, here's what we found:

Filter

Name	Cheapest Flight	Average Rating	Key Landmark	
Paris	£99 pp	4.3	Eiffel Tower	Add
Tokyo	£620 pp	4.7	Shibuya Crossing	Add
London	£80 pp	4.5	Big Ben	Add
Rome	£120 pp	4.6	Colosseum	Add

[Previous Page](#) [Next Page](#)

I also tested going to the final page and pressing 'Next Page' again, which didn't take me direct me to anywhere else as expected. This same result occurred for when I pressed on 'Previous Page' from the first page.



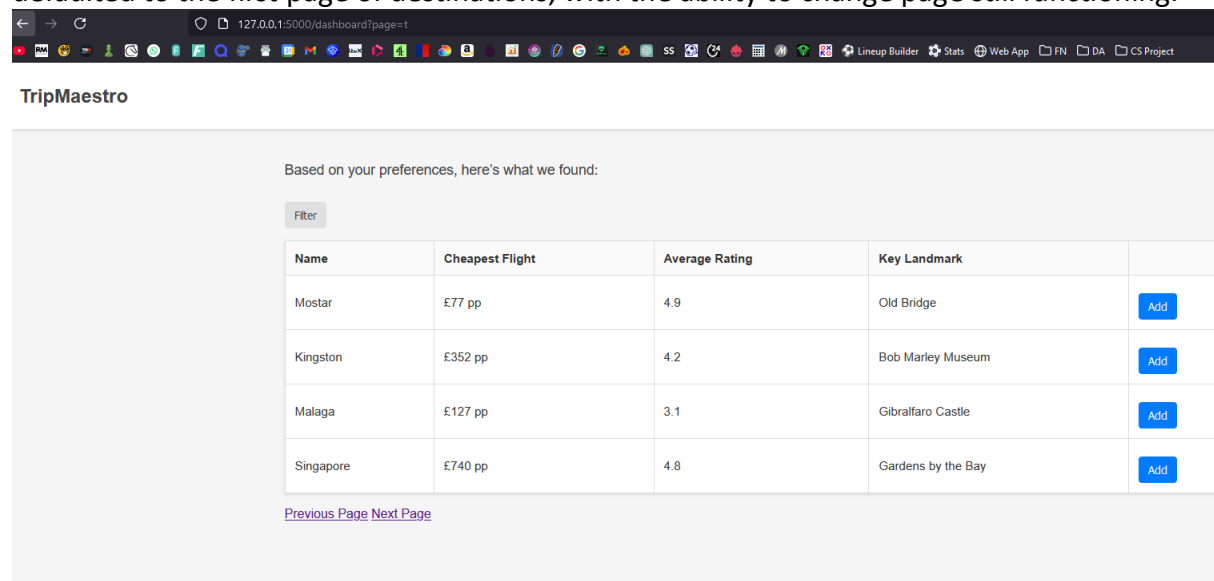
Based on your preferences, here's what we found:

Filter

Name	Cheapest Flight	Average Rating	Key Landmark	
Berlin	£65 pp	4.1	Brandenburg Gate	Add
New York	£350 pp	4.4	Statue of Liberty	Add

[Previous Page](#) [Next Page](#)

I also tested non-integer values in the URL for the page number, and as expected, it defaulted to the first page of destinations, with the ability to change page still functioning.



To fully meet the success criteria, it is important to retrieve data from an API rather than a fixed list of destinations like I had used previously. This led me to the TripAdvisor API, and made me consider the different attributes that I could include in the table on the dashboard. I decided to replace 'Cheapest Flight' with Price Range, and added a column for Awards. This can help the user feel more attached to the destinations they read about, making them want to learn more through the website. This can in turn cause more traffic to their blog site which the stakeholders want.

I first created a table in a new destinations.db file that give each country scores about the three preferences that they had previously been asked for. This doesn't give every country in the world but is a compromise worth making for the stakeholder, given the more niche countries that this table designed for tourists are often ignored by tourists when considering where to spend their holidays.

Table: countries

	name	cuisine_score	peaceful_score	activities_score
	Filter	Filter	Filter	Filter
1	Italy	1000	472	880
2	Japan	980	420	850
3	India	970	392	780
4	Lebanon	960	349	520
5	Thailand	950	506	790
6	Mexico	940	581	810
7	China	930	501	840
8	Turkey	920	531	760
9	Greece	910	563	740
10	Vietnam	900	429	580
11	France	890	518	830
12	Spain	880	544	800
13	USA	870	644	860
14	South Korea	860	376	550
15	Brazil	850	679	670
16	Malaysia	840	533	610
17	Bangladesh	830	280	220
18	Indonesia	820	501	630
19	Germany	810	455	720
20	Portugal	800	526	580
21	Morocco	790	559	560
22	Jordan	780	525	500
23	Bosnia and ...	770	586	450
24	Argentina	760	720	600
25	Singapore	750	95	480
26	Russia	740	787	650
27	Peru	730	677	640
28	Australia	720	878	690
29	South Africa	710	605	620
30	Philippines	700	405	570
31	United Kingdom	690	436	650

1 - 31 of 96

To then make this table useful, I created a function called `fetch_destinations(country_name)` that uses the TripAdvisor API to find destinations that correspond to a country name.

```
67
68 #Function to fetch destinations from Tripadvisor API
69 def fetch_destinations(country_name):
70     api_key = '0F0E9E89119248B2AFE0832BFE031E7' #API Key from TripAdvisor
71     url = 'https://api.content.tripadvisor.com/api/v1/location/search' #base URL of API
72     headers = {
73         'x-api-key': api_key #API Key held like this for TripAdvisor API syntax
74     }
75     params = {
76         'query': country_name, #Searches for this country
77         'category': 'geos', #Looks for destinations
78         'language': 'en'
79     }
80     response = requests.get(url, headers=headers, params=params) #Sends request to API
81     if response.status_code == 200: #If API request is successful
82         data = response.json() #Converts API response to JSON
83         return data.get('data', []) #Returns the data from the API
84     else:
85         print(f'Error fetching data: {response.status_code}')
86         return []
87
```

To test this works, I took this into a separate Python file and manually entered a parameter of 'France' to see what is returned.

```
1  ✓ from flask import *
2      import sqlite3
3      import requests
4
5  ✓ def fetch_destinations(country_name):
6      api_key = '0F0E9E89119248B2AFE0832BFE031E7' # Replace with your actual API key
7      url = 'https://api.content.tripadvisor.com/api/v1/location/search' #base URL of API
8      ✓ headers = {
9          'x-api-key': api_key #API Key held like this for TripAdvisor API syntax
10     }
11  ✓ params = {
12      'query': country_name, #Searches for this country
13      'category': 'geos', #Looks for destinations
14      'language': 'en'
15  }
16      response = requests.get(url, headers=headers, params=params) #Sends request to API
17  ✓ if response.status_code == 200: #If API request is successful
18      data = response.json() #Converts API response to JSON
19      print(data)
20  ✓ else:
21      print(f'Error fetching data: {response.status_code}')
22      return []
23
24  fetch_destinations('France')
```

This returned an error 401 indicating an authentication error as shown below:

```
/trip_maestro/fetch_test.py
Error fetching data: 401
```

To try and tackle this, I added some debugging lines (in white) to work out what is passed into the request unexpectedly:

```
1  from flask import *
2  import sqlite3
3  import requests
4
5  def fetch_destinations(country_name):
6      api_key = '0F0E9E89119248B2AFEF0832BFE031E7' #API Key from TripAdvisor
7      url = 'https://api.content.tripadvisor.com/api/v1/location/search' # Base URL of API
8      headers = {
9          'x-api-key': api_key # API Key held like this for TripAdvisor API syntax
10     }
11     params = {
12         'query': country_name, # Searches for this country
13         'category': 'geos', # Looks for destinations
14         'language': 'en'
15     }
16     response = requests.get(url, headers=headers, params=params) # Sends request to API
17
18     print("Request URL:", response.url)
19     print("Request Headers:", headers)
20     print("Response Status Code:", response.status_code)
21     print("Response Text:", response.text)
22
23     if response.status_code == 200: # If API request is successful
24         data = response.json() # Converts API response to JSON
25         print(data)
26     else:
27         print(f'Error fetching data: {response.status_code}')
28         return []
29
30 fetch_destinations('France')
```

```
Request URL: https://api.content.tripadvisor.com/api/v1/location/search?query=France&category=geos&language=en
Request Headers: {'x-api-key': '0F0E9E89119248B2AFEF0832BFE031E7'}
Response Status Code: 401
Response Text: {"message": "Unauthorized"}
Error fetching data: 401
```

This is the output I received.

My first port of call in troubleshooting was to refer back to the documentation to see what I have missed or if there was an example I could compare to.

This led me to find an exemplar response on the website:

```
import requests
```

```
url =
```

```
"https://api.content.tripadvisor.com/api/v1/location/search?key=0F0E9E89119248B2AFEF0832BFE031E7&searchQuery=France&category=geos&language=en"
```

```
headers = {"accept": "application/json"}
```

```
response = requests.get(url, headers=headers)
```

```
print(response.text)
```

I immediately noticed differences in how I had approached the headers and the example did. The example was able to change the URL entirely and had a plain “accept” header as opposed to trying to add the query, language, and api key passed in along with that. What I had done also resulted in a different URL being requested from, where the key wasn’t actually in the URL. Although this would be expected due to the nature of the key needed to be kept from the user, I opted to directly enter the key into the URL using an fstring along with the other parameters to make the following code:

```
def fetch_destinations(country_name):
    key = "0F0E9E89119248B2AFEF08328FE031E7"
    url = f"https://api.content.tripadvisor.com/api/v1/location/search?key={key}&searchQuery={country_name}&category=geos&language=en" # Base URL of API
    headers = {"accept": "application/json"}
    response = requests.get(url, headers=headers) # Sends request to API
```

This resulted in me getting the expected output shown below, where it shows top results of the search “France” in TripAdvisor.

```
{'data': [{'location_id': '187070', 'name': 'France', 'address_obj': {'address_string': 'France'}}, {'location_id': '187144', 'name': 'Ile-de-France', 'address_obj': {'country': 'France', 'address_string': 'France'}}, {'location_id': '147328', 'name': 'Fort-de-France', 'address_obj': {'city': 'Arrondissement of Fort-de-France', 'state': 'Martinique', 'country': 'Caribbean', 'address_string': 'Fort-de-France Martinique'}}, {'location_id': '196587', 'name': 'Roissy-en-France', 'address_obj': {'city': 'Ile-de-France', 'country': 'France', 'address_string': 'Roissy-en-France France'}}, {'location_id': '226863', 'name': 'Tremblay-En-France', 'address_obj': {'city': 'Ile-de-France', 'country': 'France', 'address_string': 'Tremblay-En-France France'}}, {'location_id': '499259', 'name': 'Fort Frances', 'address_obj': {'street1': '', 'street2': '', 'state': 'Ontario', 'country': 'Canada', 'postalcode': '', 'address_string': 'Fort Frances, Ontario Canada'}}, {'location_id': '11038886', 'name': 'Hauts-de-France', 'address_obj': {'country': 'France', 'address_string': 'France'}}, {'location_id': '3842848', 'name': 'Praia do Frances', 'address_obj': {'city': 'Marechal Deodoro', 'state': 'State of Alagoas', 'country': 'Brazil', 'address_string': 'Marechal Deodoro, State of Alagoas Brazil'}}, {'location_id': '187269', 'name': 'Saint-Etienne', 'address_obj': {'city': 'Auvergne-Rhone-Alpes', 'country': 'France', 'address_string': 'Saint-Etienne France'}}, {'location_id': '187242', 'name': 'Saint-Tropez', 'address_obj': {'city': 'Provence-Alpes-Cote d'Azur', 'country': 'France', 'address_string': 'Saint-Tropez France'}}]}
```

However, this did notably give priority to results that directly have the word “France” in it , which led to results such as ‘Fort-de-France’ located in the Caribbean.

I realised at this point that I didn’t need to entirely rely on the search endpoint as the only search, as I was able to use the locationID obtained from this, and can give it to the locationdetails endpoint to find more information about the country.

When programming this, I was able to abuse the fact that the API returned the country first when searching the country’s name. As trivial as this sounds, this could be something changed in the future if the endpoint returns items in a different order such as in order of longitude.

Below shows what I mean:

The image shows two parts. On the left is a snippet of the TripAdvisor API documentation for the search endpoint. It includes a note: "This endpoint may also return bookable products (activities). For those activities that are returned there are no additional details available on the Location Details endpoint." Below this, the "QUERY PARAMS" section shows:

- key** (string, required): The Partner API Key. Value: 1FEF08328FE031E7
- searchQuery** (string, required): Text to use for searching based on the name of the location. Value: Germany

 On the right is a REST client interface showing the "RESPONSE" for a GET request. The response status is 200 OK. The JSON response is:


```
{
  "data": [
    {
      "location_id": "187275",
      "name": "Germany",
      "address_obj": {
        "address_string": "Germany"
      }
    }
  ]
}
```

 A blue circle is drawn around the first (and only) item in the response array, highlighting the result for Germany.

Chaining the endpoints saw me make the following coded:

```

1 from flask import *
2 import sqlite3
3 import requests
4
5 def fetch_destinations(country_name):
6     location_id = None
7     key = "0F0E9E89119248B2AFEF0832BFE031E7"
8     searchURL = f"https://api.content.tripadvisor.com/api/v1/location/search?key={key}&searchQuery={country_name}&category=geos&language=en" #URL for search endpoint
9     locationDetailsURL = f"https://api.content.tripadvisor.com/api/v1/location/{location_id}/details?key={key}&language=en" #URL for location detail endpoint
10    headers = {"accept": "application/json"}
11    response = requests.get(searchURL, headers=headers) # Sends request to API
12
13    print("Request URL:", response.url)
14    print("Request Headers:", headers)
15    print("Response Status Code:", response.status_code)
16    print("Response Text:", response.text)
17
18    if response.status_code == 200: #If API request is successful
19        data = response.json() #Converts API response to JSON
20        first_item = data['data'][0] #Looks into the data variable, for the data section of response, for first item
21        location_id = first_item.get('location_id') #Extract the location ID
22        name = first_item.get('name') #Extract the name
23
24        response = requests.get(locationDetailsURL, headers=headers) #Sends request to API
25        if response.status_code == 200:
26            data = response.json()
27            description = data.get('description', 'No description available')
28            web_url = data.get('web_url', 'No web URL available')
29            print(f'Name: {name}, Description: {description}, URL: {web_url}')
30        else:
31            print(f'Error fetching location details: {response.status_code}')
32            return []
33    else:
34        print(f'Error fetching data: {response.status_code}')
35        return []
36
37 fetch_destinations('France')

```

This is still in the separate test file so will be properly annotated later, but does cover for failures in the API at each call, each returning its own error code should that occur.

Subsequent testing gave me the following error:

```

"address_obj": {
  "city": "Provence-Alpes-Cote d'Azur",
  "country": "France",
  "address_string": "Saint-Tropez France"
}
]
}
}
Error fetching location details: 400

```

After reviewing my code, I thought that the error lay in the fact that I hadn't defined my location URL properly, where it used a placeholder location ID:

```

9 locationDetailsURL = f"https://api.content.tripadvisor.com/api/v1/location/{location_id}/details?key={key}&language=en" #URL for location detail endpoint
10 headers = {"accept": "application/json"}
11 response = requests.get(searchURL, headers=headers) # Sends request to API
12
13 if response.status_code == 200: #If API request is successful
14     data = response.json() #Converts API response to JSON
15     first_item = data['data'][0] #Looks into the data variable, for the data section of response, for first item
16     location_id = first_item.get('location_id') #Extract the location ID
17     name = first_item.get('name') #Extract the name
18
19     response = requests.get(locationDetailsURL, headers=headers) #Sends request to API
20     if response.status_code == 200:
21         data = response.json()
22         description = data.get('description', 'No description available')
23         web_url = data.get('web_url', 'No web URL available')
24         print(f'Name: {name}, Description: {description}, URL: {web_url}')
25     else:
26         print(f'Error fetching location details: {response.status_code}')
27         return []
28
29 else:
30     print(f'Error fetching data: {response.status_code}')
31     return []
32 fetch_destinations('France')

```

I fixed this by moving the definition of locationDetailsURL to next to the request that uses it.

```

10
11 if response.status_code == 200: #If API request is successful
12     data = response.json() #Converts API response to JSON
13     first_item = data['data'][0] #Looks into the data variable, for the data section of response, for first item
14     location_id = first_item.get('location_id') #Extract the location ID
15     name = first_item.get('name') #Extract the name
16
17     locationDetailsURL = f"https://api.content.tripadvisor.com/api/v1/location/{location_id}/details?key={key}&language=en" #URL for location detail endpoint
18     response = requests.get(locationDetailsURL, headers=headers) #Sends request to API
19     if response.status_code == 200:
20         data = response.json()
21         description = data.get('description', 'No description available')
22         web_url = data.get('web_url', 'No web URL available')
23         print(f'Name: {name}, Description: {description}, URL: {web_url}')
24     else:
25         print(f'Error fetching location details: {response.status_code}')
26         return []
27
28

```

This gave me a partially completed set of what I wanted to display to the user, as I also wanted to include images to replace the 'Key Landmark' I had previously put in as a placeholder.

At this point, I had realised my approach to dealing with the API was wrong, and it would be more advantageous to use object-oriented programming to collect common attributes and keep the methods modular to help future maintainability where errors could be more easily identified.

I rebuilt each of the previous requests into their own methods, with common attributes being the API key, the base URL, and the headers that were kept common.

```

1 from flask import *
2 import sqlite3
3 import requests
4
5 class TripAdvisorAPI:
6     def __init__(self, api_key):
7         self.api_key = api_key
8         self.base_url = "https://api.content.tripadvisor.com/api/v1/location"
9         self.headers = {"accept": "application/json"}
10
11     def search_destinations(self, country_name):
12         search_url = f"{self.base_url}/search?key={self.api_key}&searchQuery={country_name}&category=geos&language=en"
13         response = requests.get(search_url, headers=self.headers)
14         if response.status_code == 200:
15             data = response.json()
16             first_item = data['data'][0] #Looks into the data variable, for the data section of response, for first item
17             location_id = first_item.get('location_id') #Extract the location ID
18             name = first_item.get('name') #Extract the name
19             return location_id, name
20         else:
21             print(f"Error fetching destinations: {response.status_code}")
22             return None, None
23
24     def get_location_details(self, location_id):
25         details_url = f"{self.base_url}/{location_id}/details?key={self.api_key}&language=en"
26         response = requests.get(details_url, headers=self.headers)
27         if response.status_code == 200:
28             data = response.json()
29             description = data.get('description', 'No description available')
30             web_url = data.get('web_url', 'No web URL available')
31             return description, web_url
32         else:
33             print(f"Error fetching location details: {response.status_code}")
34             return None, None
35

```

```

36     def get_location_image(self, location_id):
37         images_url = f"{self.base_url}/{location_id}/photos?key={self.api_key}&language=en"
38         response = requests.get(images_url, headers=self.headers)
39         if response.status_code == 200:
40             data = response.json()
41             images = data.get('data', [])
42             first_image = images[0]
43             medium_image = first_image.get('images', {}).get('medium', {})
44             image_url = medium_image.get('url', 'No image URL available')
45             return image_url
46         else:
47             print(f"Error fetching location images: {response.status_code}")
48             return []
49
50     def fetch_and_display_details(self, country_name):
51         location_id, name = self.search_destinations(country_name)
52         description, web_url = self.get_location_details(location_id)
53         image_url = self.get_location_image(location_id)
54         print(location_id, name, description, web_url, image_url)
55
56 api = TripAdvisorAPI("0F0E9E89119248B2AFEF0832BFE031E7")
57 api.fetch_and_display_details("France")
58
59

```

I initially had some trouble with the final method, `get_location_image()`, where I was told by the terminal that I had a list index out of range. The issue ended up being that I tried to jump steps within lines 41 – 44 where I only had 3 occurrences of `“.get()”` appearing when there should have been 4 according to the example responses of the API from the website.

Testing this individual program was successful, returning all of the information I wanted to display on the dashboard:

187070 France there's much more to France than Paris—from the fairy-tale châteaux of the Loire Valley to the lavender fields of Provence, and the French Riviera's celebrity-studded beaches. World-famous gastronomy and fine wines provide the perfect complement to the country's alpine views and architectural masterpieces. <https://www.tripadvisor.com/Tourism-g187070-France-Vacations.html> <https://media-cdn.tripadvisor.com/media/photo-f/04/84/90/ff/france.jpg>

I moved the code over to `app.py`, commenting it appropriately as shown below, but realised it would be quite difficult to test further without then developing the actual preference logic of weighting what is important to the user.

```

144 class TripAdvisorAPI: #defines class
145     def __init__(self, api_key): #initializes class
146         self.api_key = api_key #API key attribute
147         self.base_url = "https://api.content.tripadvisor.com/api/v1/location" #Base URL for API
148         self.headers = {"accept": "application/json"} #Common headers used for API calls
149
150     def search_destinations(self, country_name): #Method to search for destinations
151         search_url = f"{self.base_url}/search?key={self.api_key}&searchQuery={country_name}&category=geos&language=en" #URL for search endpoint
152         response = requests.get(search_url, headers=self.headers) # Sends request to API
153         if response.status_code == 200: #If the request is successful
154             data = response.json() #Converts response to JSON
155             first_item = data["data"][0] #Looks into the data variable, for the data section of response, for first item
156             location_id = first_item.get('location_id') #Extract the location ID
157             name = first_item.get('name') #Extract the name
158             return location_id, name #Returns location ID and name
159         else: #If request is unsuccessful
160             print(f"Error fetching destinations: {response.status_code}") #Prints error message
161             return None, None #Returns nothing for both location ID and name
162
163     def get_location_details(self, location_id): #Method to get location details
164         details_url = f"{self.base_url}/{location_id}/details?key={self.api_key}&language=en" #URL for details endpoint
165         response = requests.get(details_url, headers=self.headers) #Sends request to API
166         if response.status_code == 200:
167             data = response.json()
168             description = data.get('description', 'No description available') #Extracts description from data
169             web_url = data.get('web_url', 'No web URL available') #Extracts web URL from data
170             return description, web_url
171         else:
172             print(f"Error fetching location details: {response.status_code}")
173             return None, None

```

```

174
175     def get_location_image(self, location_id): #Method to get location image
176         images_url = f"{self.base_url}/{location_id}/photos?key={self.api_key}&language=en"
177         response = requests.get(images_url, headers=self.headers)
178         if response.status_code == 200:
179             data = response.json()
180             images = data.get('data', []) #Extracts images from data
181             first_image = images[0] #Extracts first image from images
182             medium_image = first_image.get('images', {}).get('medium', {}) #Extracts all types of the first image
183             # then the middle image from there
184             image_url = medium_image.get('url', 'No image URL available') #Extracts image URL from medium image
185             return image_url
186         else:
187             print(f"Error fetching location images: {response.status_code}")
188             return []
189
190     def fetch_and_display_details(self, country_name): #Method to fetch and display details
191         location_id, name = self.search_destinations(country_name) #Searches for destinations
192         description, web_url = self.get_location_details(location_id) #Gets location details
193         image_url = self.get_location_image(location_id) #Gets location image
194         print(location_id, name, description, web_url, image_url)

```

I began to work on establishing user preferences with start of this function:

```

60     def determinePreferences():
61         # Get user preferences from database
62         userConnection = sqlite3.connect('users.db')
63         userCursor = userConnection.cursor()
64         userCursor.execute("SELECT explorativeRating, isolatedRating, uniqueCuisineRating FROM users WHERE id = ?",
65                             (session['user_id'],))
66         user_prefs = userCursor.fetchone()
67         userConnection.close()
68
69
70         explore_pref = user_prefs[0] * 0.1
71         peaceful_pref = user_prefs[1] * 0.1
72         cuisine_pref = user_prefs[2] * 0.1
73
74
75         destConnection = sqlite3.connect('destinations.db')
76         destCursor = destConnection.cursor()
77         destCursor.execute("SELECT * FROM countries")
78         countries = destCursor.fetchall()
79         destConnection.close()
80         ranking = []
81         scoreDict = {}
82         for each in countries:
83             score = each[1] * cuisine_pref + each[2] * peaceful_pref + each[3] * explore_pref
84             scoreDict[each[0]] = score
85             ranking.append(score)
86             ranking.sort()
87             ranking.reverse()
88

```

The idea was to combine the scores from the database, and the preference rating of the user to create a personalised rating of each destination, and then show the user the highest ones. However, I realised through writing this that the way I had the dictionary setup meant that if there were two destinations with the same weighted value, then one of them would not be displayed to the user at all. This meant I had to create a check to see that when a destination and its score was being added to the dictionary, the score didn't exist already, or if it did, then the score would be offset by a very small amount to avoid conflict.

```

60 def determinePreferences():
61     userConnection = sqlite3.connect('users.db') #Same old database connection
62     userCursor = userConnection.cursor()
63     userCursor.execute("SELECT explorativeRating, isolatedRating, uniqueCuisineRating FROM users WHERE id = ?",
64         (session['user_id'],)) #Gets user preferences from database
65     user_prefs = userCursor.fetchone() #Takes the user's record from the database
66     userConnection.close() #Ends connection between program and database to avoid locking
67
68     explore_pref = user_prefs[0] * 0.1 #Converts user preferences to a multiplier of each destinations score
69     peaceful_pref = user_prefs[1] * 0.1
70     cuisine_pref = user_prefs[2] * 0.1
71
72     destConnection = sqlite3.connect('destinations.db')
73     destCursor = destConnection.cursor()
74     destCursor.execute("SELECT * FROM countries")
75     countries = destCursor.fetchall() #Makes a complete array of every country in the database
76     destConnection.close()
77     ranking = [] #Makes an empty array to store the rankings of the numerical scores
78     destRanking = [] #Makes an empty array to store the rankings of the countries
79     scoreDict = {} #Makes a dictionary to hold scores and their corresponding countries
80     for each in countries:
81         score = round(each[1] * cuisine_pref + each[2] * peaceful_pref + each[3] * explore_pref)
82         #Calculates the score of each country based on user preferences
83         if score not in scoreDict: #If the score is not already in the dictionary
84             scoreDict[score] = each[0] #Adds the score and country to the dictionary
85         else: #If the score is already in the dictionary
86             while scoreDict[score] in scoreDict: #Makes sure that it doesn't increment to another score
87                 score += 0.01 #Increments the score by a small amount
88             ranking.append(score) #Adds that country's scr to the ranking array
89             ranking.sort() #Sorts the ranking array
90             ranking.reverse() #Reverses the ranking array so that the highest score is first
91     for each in ranking: #Goes through each score in the ranking array to rank the countries
92         destRanking.append(scoreDict[each]) #Adds the country to the destRanking array based on the score
93     return destRanking #Returns the array of countries ranked by user preferences
94

```

This function calculates the score for each destination and ranks them. The function then returns an array containing destinations in descending order of their score, to be displayed on the dashboard.

The dashboard route in app.py was also updated to take advantage of the new function `determinePreferences()`.

```

155 @app.route('/dashboard', methods=['GET', 'POST'])
156 def dashboard():
157     if 'user_id' not in session: #If the user is not logged in
158         return redirect(url_for('login')) #Redirects user to login page
159
160     ranked_destinations = determinePreferences() #Gets array of ranked destinations
161     page = request.args.get('page', 1, type=int) #Gets the page number from the URL, defaults to 1
162     prev_page, next_page = page, page #Initializes variables for previous and next page
163     per_page = 4 #Constant used to determine how many destinations are shown on each page
164     start_index = (page - 1) * per_page #Finds first destination to be shown on a page
165     end_index = start_index + per_page #Finds last destination to be shown on a page
166     page_items = ranked_destinations[start_index:end_index] #Slices list to see which destinations are to be displayed
167     if end_index < len(ranked_destinations):
168         next_page = page + 1
169     if page > 1:
170         prev_page = page - 1
171
172     return render_template("dashboard.html",
173         page_items=page_items,
174         page=page,
175         next_page=next_page,
176         prev_page=prev_page)

```

To fully adjust to the new columns displayed on the dashboard, dashboard.html needs to be updated to contain the new description as well as image for the destination:

```

55 <table class="destination-table">
56 <thead>
57 <tr>
58 <th>Location ID</th>
59 <th>Name</th>
60 <th>Description</th>
61 <th>Web URL</th>
62 <th>Image</th>
63 </tr>
64 </thead>
65 <tbody>
66 {% for dest in page_items %}
67 <tr>
68 <td>{{ dest.location_id }}</td>
69 <td>{{ dest.name }}</td>
70 <td>{{ dest.description }}</td>
71 <td><a href="{{ dest.web_url }}" target="_blank">View</a></td>
72 <!--target="_blank" opens the link in a new tab-->
73 <td>
74 {% if dest.image_url %}
75 
76 {% else %}
77 No Image
78 {% endif %}
79 </td>
80 </tr>
81 {% endfor %}
82 </tbody>
83 </table>

```

When testing this in full on the website, the dashboard looked like this:

Based on your preferences, here's what we found:

Filter

Location ID	Name	Description	Web URL	Image
			View	No Image
			View	No Image
			View	No Image
			View	No Image

[Previous Page](#) [Next Page](#)

I realised this was because I did not actually make a call to the API, and I also for some reason added a column for location ID when that's not meant to be displayed to the user.

```

54  def fetch_and_display_details(self, country_name): #Method to fetch and display details
55      location_id, name = self.search_destinations(country_name) #Searches for destinations
56      description, web_url = self.get_location_details(location_id) #Gets location details
57      image_url = self.get_location_image(location_id) #Gets location image
58      return name, description, web_url, image_url #Returns name, description, web URL, and image URL
59

```

Line 58 was added so the main method of the API class actually returned something.

I also realised that `page_items` in `dashboard.html` was expecting a list of dictionaries to cycle through and display the contents of each, however, I had set it up so that `page_items` was just a sliced part of `ranked_destinations` as shown below:

```

165  end_index = start_index + per_page #Finds last destination to be shown on a page
166  page_items = ranked_destinations[start_index:end_index] #Slices list to see which destinations are to be displayed
167  if end_index < len(ranked_destinations):

```

To fix this, I made a loop that goes through the index of destinations to be shown, and appends all of the relevant information of each destination as a dictionary into the `page_items` list, which is used in loading the HTML file.

```

155  @app.route('/dashboard', methods=['GET', 'POST'])
156  def dashboard():
157      if 'user_id' not in session: #If the user is not logged in
158          return redirect(url_for('login')) #Redirects user to login page
159
160      tripadvisorapi = TripAdvisorAPI() #Creates an instance of the TripAdvisorAPI class
161      ranked_destinations = determinePreferences() #Gets array of ranked destinations
162      page = request.args.get('page', 1, type=int) #Gets the page number from the URL, defaults to 1
163      prev_page, next_page = page, page #Initializes variables for previous and next page
164      per_page = 4 #Constant used to determine how many destinations are shown on each page
165      start_index = (page - 1) * per_page #Finds first destination to be shown on a page
166      end_index = start_index + per_page #Finds last destination to be shown on a page
167      page_items = [] #Creates an array to store dictionaries of destination data
168      for each in range(start_index - end_index): #Goes through each destination in the range of destinations to be shown on the page
169          name, description, web_url, image_url = tripadvisorapi.fetch_and_display_details(ranked_destinations[each])
170          #Gets the name, description, web URL, and image URL of each destination
171          page_items.append({'name': name, 'description': description, 'web_url': web_url, 'image_url': image_url})
172          #Adds the name, description, web URL, and image URL to the page_items array
173      if end_index < len(ranked_destinations): #If the end index is less than the total number of destinations
174          next_page = page + 1 #Sets the next page to the current page plus 1
175      if page > 1: #If the page is greater than 1
176          prev_page = page - 1 #There has to be a page before it so set it to be that
177
178      return render_template("dashboard.html",
179                             page_items=page_items,
180                             page=page,
181                             next_page=next_page,
182                             prev_page=prev_page)
183

```

When testing this updated code, this was the page I saw:

Based on your preferences, here's what we found:

Filter

Name	Description	Web URL	Image
Previous Page Next Page			

My initial thought was to check the HTML, but after some deeper thought, I noticed that on line 168, I didn't properly search through the index, as `start_index - end_index` is always going to be less than 0 so wouldn't display anything. I subsequently changed it to the following:

```

167     page_items = [] #Creates an array to store dictionaries of destination data
168     for each in range(start_index, end_index): #Goes through each destination in the range of destinations to be shown on the page
169         name, description, web_url, image_url = tripadvisorapi.fetch_and_display_details(ranked_destinations[each])

```

(now a comma was between the start and end indexes rather than a subtract symbol)

This was the result of the subsequent test:

Based on your preferences, here's what we found:

Filter

Name	Description	Web URL	Image
United States	Spanning 2,800 miles (4,506 kilometers) from east to west, the US is a vast patchwork of landscapes and metropolises. From Arizona's red-rocked canyons and California's coastal beaches to New York's big city and Virginia's storied past, it covers any topic any traveler could want or need.	View	
Italy	From the silvery crests of the Alps to the sparkling Mediterranean Sea, Italy's beauty is unparalleled. A visit could mean marveling at the Colosseum after a tender plate of pasta, or drifting lazily down a canal, your belly full of local Sangiovese. Explore Tuscan vineyards and olive groves or drool over boutiques in fashionable Milan. Look for mermaids from the cliffs of Sorrento or nibble on Parma's famous cheese and prosciutto. Anywhere you choose, the country will captivate you.	View	
Mexico	With 26 UNESCO-declared world heritage sites, charming colonial towns and dozens of thrilling cities, there's plenty to explore in this country of 109 million. Outside the cities, stunning Pacific beaches, stark deserts, mangrove swamps and swimming holes provide all you need for a relaxing, romantic or adventurous vacation. Captivating, cosmopolitan and chaotic Mexico City and the 32 states offer an incredible abundance of experiences, from laid-back and leisurely to upbeat to adrenalin-charging.	View	
Australia	Australia is as varied as it is enormous-- it is home to thriving cities, vast wilderness, and earnest, friendly people. Sydney is for sophisticates, with its famous opera house and annual Sculpture By the Sea outdoor art exhibits. Port Douglas makes a great base for snorkeling or diving the Great Barrier Reef, and quaint lighthouses keep vigil along the beaches of Byron Bay. Melbourne is a hub of rousing cricket activity, and the limestone pillars of the Pinnacles Desert are a natural wonder.	View	

[Previous Page](#) [Next Page](#)

This is exactly what was expected (although still with work to be done regarding an Add to holiday button), with each of the columns displaying the right contents, as well as the web URL having the TripAdvisor link as shown below:

Mexico	With 26 UNESCO-declared world heritage sites, charming colonial towns and dozens of thrilling cities, there's plenty to explore in this country of 109 million. Outside the cities, stunning Pacific beaches, stark deserts, mangrove swamps and swimming holes provide all you need for a relaxing, romantic or adventurous vacation. Captivating, cosmopolitan and chaotic Mexico City and the 32 states offer an incredible abundance of experiences, from laid-back and leisurely to upbeat to adrenalin-charging.	View
Australia	Australia is as varied as it is enormous-- it is home to thriving cities, vast wilderness, and earnest, friendly people. Sydney is for sophisticates, with its famous opera house and annual Sculpture By the Sea outdoor art exhibits. Port Douglas makes a great base for snorkeling or diving the Great Barrier Reef, and quaint lighthouses keep vigil along the beaches of Byron Bay. Melbourne is a hub of rousing cricket activity, and the limestone pillars of the Pinnacles Desert are a natural wonder.	View

[Previous Page](#) [Next Page](#)

<https://www.tripadvisor.com/Tourism-g150768-Mexico-Vacations.html?m=66827>

However, when going to the next page, some of the results seem to be duplicated in the list, implying that the check made on lines 86 and 87 of page 66 isn't effective.

Based on your preferences, here's what we found:

Filter

Name	Description	Web URL	Image
China	The grand story of China features a great wall, a forbidden city, and a few thousand Terracotta warriors. Settings include the sci-fi skyline of Shanghai, the jutting peaks of Zhangjiajie, and the tropical beaches of Hainan, but the tale of this ancient kingdom began long ago.	View	
Japan	In the Land of the Rising Sun, ancient temples sit alongside neon wonderlands and shinto shrines offer pockets of peace amid metropolises. Add tea ceremonies, snow monkeys, sushi, kimonos, and karaoke to the mix, and you've got one of the world's most fascinating countries.	View	
Japan	In the Land of the Rising Sun, ancient temples sit alongside neon wonderlands and shinto shrines offer pockets of peace amid metropolises. Add tea ceremonies, snow monkeys, sushi, kimonos, and karaoke to the mix, and you've got one of the world's most fascinating countries.	View	
France	There's much more to France than Paris—from the fairy-tale châteaux of the Loire Valley to the lavender fields of Provence, and the French Riviera's celebrity-studded beaches. World-famous gastronomy and fine wines provide the perfect complement to the country's alpine views and architectural masterpieces.	View	

[Previous Page](#) [Next Page](#)

Looking back over my code, I saw that only in the if statement did I actually write to the dictionary, but in the else part of this statement I only incremented score to find a unique value rather than putting the corresponding destination in the dictionary. I fixed this just by adding in line 88.

```

83  if score not in scoreDict: #If the score is not already in the dictionary
84      scoreDict[score] = each[0] #Adds the score and country to the dictionary
85  else: #If the score is already in the dictionary
86      while scoreDict[score] in scoreDict: #Makes sure that it doesn't increment to a
87          score += 0.01 #Increments the score by a small amount
88      scoreDict[score] = each[0] #Adds the score and country to the dictionary
89      ranking.append(score) #Adds that country's scr to the ranking array
90      ranking.sort() #Sorts the ranking array

```

This resulted in the converse issue where now the destination that was hidden is now duplicated, and the previously duplicated destination is now entirely hidden. This also seemed to be consistent across later pages, although no issues seem to arise on page 1 once more.

Name	Description	Web URL	Image
China	The grand story of China features a great wall, a forbidden city, and a few thousand Terracotta warriors. Settings include the sci-fi skyline of Shanghai, the jutting peaks of Zhangjiajie, and the tropical beaches of Hainan, but the tale of this ancient kingdom began long ago.	View	
Thailand	The lush jungles of Thailand promise adventure, while the serene beaches are the perfect place to splash in the sun. The Similan Islands feature some of the best dive sites in the world, where barracuda dart amid coral reefs and rock formations. Party in the nightclubs of Patong or linger over mango sticky rice at the famous Bangkok family restaurant Kao Neoo Korpanich. The city of Chiang Mai is an absolute must-visit; a mosaic of ancient temples, massage parlors, music venues, and markets.	View	
Thailand	The lush jungles of Thailand promise adventure, while the serene beaches are the perfect place to splash in the sun. The Similan Islands feature some of the best dive sites in the world, where barracuda dart amid coral reefs and rock formations. Party in the nightclubs of Patong or linger over mango sticky rice at the famous Bangkok family restaurant Kao Neoo Korpanich. The city of Chiang Mai is an absolute must-visit; a mosaic of ancient temples, massage parlors, music venues, and markets.	View	
France	There's much more to France than Paris—from the fairy-tale châteaux of the Loire Valley to the lavender fields of Provence, and the French Riviera's celebrity-studded beaches. World-famous gastronomy and fine wines provide the perfect complement to the country's alpine views and architectural masterpieces.	View	

[Previous Page](#) [Next Page](#)

I think the error lay in line 86 where `scoreDict[score]` returns a destination which is always going to be in `scoreDict` regardless of score changing causing an infinite, non-progressing loop.

```

85  else: #If the score is already in the dictionary
86  while scoreDict[score] in scoreDict: #Makes sure that it doesn't increment to another score
87      score += 0.01 #Increments the score by a small amount
88      scoreDict[score] = each[0] #Adds the score and country to the dictionary

```

To correct this, I changed line 86 to below:

```

84      scoreDict[score] = each[0] #Adds the score and country to the dictionary
85  else: #If the score is already in the dictionary
86  while score in scoreDict: #Makes sure that it doesn't increment to another score
87      score += 0.01 #Increments the score by a small amount
88      scoreDict[score] = each[0] #Adds the score and country to the dictionary

```

This fixed the problem, and also sees the webpage load quicker, being consistent with how an infinite loop had been prevented.

Based on your preferences, here's what we found:

Filter

Name	Description	Web URL	Image
China	The grand story of China features a great wall, a forbidden city, and a few thousand Terracotta warriors. Settings include the sci-fi skyline of Shanghai, the jutting peaks of Zhangjiajie, and the tropical beaches of Hainan, but the tale of this ancient kingdom began long ago.	View	
Thailand	The lush jungles of Thailand promise adventure, while the serene beaches are the perfect place to splash in the sun. The Similan Islands feature some of the best dive sites in the world, where barracuda dart amid coral reefs and rock formations. Party in the nightclubs of Patong or linger over mango sticky rice at the famous Bangkok family restaurant Kao Neoo Korpanich. The city of Chiang Mai is an absolute must-visit; a mosaic of ancient temples, massage parlors, music venues, and markets.	View	
Japan	In the Land of the Rising Sun, ancient temples sit alongside neon wonderlands and shinto shrines offer pockets of peace amid metropolises. Add tea ceremonies, snow monkeys, sushi, kimonos, and karaoke to the mix, and you've got one of the world's most fascinating countries.	View	
France	There's much more to France than Paris—from the fairy-tale châteaux of the Loire Valley to the lavender fields of Provence, and the French Riviera's celebrity-studded beaches. World-famous gastronomy and fine wines provide the perfect complement to the country's alpine views and architectural masterpieces.	View	

[Previous Page](#) [Next Page](#)

At this point, I wanted to work on the filter that would allow the search to be more selective, however the endpoints that I wanted to use for this didn't give reviews and distance for a specific country. I figured it would be worth compromising that, partly due to the large effort that would be required to do that, and also due to the stakeholder's intention of keeping this website linked with the main blog. This suggests that websites that are linked with a main blog need to be accurate in order to maintain the integrity of the initial blog by association. Due to this, I thought it would be better to not try and aggregate entire countries under one filter at a time and would be better to scrap the functionality entirely.

This made me remove the JavaScript from dashboard.html as well as the HTML that is displayed with the filter menu.

The next progression within this success criteria was to add a button on the end of each row of destinations that allows a holiday to be created at that location. This process began with updating dashboard.html to add this button:

```

27 </thead>
28 <tbody>
29     {% for dest in page_items %}
30     <tr>
31         <td>{{ dest.name }}</td>
32         <td>{{ dest.description }}</td>
33         <td><a href="{{ dest.web_url }}" target="_blank">View</a></td>
34         <!--target="_blank" opens the link in a new tab-->
35         <td>
36             {% if dest.image_url %}
37                 
38             {% else %}
39                 No Image
40             {% endif %}
41         </td>
42         <td>
43             <form action="/plan" method="GET">
44                 <input type="hidden" name="destination" value="{{ dest.name }}">
45                 <button type="submit" class="btn-plan">Plan a holiday here</button>
46             </form>
47         </td>
48     </tr>
49     {% endfor %}
50 </tbody>
51 </table>
52
53
54 {% if prev_page %}
55 <a class="btn-page" href="{{ url_for('dashboard', page=prev_page) }}">Previous Page</a>
56 {% endif %}

```

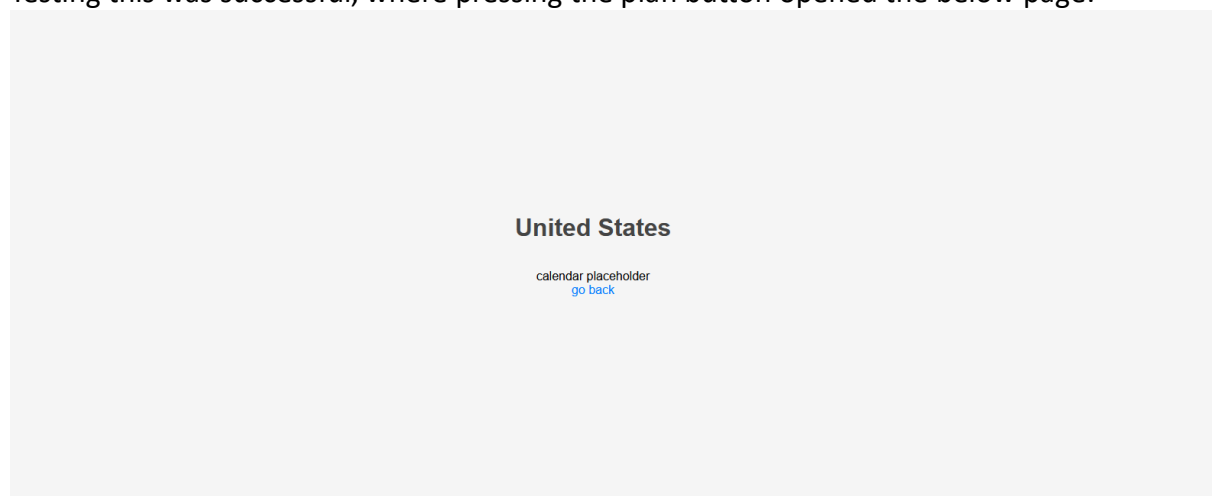
To create a “Plan a holiday here” button, first I made a placeholder plan.html site, and an app.route(/plan) in app.py that directs to plan.html taking the chosen destination as the parameter. This has been implemented below:

```

121
122 @app.route('/plan', methods=['GET']) #Run for www.tripmaestro.com/plan
123 def plan_holiday():
124     destination = request.args.get('destination') # Get the destination name from the request
125     if destination is None: #If the destination is not specified
126         return redirect(url_for('dashboard'))
127     return render_template('plan.html', destination=destination) #Render the plan.html template with the destination name
128

```

Testing this was successful, where pressing the plan button opened the below page:



During testing, I noticed that I was able to open the plan.html straight from a link so long as I included a destination in the URL. This contradicted with how the original design was meant to see each holiday planned be attributed to the user currently logged in. This prompted me to create some verification that the user is logged in, using the session module from Flask:

```

122 @app.route('/plan', methods=['GET']) #Run for www.tripmaestro.com/plan
123 def plan_holiday():
124     if 'user_id' not in session:
125         return redirect(url_for('login')) # Redirect to the login page
126     destination = request.args.get('destination') # Get the destination name from the request
127     if destination is None: #If the destination is not specified
128         return redirect(url_for('dashboard'))
129     return render_template('plan.html', destination=destination) #Render the plan.html template with the destination name

```

(additions made on lines 124 and 125)

This completed the first success criteria of page 8, with the list of destinations appearing with a description being displayed to the user to guide their choice, as well as a link to an external website to find out more.

The next success criteria to be worked on is the 4th point:

When the user decides on their destination, they are prompted with different categories of activities to select from. When opening these categories, the user is given a prompt to select how much time should be allocated for that. This time is then represented on a block on a calendar for the user to drag for when they want it.

This would first require developing plan.html:

```

1 <html>
2 <head>
3     <link rel="stylesheet" href="{{ url_for('static', filename='plan.css') }}">
4     <title>Holiday Planner</title>
5 </head>
6 <body>
7     <header class="planner-header">
8         <h1>Holiday Planner</h1>
9         <a href="/logout" class="small-text">Sign out</a>
10    </header>
11
12    <div class="settings">
13        <h2>
14            <label for="holiday-title">Title:</label>
15            <input type="text" id="holiday-title" name="holiday-title" value="My Holiday" placeholder="Enter a title">
16        </h2>
17        <p class="small-text">
18            First day of the week:
19            <select name="first-day" id="first-day">
20                <option value="Monday">Monday (Default)</option>
21                <option value="Sunday">Sunday</option>
22            </select>
23        </p>
24    </div>
25
26    <div class="calendar-container">
27        <div class="time-row-header"></div>
28        <div class="day-header">Monday</div>
29        <div class="day-header">Tuesday</div>
30        <div class="day-header">Wednesday</div>
31        <div class="day-header">Thursday</div>
32        <div class="day-header">Friday</div>
33        <div class="day-header">Saturday</div>
34        <div class="day-header">Sunday</div>
35    </div>

```

```

36     {% for hour in range(7, 23) %}
37         <div class="time-row-header">{{ "%02d:00" | format(hour) }}</div>
38     {% for day in range(7) %}
39         <div class="calendar-cell" data-day="{{ day }}" data-hour="{{ hour }}"></div>
40     {% endfor %}
41 {% endfor %}
42 </div>
43
44 <form action="/save_holiday" method="POST">
45     <input type="hidden" id="calendar-data" name="calendar-data" value="">
46     <button type="submit" class="save-button">Save</button>
47 </form>
48
49 <p class="footer-note">Toggle hidden hours: 11 PM → 7 AM</p>
50

```

A Jinja loop is used to create rows for each hour in the calendar rather than having to manually add each hour of the day. I realised here though that the user still hasn't had an opportunity to indicate how long their holiday would be and when it would start/end. This led me to create an intermediary page of `new_holiday.html` to go between the dashboard and `calendar.html` (renamed from `plan.html`) that would minimise the clutter for the user on the calendar page.

```

1  <html>
2  <head>
3      <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
4      <title>Create Holiday</title>
5  </head>
6  <body>
7      <header class="planner-header">
8          <h1>Plan Your Holiday</h1>
9          <a href="/dashboard" class="small-text">Back to Dashboard</a>
10     </header>
11
12     <main class="setup-form">
13         <form action="/new_holiday" method="POST">
14             <div class="form-group">
15                 <label for="holiday-title">Holiday Title:</label>
16                 <input type="text" id="holiday-title" name="holiday-title" placeholder="Enter a title for your holiday" required>
17             </div>
18
19             <div class="form-group">
20                 <label for="start-date">Start Date:</label>
21                 <input type="date" id="start-date" name="start-date" required>
22             </div>
23
24             <div class="form-group">
25                 <label for="end-date">End Date:</label>
26                 <input type="date" id="end-date" name="end-date" required>
27             </div>
28
29             <button type="submit" class="save-button">Continue to Planner</button>
30         </form>
31     </main>
32 </body>
33 </html>
34

```

This is what was made for `new_holiday.html` and contains a form that posts to the `/new_holiday` route with the date and title of the holiday.

When updating app.py to accommodate for this, the decision has to be made whether to store these holiday details in the user session, or the database. I decided to not store every detail in the session as if the user would want to edit another holiday during the same session, it might be very tedious to try and overwrite everything already in the session. This is why I am opting for the session to store the primary key of the holiday's record in the database so only that needs to be changed in order to start working on a new holiday.

I also decided to remake the database system, now combining the tables in destinations.db and users.db to all be in one file users.db. This is because the table I envision for each holiday would require a user and a destination for each, meaning it needs to be able to create a relationship with those tables in order to use those as foreign keys.

Rebuilding the database:

```
SQL 1* x
1 CREATE TABLE countries (
2     name VARCHAR(100) NOT NULL PRIMARY KEY,
3     cuisine_score INTEGER NOT NULL,
4     peace_score INTEGER NOT NULL,
5     activities_score INTEGER NOT NULL,
6 );

1 INSERT INTO countries (name, cuisine_score, peace_score, activities_score) VALUES
2 ('Italy', 1000, 472, 880),
3 ('Japan', 980, 420, 850),
4 ('India', 970, 392, 780),
5 ('Lebanon', 960, 349, 520),
6 ('Thailand', 950, 506, 790),
7 ('Mexico', 940, 581, 810),
8 ('China', 930, 501, 840),
9 ('Turkey', 920, 531, 760),
10 ('Greece', 910, 563, 740),
11 ('Vietnam', 900, 429, 580),
12 ('France', 890, 518, 830),
13 ('Spain', 880, 544, 800),
14 ('USA', 870, 644, 860),
15 ('South Korea', 860, 376, 550),
16 ('Brazil', 850, 679, 670),
```

Changing reference in Python to the database:

```

72     destConnection = sqlite3.connect('users.db')
73     destCursor = destConnection.cursor()
74     destCursor.execute("SELECT * FROM countries")
75     countries = destCursor.fetchall() #Makes a complete array of
76     destConnection.close()

```

Creating an “activities” table, in which entries into the calendar are stored. They hold userID and holidayID as foreign keys, and activityID as a primary key. They also hold other information about the activity such as start and end time, and also any notes that the user would want to write about it, thinking ahead to meet the 6th success criteria.

To make this useful in app.py, when the form on new_holiday.html is completed, a new record is to be made in the holidays table containing this data. To create a complete record, the Python file needs to have the destination, and the easiest solution I was able to get for this was to create a hidden input that submits the value of the destination along with the relevant dates:

```

<form action="/new_holiday" method="POST">
  <!-- Hidden input to pass the destination -->
  <input type="hidden" name="destination" value="{{ destination }}">

```

Then to make the record in app.py:

```

194 @app.route('/new_holiday', methods=['POST'])
195 def new_holiday():
196     destination = request.form['destination']
197     startDate = request.form['startDate']
198     endDate = request.form['endDate']
199     userID = session['user_id'] #Gets all relevant data for holiday record
200     holidayConnection = sqlite3.connect('users.db')
201     holidayCursor = holidayConnection.cursor()
202     holidayCursor.execute("INSERT INTO holidays (destination, startDate, endDate, userID) VALUES (?, ?, ?, ?)",
203                           (destination, startDate, endDate, userID))
204     holidayConnection.commit()
205     holidayConnection.close()
206

```

When pressing ‘Plan a holiday to here’ on the dashboard, I was met with the page for the calendar, suggesting that the previous button wasn’t appropriately linked with the new_holiday.html page.

This was fixed by changing line 43 in dashboard.html:

```

40     {% endif %}
41     </td>
42     <td>
43         <form action="/new_holiday" method="GET">
44             <input type="hidden" name="destination" value="{{ dest.name }}">
45             <button type="submit" class="btn-plan">Plan a holiday here</button>
46         </form>

```

When testing this again, I saw the following page:

Method Not Allowed

The method is not allowed for the requested URL.

I eventually realised that my logic for the Python was flawed, in that this route was meant to be where the form for the new holiday was to be filled in, whereas having the 'POST' method, and all of the requests from the form couldn't be possible as that was meant to come from this site. I fixed this by moving the code at the bottom of page 77 to a new route for calendar.html, and simply made /new_holiday return new_holiday.html:

```
185 @app.route('/new_holiday', methods=['GET', 'POST']) #Expects form submission so needs both GET and POST methods
186 def new_holiday():
187     if 'user_id' not in session:
188         return redirect(url_for('login'))
189     if request.method == 'POST': #If the form has been submitted
190         destination = request.form['destination']
191         startDate = request.form['startDate']
192         endDate = request.form['endDate']
193         holiday_title = request.form['holiday-title']
194         userID = session['user_id'] #Gets all data to be stored in database
195         if not destination or not startDate or not endDate or not holiday_title: #If any fields are empty
196             flash('Please fill in all fields')
197             return render_template('new_holiday.html', destination=destination) #Sends user back to the form
198
199         holidayConnection = sqlite3.connect('users.db')
200         holidayCursor = holidayConnection.cursor()
201         holidayCursor.execute("INSERT INTO holidays (destination, startDate, endDate, userID, holidayTitle ) VALUES (?, ?, ?, ?, ?)",
202                                (destination, startDate, endDate, userID, holiday_title)) #Inserts data into database
203         holidayID = holidayCursor.lastrowid #Gets the primary key of the user's holiday
204         holidayConnection.commit()
205         holidayConnection.close()
206         return redirect(url_for('calendar', holidayID = holidayID)) #Redirects user to the calendar page
207     destination = request.args.get('destination') #Gets the destination from the URL if form not submitted
208     if not destination: #If no destination is in the URL
209         return redirect(url_for('dashboard')) #Redirects user to the dashboard
210     return render_template('new_holiday.html', destination=destination) #If destination is in the URL, renders the new holiday page
211
212 @app.route('/calendar', methods=['GET', 'POST'])
213 def calendar():
214     holidayID = request.args.get('holidayID')
215     print(holidayID) #To test if the holidayID is being passed correctly
216     if 'user_id' not in session:
217         return redirect(url_for('login'))
218     else:
219         return render_template('dashboard.html')
220
```

I rewrote the new_holiday() function to make sure that form submission had its intended consequences, compared to just opening the page (using the GET method).

The calendar() function is shown below as a placeholder to see if the required holidayID value is passed through to aid future work on that holiday.

It was important to validate here that the form was actually filled in, and was checked on line 195, where otherwise the user was told to back to fill in all of the fields.

Testing this, I was able to get to the following page:

Plan Your Holiday

[Back to Dashboard](#)

Destination: United States

Holiday Title:

Start Date:

End Date:

[Continue to Planner](#)

However, pressing the button to continue to planner saw the following error:

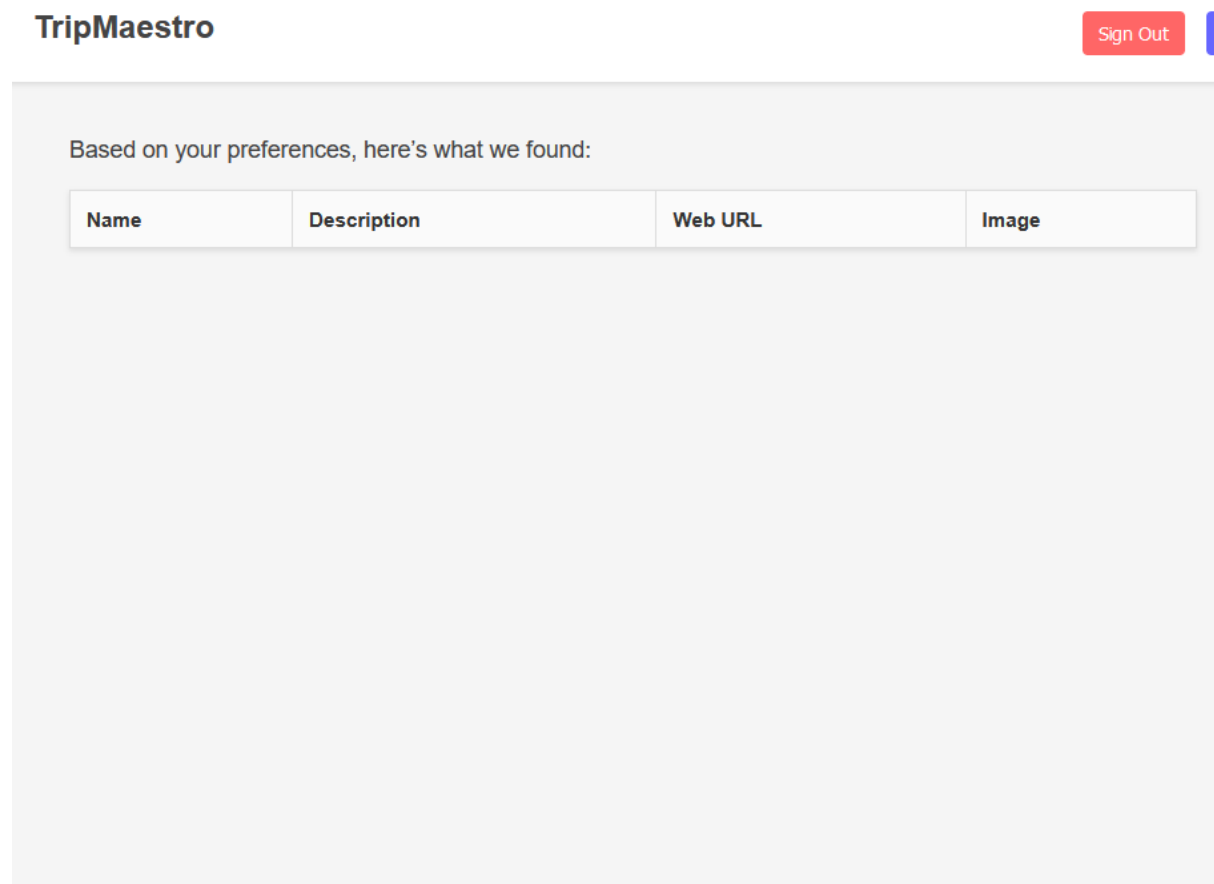
BadRequestKeyError

```
werkzeug.exceptions.BadRequestKeyError: 400 Bad Request: The browser (or proxy) sent a request that this
server could not understand.
KeyError: 'startDate'
```

Traceback (most recent call last)

```
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
File "c:\Users\mabed\Documents\trip_maestro\app.py", line 191, in new_holiday
    startDate = request.form['startDate']
```


The terminal displayed the holidayID (1) showing that the program had reached the calendar() function, however the webpage displayed is shown below:



This is what was displayed for dashboard.html, implying that the wrong HTML template was being displayed.

Reviewing calendar() showed me that the else statement on line 218 directs the user to dashboard.html, so modifying this to calendar.html was the next correction made.

```
212 @app.route('/calendar', methods=['GET', 'POST'])
213 def calendar():
214     holidayID = request.args.get('holidayID')
215     print(holidayID) #To test if the holidayID is being passed correctly
216     if 'user_id' not in session:
217         return redirect(url_for('login'))
218     else:
219         return render_template('dashboard.html')
220
```

Correcting this gave me the expected result of the calendar appearing:

Holiday Planner

Title:

First day of the week:

	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
07:00							
08:00							
09:00							
10:00							
11:00							
12:00							
13:00							
14:00							
15:00							
16:00							
17:00							
18:00							
19:00							
20:00							
21:00							
22:00							

Save

When testing an end date earlier than the start date though, the website allowed me to continue to the calendar even though the date shouldn't work, prompting me to add validation to ensure that the ending date is always later than the start date:

```
if endDate < startDate: #If the end date is before the start date
    flash('End date must be after start date')
    return render_template('new_holiday.html', destination=destination) #Sends user back to the form
```

However, I hadn't setup the flash yet in new_holiday.html, so no message was appearing to the user.

```
19 <div>
20     {% for message in get_flashed_messages() %}
21     <div>{{ message }}</div>
22     {% endfor %}
23 </div>
```

Adding this block in new_holiday.html gave the following result:

The screenshot shows a web form titled "Plan Your Holiday" on a light gray background. At the top, there is a blue link "Back to Dashboard". Below it, the text "Destination: United States" is displayed. A message "End date must be after start date" is shown in red. The form itself is a white card with a shadow. It contains a label "Holiday Title:" followed by a text input field with the placeholder "Enter a title for your holiday". Below this is a label "Start Date:" followed by a date input field showing "dd / mm / yyyy" and a calendar icon. Then, a label "End Date:" is followed by another date input field showing "dd / mm / yyyy" and a calendar icon. At the bottom of the card is a blue button with the text "Continue to Planner".

I also decided to create a function that checks if a user is signed in:

```
96 def checkUserLoggedIn():
97     if 'user_id' not in session: #If the user is not logged in
98         return redirect(url_for('login')) #Redirects user to login page
```

When testing this however, I realised that this wouldn't actually redirect the user as it is returned in a function that any original function wouldn't use. This made me revert this change as at that point I'm not actually making the code more efficient.

I also made the log out button across the pages functional:

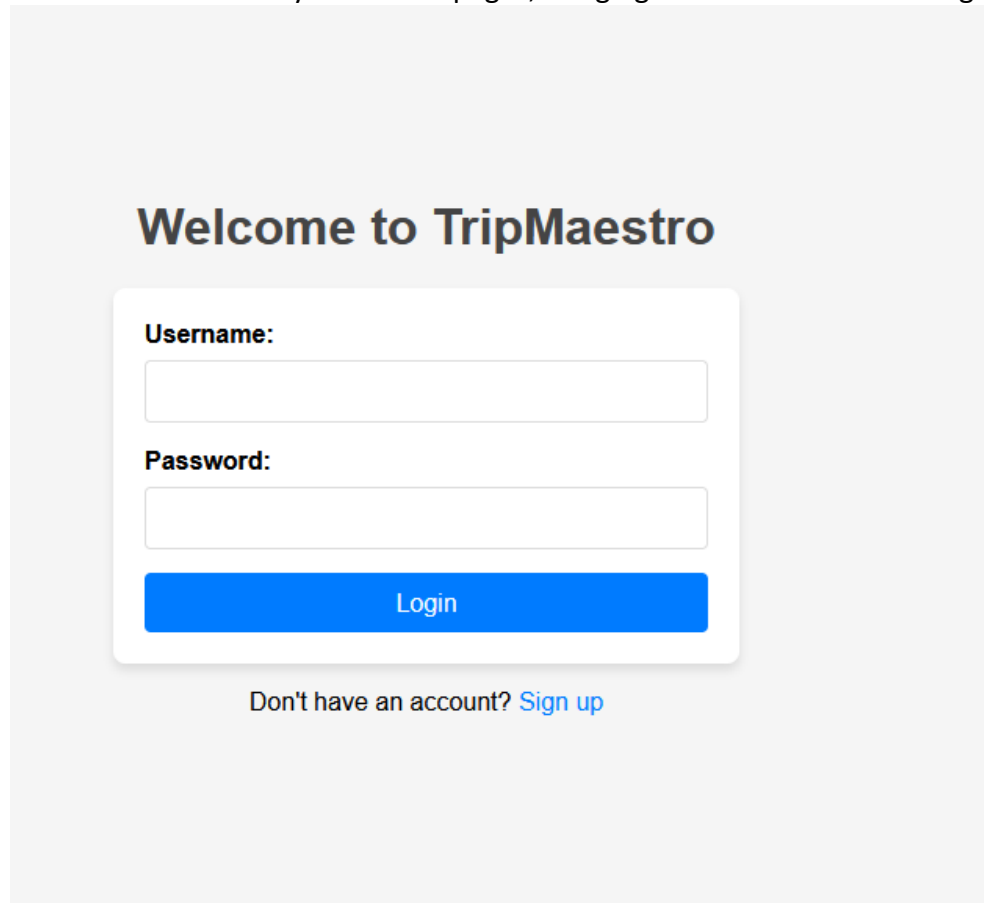
This involved adding this to each page of the website in which a user would be signed in:

```
11 <div class="header-right">
12     <a href="/logout" class="btn-signout">Sign Out</a>
13     <button class="btn-profile">Profile</button>
14 </div>
```

In terms of additions to Python, the /logout route was created that cleared the session:

```
154 @app.route('/logout')
155 def logout():
156     session.clear() #Clears all details held about user
157     return redirect(url_for('login'))
158
```

This works successfully across the pages, bringing the user back to the original login page:



Welcome to TripMaestro

Username:

Password:

Login

Don't have an account? [Sign up](#)

To make the calendar properly encompass a holiday, I worked on pagination for the calendar, so there would be as many pages of the calendar as there would need to be for the entire duration of the holiday that the user entered at new_holiday.html.

This required the datetime library, specifically importing datetime and timedelta to allow the difference between the beginning and end of the weeks of the holiday to be manipulated.

I began by obtaining the start and end dates of the holiday from the database using the holidayID. I calculated the number of days and complete weeks that the holiday would last, so the calendar knows how many to display.

```

214 @app.route('/calendar', methods=['GET', 'POST'])
215 def calendar():
216     holidayID = request.args.get('holidayID', -1) #Get the holiday ID from query parameters
217     if 'user_id' not in session:
218         return redirect(url_for('login'))
219     if holidayID == -1:
220         return redirect(url_for('dashboard'))
221     dateConnection = sqlite3.connect('users.db')
222     dateCursor = dateConnection.cursor()
223     dateCursor.execute("SELECT startDate, endDate FROM holidays WHERE id = ?", (holidayID,))
224     dates = dateCursor.fetchone() #Fetches row of holiday and its dates
225     start_date = dates[0] #Get the start date of the holiday
226     end_date = dates[1]
227     start_date = datetime.strptime(start_date, '%Y-%m-%d') #Converts the start date to a datetime object
228     end_date = datetime.strptime(end_date, '%Y-%m-%d') #Converts the end date to a datetime object
229
230     #Calculate no. of weeks of holiday
231     num_days = (end_date - start_date).days + 1
232     num_weeks = (num_days + 6) // 7

```

To find the current week of any page on the calendar, I obtained it from the URL, but had to validate that it was reasonable given the duration of the holiday:

```

249     current_week = int(request.args.get('week_number', 1))
250
251     #Validates the selected page (current_week)
252     if current_week < 1:
253         current_week = 1 #Redirects to the first week
254         return redirect(url_for('calendar', holidayID=holidayID, start_date=start_date.strftime('%Y-%m-%d'),
255                                     end_date=end_date.strftime('%Y-%m-%d'), week_number=current_week))
256
257     if current_week > num_weeks:
258         current_week = num_weeks #Redirects to the final week
259         return redirect(url_for('calendar', holidayID=holidayID, start_date=start_date.strftime('%Y-%m-%d'),
260                                     end_date=end_date.strftime('%Y-%m-%d'), week_number=current_week))
261

```

The range of dates for any given week is then calculated making sure the end of the week isn't shown if it isn't actually during the holiday period. This is then returned as a template to the user with all of the necessary parameters.

```

247     #Calculate the date range for the selected week
248     week_start = start_date + timedelta(days=(current_week - 1) * 7)
249     temp_week_end = week_start + timedelta(days=6)
250
251     if temp_week_end > end_date: #If the end of the week is after the end of the holiday
252         week_end = end_date #Set the week's end to be at the end of the holiday
253     else:
254         week_end = temp_week_end #Set week's end to be the calculated end of the week
255
256     return render_template( #Renders the calendar page
257         'calendar.html',
258         holidayID=holidayID,
259         num_weeks=num_weeks,
260         current_week=current_week,
261         start_date=start_date.strftime('%Y-%m-%d'),
262         end_date=end_date.strftime('%Y-%m-%d'),
263         selected_week_start=week_start.strftime('%Y-%m-%d'),
264         selected_week_end=week_end.strftime('%Y-%m-%d'),
265         #Passes all the variables to the HTML file
266     )

```

I then had to modify calendar.html to make use of the dates that were calculated in Python. I began by displaying the week and what dates it contained, to inform the user what they were looking at, and partly also as a debugging tool to show me what the website is trying to display.

```

22 <div class="week-info">
23 |   <p>Week {{ current_week }}: {{ selected_week_start }} to {{ selected_week_end }}</p>
24 </div>

```

I also had to create the page system, which required passing all of the values back through as these would also be needed in future pages for their own contents.

```

50 <div class="pagination">
51 |   {% if current_week > 1 %}
52 |   <a href="{{ url_for('calendar', holidayID=holidayID, week_number=current_week - 1, start_date=start_date, end_date=end_date) }}" class="pagination-link">Previous Week</a>
53 |   {% endif %}
54 |   {% if current_week < num_weeks %}
55 |   <a href="{{ url_for('calendar', holidayID=holidayID, week_number=current_week + 1, start_date=start_date, end_date=end_date) }}" class="pagination-link">Next Week</a>
56 |   {% endif %}
57 </div>
58

```

Testing this gave me the following error:

OperationalError

sqlite3.OperationalError: no such column: id

Traceback (most recent call last)

```

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
    ~~~~~^~~~~~

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
    ~~~~~^~~~~~

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
    ~~~~~^~~~~~

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
    ~~~^~~~~~

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
    ~~~^~~~~~

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
    ~~~~~^~~~~~

File "c:\Users\mabed\Documents\trip_maestro\app.py", line 223, in calendar
    dateCursor.execute("SELECT startDate, endDate FROM holidays WHERE id = ?", (holidayID,))
    ~~~~~^~~~~~

```

sqlite3.OperationalError: no such column: id

This made sense as the column in the actual table is called holidayID yet I referred to it as just "id".

After correcting this, the calendar functions entirely as expected, with the first page being shown below. The “Previous Week” button has disappeared as expected due to this being the first week.

Holiday Planner

Week 1: 2025-01-11 to 2025-01-17

Title: My Holiday

	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
07:00							
08:00							
09:00							
10:00							
11:00							
12:00							
13:00							
14:00							
15:00							
16:00							
17:00							
18:00							
19:00							
20:00							
21:00							
22:00							

Next Week

(Apologies for low quality) Going to the end of the holiday also works as expected where the top left shows that the webpage has realised that the holiday ends on the 26th, and that this week will only last 2 days on the calendar. Moreover, the ‘Next Page’ button is hidden.

Holiday Planner

Week 3: 2025-01-25 to 2025-01-26

Title: My Holiday

	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
07:00							
08:00							
09:00							
10:00							
11:00							
12:00							
13:00							
14:00							
15:00							
16:00							
17:00							
18:00							
19:00							
20:00							
21:00							
22:00							

Previous Week

However, I also realised here that the dates shown at the top weren't guaranteed to be Monday to Sunday, and the user had no indication on the date of each of the days in the week. This can obviously be a problem to the user where they aren't aware which days on the calendar are actually in their holiday range and which are outside of it.

To fix this, I modified `calendar()` in `app.py` to calculate the first Monday of the week in which the holiday starts.

```

256     monday_date = start_date #Sets the first day of the week to be the start date
257     for i in range(7): #Loops through 7 days of the week, one of which must be Monday
258         if monday_date.weekday() == 0: #If the day is Monday
259             break #Ends the loop when monday_date is Monday
260         else:
261             monday_date = monday_date - timedelta(days=1) #Subtracts a day from the date to find Monday
262
263     return render_template( #Renders the calendar page
264         'calendar.html',
265         holidayID=holidayID,
266         num_weeks=num_weeks,
267         current_week=current_week,
268         start_date=start_date.strftime('%Y-%m-%d'),
269         end_date=end_date.strftime('%Y-%m-%d'),
270         selected_week_start=week_start.strftime('%Y-%m-%d'),
271         selected_week_end=week_end.strftime('%Y-%m-%d'),
272         monday_date=monday_date.strftime('%Y-%m-%d')
273     ) #Passes all the variables to the HTML file

```

I added lines 256 to 261 as well as included line 272 to pass the first Monday to `calendar.html`. Replacing the headers for each day in the calendar required me to use Jinja2 to take in the `monday_date` and then create a `current_date` that cycles through for each day to provide the date for each day of the week.

```

36 <div class="calendar-container">
37     <div class="time-row-header"></div>
38     {% set current_date = monday_date | datetime.strptime('%Y-%m-%d') %}
39
40     {% for day in range(7) %}
41         <div class="day-header">
42             {{ current_date.strftime('%A') }}<br>
43             {{ current_date.strftime('%Y-%m-%d') }}
44         </div>
45         {% set current_date = current_date + timedelta(days=1) %}
46     {% endfor %}
47

```

This gave me the following error though when tested:

TemplateAssertionError

`jinja2.exceptions.TemplateAssertionError: No filter named 'datetime.strptime'.`

Traceback (most recent call last)

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in `__call__`

```

return self.wsgi_app(environ, start_response)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

I identified a significant problem in my code that could cause this, being how the `monday_date` that I had passed into `calendar.html` was passed as a string rather than a datetime object, meaning `".strftime()"` wasn't able to work.

Adding this line gave me the following results:

```
262     monday_date = datetime.strptime(monday_date, '%Y-%m-%d') #Converts the date to a datetime object
```

UndefinedError

`jinja2.exceptions.UndefinedError: 'timedelta' is undefined`

Traceback (most recent call last)

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in `__call__`

```
    return self.wsgi_app(environ, start_response)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1514, in `wsgi_app`

```
    response = self.handle_exception(e)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

I realised here that my whole approach to this was over the top and unnecessary. There would be no need to sort the days of the holiday into weeks if they could all be displayed on one page, and the Python can just handle text going into the HTML rather than rely on Jinja2 to do its own calculations.

This meant I was able to remove all of the pagination, but this did have some utility in how it prevented the effects of a 100 year holiday in consuming system resources. Although the stakeholder didn't require an emphasis on optimisation, this could grind the whole page to a halt, so to cover this I also now plan to add a limit to the holiday length.

New calendar route in `app.py`:

```
214 @app.route('/calendar', methods=['GET', 'POST'])
215 def calendar():
216     holidayID = request.args.get('holidayID', -1) # Get the holiday ID from query parameters
217     if 'user_id' not in session:
218         return redirect(url_for('login'))
219     if holidayID == -1:
220         return redirect(url_for('dashboard'))
221
222     # Fetch holiday start and end dates
223     dateConnection = sqlite3.connect('users.db')
224     dateCursor = dateConnection.cursor()
225     dateCursor.execute("SELECT startDate, endDate FROM holidays WHERE holidayID = ?", (holidayID,))
226     dates = dateCursor.fetchone()
227     dateConnection.close()
228
229     start_date = datetime.strptime(dates[0], '%Y-%m-%d') # Takes start date and converts it to a datetime object
230     end_date = datetime.strptime(dates[1], '%Y-%m-%d') # "" with end date
231
232     # Generates a list of all dates for the holiday
233     all_dates = []
234     for i in range((end_date - start_date).days + 1):
235         all_dates.append((start_date + timedelta(days=i)).strftime('%A, %Y-%m-%d'))
236
237     # Pass the full list of dates to the template
238     return render_template(
239         'calendar.html',
240         holidayID=holidayID,
241         all_dates=all_dates, # Send the list of all dates
242     )
243
```

New calendar container in calendar.html:

```

36 <div class="calendar-container">
37   <div class="day-row-header"></div>
38   {% for hour in range(7, 23) %}
39     <div class="time-slot-header">{{ "%02d:00" | format(hour) }}</div>
40   {% endfor %}
41
42   {% for date in all_dates %}
43     <div class="day-row-header">{{ date }}</div>
44     {% for hour in range(7, 23) %}
45       <div class="calendar-cell" data-date="{{ date }}" data-hour="{{ hour }}"></div>
46     {% endfor %}
47   {% endfor %}
48 </div>
49

```

I also at this point made an empty button to redirect the user to a page where they can add a new activity.

Opening the calendar webpage to test for this gave the following result:

TripMaestro

Enter a title to save holiday

Week : to

Add New Activity

	07:00 14:00 22:00	08:00 15:00 16:00	09:00 17:00	10:00 18:00	11:00 19:00	12:00 20:00	13:00 21:00
Thursday, 2025-01-09							
Friday, 2025-01-10							
Saturday, 2025-01-11							
Sunday, 2025-01-12							
Monday, 2025-01-13							
Tuesday, 2025-01-14							
Wednesday, 2025-01-15							

Many things are going wrong here, but an easy fix to make is above the calendar where it is trying to show the dates in which the week lie. This isn't required anymore due to the page displaying the entire calendar so this component can be removed from calendar.html. I also checked the CSS file linked to the page, and found that the table still had 7 columns, so I changed that to be 16 for each of the hours in the day.

Finally, the calendar displayed properly:

I then added validation that the holiday length was less than 90 days:

```
200 if (endDate - startDate).days > 90: #If the holiday is longer than 90 days
201     flash('Holidays must be less than 90 days')
202     return render_template('new_holiday.html', destination=destination) #Sends user back to the form
203
```

TypeError

TypeError: unsupported operand type(s) for -: 'str' and 'str'

Traceback (most recent call last)

This error then appeared indicating that my data types were incorrect, so I fixed this by using `.strptime()` of the `datetime` library:

```
200 startDate = datetime.strptime(startDate, '%Y-%m-%d') #Takes start date and converts it to a datetime object
201 endDate = datetime.strptime(endDate, '%Y-%m-%d') #"" with end date
202 if (endDate - startDate).days > 90: #If the holiday is longer than 90 days
203     flash('Holidays must be less than 90 days')
204     return render_template('new_holiday.html', destination=destination) #Sends user back to the form
```


Changing this gave me the right error message when I tried to enter a holiday that was over 90 days:

[Back to Dashboard](#)

Destination: United States

Holidays must be less than 90 days

Holiday Title:

Start Date:
 

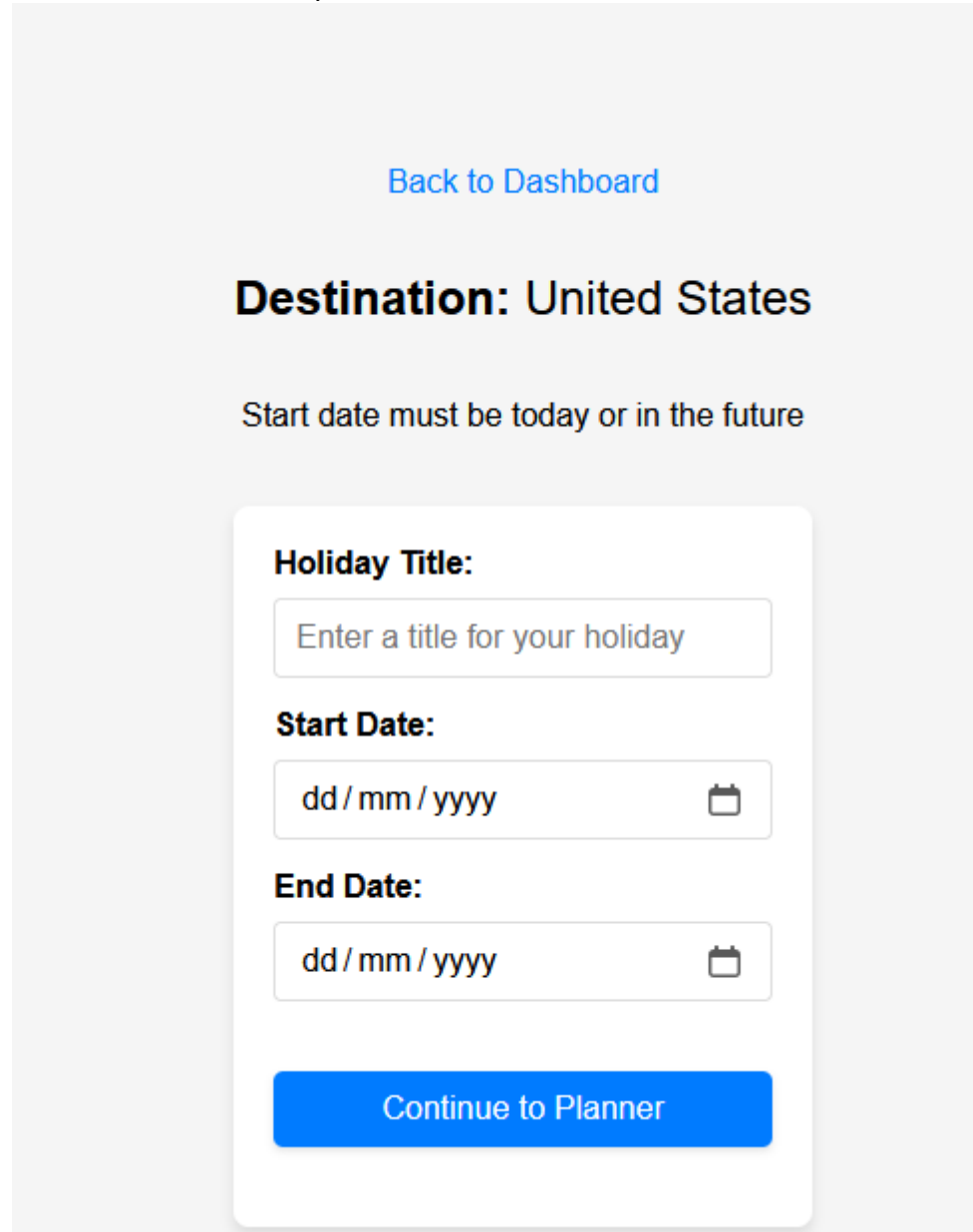
End Date:
 

[Continue to Planner](#)

I also wanted to validate that the holiday was actually not happening in the past because the current code doesn't prevent that.

```
205  if tempStartDate < datetime.today():  
206      flash('Start date must be today or in the future')  
207      return render_template('new_holiday.html', destination=destination)  
208
```

This worked successfully:



The screenshot shows a web interface with a light gray background. At the top, there is a blue link that says "Back to Dashboard". Below this, the text "Destination: United States" is displayed in a large, bold, black font. Underneath, a message in a smaller black font states "Start date must be today or in the future". The main part of the form is a white card with rounded corners and a subtle shadow. Inside the card, there are three sections: "Holiday Title:" followed by a text input field with the placeholder "Enter a title for your holiday"; "Start Date:" followed by a date input field showing "dd/mm/yyyy" and a calendar icon; and "End Date:" followed by another date input field showing "dd/mm/yyyy" and a calendar icon. At the bottom of the card is a prominent blue button with the text "Continue to Planner" in white.

To complete the rest of the success criteria, I began to work on the drag and drop functionality of the calendar. This first saw me build the frontend that would enable me to drag blocks around, meaning calendar.html had to be edited to add a container for the activities to be a part of, where activities was passed through from the Python:

```

26 <div class="activity-palette">
27     {% for activity in activity_names %}
28     <div class="activity-block" draggable="true" data-activity="{{ activity }}">{{ activity }}</div>
29     {% endfor %}
30 </div>
31

```

At this point, I had also made new_activity.html, which would be the page that is opened when the user presses on “Add new activity” from the calendar page. I added in holidayID as a hidden input in the form as I knew that would be required for the Python to be able to insert the new activity into the database, as without the holidayID, the database doesn’t know what holiday the activity is for.

```

1 <html>
2 <head>
3     <title>Add Activity</title>
4     <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
5 </head>
6 <body>
7     <header>
8         <div class="planner-header">
9             <h1>TripMaestro</h1>
10        </div>
11        <div class="header-right">
12            <a href="/logout" class="btn-signout">Sign Out</a>
13        </div>
14    </header>
15
16    <a href="/calendar?holidayID={{ holidayID }}" class="back-link">Go back to holiday list</a>
17
18    <form action="/new_activity" method="POST">
19        <input type="hidden" name="holidayID" value="{{ holidayID }}">
20
21        <!-- Fields to create a new activity -->
22        <div class="form-group">
23            <label for="activity-name">Activity Name:</label>
24            <input type="text" id="activity-name" name="activity_name" placeholder="Enter activity name" required>
25        </div>
26
27        <div class="form-group inline">
28            <label for="activity-date">Date:</label>
29            <input type="date" id="activity-date" name="activity_date" required>
30        </div>
31
32        <div class="form-group">
33            <label for="activity-location">Location (optional):</label>
34            <input type="text" id="activity-location" name="activity_location" placeholder="Enter location">
35        </div>

```

```

27 <div class="form-group inline">
28     <label for="activity-date">Date:</label>
29     <input type="date" id="activity-date" name="activity_date" required>
30 </div>
31
32 <div class="form-group">
33     <label for="activity-location">Location (optional):</label>
34     <input type="text" id="activity-location" name="activity_location" placeholder="Enter location">
35 </div>
36
37 <div class="form-group">
38     <label for="booking-link">Link to Booking (optional):</label>
39     <input type="url" id="booking-link" name="booking_link" placeholder="Enter a link to booking, include https://www..">
40 </div>
41
42 <div class="form-group">
43     <label for="additional-notes">Notes:</label>
44     <textarea id="additional-notes" name="additional_notes" placeholder="Add any additional notes here"></textarea>
45 </div>
46
47 <button type="submit">Save</button>
48 </form>
49 </body>
50 </html>
51

```

For the new_activity route in app.py, the following was made:

```

253 @app.route('/new_activity', methods=['GET', 'POST'])
254 def new_activity(): #Defines the new_activity function
255     if 'user_id' not in session: #Checks if the user is logged in
256         return redirect(url_for('login'))
257     if request.method == 'POST':
258         user_id = session['user_id']
259         holidayID = request.form.get('holidayID') #Gets the holiday ID from the hidden input field
260         if not holidayID:
261             return redirect(url_for('dashboard')) #If no holiday ID is found, user is redirected to the dashboard
262         activity_name = request.form.get('activity_name')
263         activity_date = request.form.get('activity_date')
264         activity_location = request.form.get('activity_location')
265         booking_link = request.form.get('booking_link')
266         additional_notes = request.form.get('additional_notes') #Gets all data to be stored in database
267
268         conn = sqlite3.connect('users.db')
269         cursor = conn.cursor()
270
271         cursor.execute("""
272             INSERT INTO activities (holidayID, userID, startTime, location, bookingLink, additionalNotes, activityName)
273             VALUES (?, ?, ?, ?, ?, ?, ?)
274             """, (holidayID, user_id, activity_date, activity_location, booking_link, additional_notes, activity_name))
275         #Inserts data into database
276         conn.commit()
277         conn.close()
278         return redirect(url_for('calendar', holidayID=holidayID))
279     holidayID = request.args.get('holidayID') #If form isn't filled in, holiday ID is taken from URL
280     if holidayID:
281         return render_template('new_activity.html', holidayID=holidayID)
282     return redirect(url_for('dashboard'))

```

This completed the 4th success criteria where the user is able to add the details of their activities and for that to be sent to and visible on the calendar.

I only currently expected the code to add a record to the database of the user's activity, with no actual response from the calendar, and that is what happened: (the third row was the test data)

	activityID	holidayID	userID	activityName	startTime	bookingLink	location	ditionalNo
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	NULL	3	NULL	NULL	NULL	NULL	NULL
2	2	NULL	3	NULL	NULL	NULL	NULL	NULL
3	3	12	3	aa	2025-0...			a

To make the calendar able to use this data, it would need to execute more SQL in order to get the relevant activities for the current holiday in order to give them to the template as shown on page 95 to be displayed. This is shown below:

```

245 conn = sqlite3.connect('users.db')
246 cursor = conn.cursor()
247 cursor.execute("SELECT activityName FROM activities WHERE holidayID = ?", (holidayID,))
248 activities = cursor.fetchall() # Returns a tuple of all activities for the holiday
249 conn.close()
250
251 activity_names = []
252 for activity in activities:
253     activity_names.append(activity[0]) #Converts tuple into a list of activity names
254
255 #Passes the full list of dates to the template
256 return render_template(
257     'calendar.html',
258     holidayID=holidayID,
259     all_dates=all_dates, #Sends the list of all dates
260     activity_names = activity_names #Sends the list of activity names
261 )

```

To allow the activities to have drag-and-drop functionality, each activity block needs to be draggable and droppable, and each cell in the calendar needs to be able to contain an activity. This is achieved with the following JavaScript:

```

51 <script>
52     //Enable drag-and-drop for activity blocks
53     document.querySelectorAll('.activity-block').forEach(block => { //Select all activity blocks
54         block.addEventListener('dragstart', event => { //Add dragstart event listener
55             event.dataTransfer.setData('text/plain', event.target.dataset.activity); //Set data to activity name
56         });
57     });
58
59     document.querySelectorAll('.calendar-cell').forEach(cell => { //Select all calendar cells
60         cell.addEventListener('dragover', event => { //Add dragover event listener
61             event.preventDefault(); //Prevents text from being draggable
62         });
63
64         cell.addEventListener('drop', event => {
65             const activityName = event.dataTransfer.getData('text/plain'); //Get activity name
66             cell.textContent = activityName; // Add activity to cell
67         });
68     });
69 </script>
70 </body>
71 </html>
72

```

When loading up the calendar to test this, the newly created activities would, when dragged onto a box, just drop its text in there.

This could cause a problem for larger strings, so a cap of 16 characters is to be placed on the title of each activity. The idea is that in the future, the user can just click on any of the activities and information from the database will appear about the activity so they can find out about whatever else they may need.

```

274     if len(activity_name) > 16:
275         flash('Activity name must be 16 characters or less')
276         return redirect(url_for('new_activity', holidayID=holidayID))

```

I now want to create a way for the user to delete activities off of the calendar, which requires more JavaScript along with adding the button to remove the activity in the HTML:

```

71 document.getElementById('delete-activity-btn').addEventListener('click', () => { //Add click event listener to delete activity button
72     const calendarCells = document.querySelectorAll('.calendar-cell'); //Select all calendar cells
73     alert('Click on a calendar cell to delete the activity.');//Alert user to click on a cell to delete activity
74
75     calendarCells.forEach(cell => { //Add click event listener to each cell
76         cell.addEventListener('click', function deleteActivity() { //Add click event listener
77             if (cell.textContent.trim() !== '') { //Check if cell has activity
78                 cell.textContent = ''; // Clear the activity from the cell
79                 alert('Activity deleted!');//Alert user that activity has been deleted
80             } else {
81                 alert('No activity to delete in this cell.');//Alert user that there is no activity to delete
82             }
83             // Remove the click event listener to prevent multiple clicks from stacking
84             cell.removeEventListener('click', deleteActivity); //Remove click event listener
85         });
86     });
87 });
88
89 </script>
90
91
92 <div class="activity-buttons">
93     <a href="{{ url_for('new_activity', holidayID=holidayID) }}" class="btn-add-activity">Add New Activity</a>
94     <button id="delete-activity-btn" class="btn-delete-activity">Delete Activity</button>
95 </div>

```

Line 24 is only new addition in HTML

When testing this, this worked, clearing out the calendar cell's contents with one click as expected. However, there is no way to get back to the mouse click being non-deleting after it is toggled on. This does concur with how the JavaScript never had a different script run based on whether the button was pressed for the 1st or 2nd time.

The screenshot shows a web application interface. At the top right, there are two buttons: "Add New Activity" (green) and "Delete Activity" (red). Below these buttons is a horizontal bar with four colored boxes: "eating" (yellow), "sleep" (orange), "efvb" (light blue), and a long yellow box containing a series of 'a' characters. Below this bar is a calendar grid. The calendar has columns for hours from 07:00 to 16:00 and rows for dates from Saturday, 2025-02-08 to Wednesday, 2025-02-12. The cell for Saturday, 2025-02-08 at 08:00 contains the text "sleep", and the cell at 09:00 contains the text "eating".

Changing the JavaScript to toggle between "Delete Mode" or not resulted in the following code:

```

71     let deleteMode = false; // Tracks whether delete mode is active
72
73     function enableDragAndDrop() {
74         document.querySelectorAll('.calendar-cell').forEach(cell => {
75             cell.addEventListener('dragover', event => {
76                 event.preventDefault(); // Allow drop
77             });
78
79             cell.addEventListener('drop', event => {
80                 const activityName = event.dataTransfer.getData('text/plain'); // Get activity name
81                 if (!deleteMode) {
82                     cell.textContent = activityName; // Set cell text to activity name
83                 }
84             });
85         });
86     }
87
88     document.getElementById('delete-activity-btn').addEventListener('click', () => {
89         deleteMode = !deleteMode; // Toggle delete mode
90         const calendarCells = document.querySelectorAll('.calendar-cell');
91
92         if (deleteMode) {
93             alert('Delete mode activated. Click on a calendar cell to remove an activity.');
```

document.getElementById('delete-activity-btn').textContent = 'Exit Delete Mode';

```

96         calendarCells.forEach(cell => {
97             cell.addEventListener('click', function deleteActivity() {
98                 if (deleteMode) {
99                     if (cell.textContent.trim() !== '') {
100                         cell.textContent = ''; // Clear the activity from the cell
101                         alert('Activity deleted!');
102                     } else {
103                         alert('No activity to delete in this cell.');
```

```
106         });
107     });
108 } else {
109     alert('Delete mode deactivated.');
```

```
110     document.getElementById('delete-activity-btn').textContent = 'Delete Activity';
111
112     // Re-enable drag-and-drop functionality
113     enableDragAndDrop();
114 }
115 });
116
117 // Initial drag-and-drop setup
118 enableDragAndDrop();
```

This fully enabled functionality of deleting and adding items as shown in the tests:

[illegible]

Further testing generally around the website saw the database lock frequently:

OperationalError

```
sqlite3.OperationalError: database is locked
```

Traceback (most recent call last)

File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536,

Normally the problem here is that a connection isn't closed with the database, and I notice I haven't guaranteed that these are closed as I haven't used try and except for those. Below shows the corrections I made to properly close the connections:

```
conn = sqlite3.connect('users.db')
cursor = conn.cursor()

try:
    cursor.execute("""
        INSERT INTO activities (holidayID, userID, startTime, location, bookingLink, additionalNotes, activityName)
        VALUES (?, ?, ?, ?, ?, ?, ?)
        """, (holidayID, user_id, activity_date, activity_location, booking_link, additional_notes, activity_name))
    #Inserts data into database
    conn.commit()
finally:
    conn.close()
```

```

try:
    userCursor = userConnection.cursor()
    userCursor.execute("SELECT explorativeRating, isolatedRating, uniqueCuisineRating FROM users WHERE id = ?",
                        (session['user_id'],)) #Gets user preferences from database
    user_prefs = userCursor.fetchone() #Takes the user's record from the database
finally:
    userConnection.close() #Ends connection between program and database to avoid locking

74
75     destConnection = sqlite3.connect('users.db')
76     try:
77         destCursor = destConnection.cursor()
78         destCursor.execute("SELECT * FROM countries")
79         countries = destCursor.fetchall() #Makes a complete array of every country in the database
80     finally:
81         destConnection.close()
try:
    holidayCursor = holidayConnection.cursor()
    holidayCursor.execute("INSERT INTO holidays (countryName, startDate, endDate, userID, holidayTitle ) VALUES (?, ?, ?, ?, ?)",
                        (destination, startDate, endDate, userID, holiday_title)) #Inserts data into database
    holidayID = holidayCursor.lastrowid #Gets the primary key of the user's holiday
    holidayConnection.commit()
finally:
    holidayConnection.close()

try:
    dateCursor = dateConnection.cursor()
    dateCursor.execute("SELECT startDate, endDate FROM holidays WHERE holidayID = ?", (holidayID,))
    dates = dateCursor.fetchone()
finally:
    dateConnection.close()

try:
    cursor = conn.cursor()
    cursor.execute("SELECT activityName FROM activities WHERE holidayID = ?", (holidayID,))
    activities = cursor.fetchall() # Returns a tuple of all activities for the holiday
finally:
    conn.close()
activity_names = []

```

```

try:
    cursor.execute("""
    INSERT INTO activities (holidayID, userID, startTime, location, bookingLink, additionalNotes, activityName)
    VALUES (?, ?, ?, ?, ?, ?, ?)
    """, (holidayID, user_id, activity_date, activity_location, booking_link, additional_notes, activity_name))
    #Inserts data into database
    conn.commit()
finally:
    conn.close()
return redirect(url_for('index', holidayID=holidayID))

```

This solved the issue of the database locking as no part of the code had a connection with the database for longer than it needed.

Next, I wanted to make it so that when the user presses on an activity's title on the calendar, it reveals all of the data about the activity at the top of the page. To implement this, firstly app.py needs to be updated to add SQL commands, along with giving the details of the activities to calendar.html:

```

267 try:
268     cursor.execute("""
269         SELECT activityName, startTime, location, bookingLink, additionalNotes
270         FROM activities
271         WHERE holidayID = ?
272         """, (holidayID,))
273     activities = cursor.fetchall() # List of tuples
274 finally:
275     conn.close()
276
277 #Passes the full list of dates to the template
278 return render_template(
279     'calendar.html',
280     holidayID=holidayID,
281     all_dates=all_dates, #Sends the list of all dates
282     activity_names = activity_names, #Sends the list of activity names
283     activity_details = activities #Sends the list of activities
284 )

```

Below shows the additions I made to calendar.html, where Jinja2 is used to take the relevant details for the activity to be displayed from the Python file and display them to the user.

```

22 <div class="activity-details" id="activity-details">
23     {% if selected_activity %}
24     <div class="details-card">
25         <h3>Activity Details</h3>
26         <p><strong>Name:</strong> {{ selected_activity.activityName }}</p>
27         <p><strong>Time:</strong> {{ selected_activity.startTime }}</p>
28         <p><strong>Location:</strong> {{ selected_activity.location or 'No location specified' }}</p>
29         {% if selected_activity.bookingLink %}
30         <p><strong>Booking:</strong> <a href="{{ selected_activity.bookingLink }}" target="_blank">View Booking</a></p>
31         {% endif %}
32         {% if selected_activity.additionalNotes %}
33         <p><strong>Notes:</strong> {{ selected_activity.additionalNotes }}</p>
34         {% endif %}
35     </div>
36     {% endif %}
37 </div>

```

Below shows the JavaScript added to the query for selecting on the calendar cell. On the click event, a form is made with the activity name and the holidayID, which retrieves the rest of the data. The completed form is then submitted for the Python to then use and pass onto the webpage.

```

87 cell.addEventListener('click', function() { //Add click event listener
88     if (this.textContent.trim() !== '') { //Check if cell has activity
89         //Get the activity name and create a form
90         const activityName = this.textContent.trim(); //Get activity name
91         const form = document.createElement('form'); //Create form element
92         form.method = 'GET'; //Set form method to GET
93         form.action = `/calendar?holidayID=${holidayID}&activity=${encodeURIComponent(activityName)}`; //Redirect to new activity page
94         document.body.appendChild(form); //Append form to body
95         form.submit(); //Submit form
96     }
97 })

```

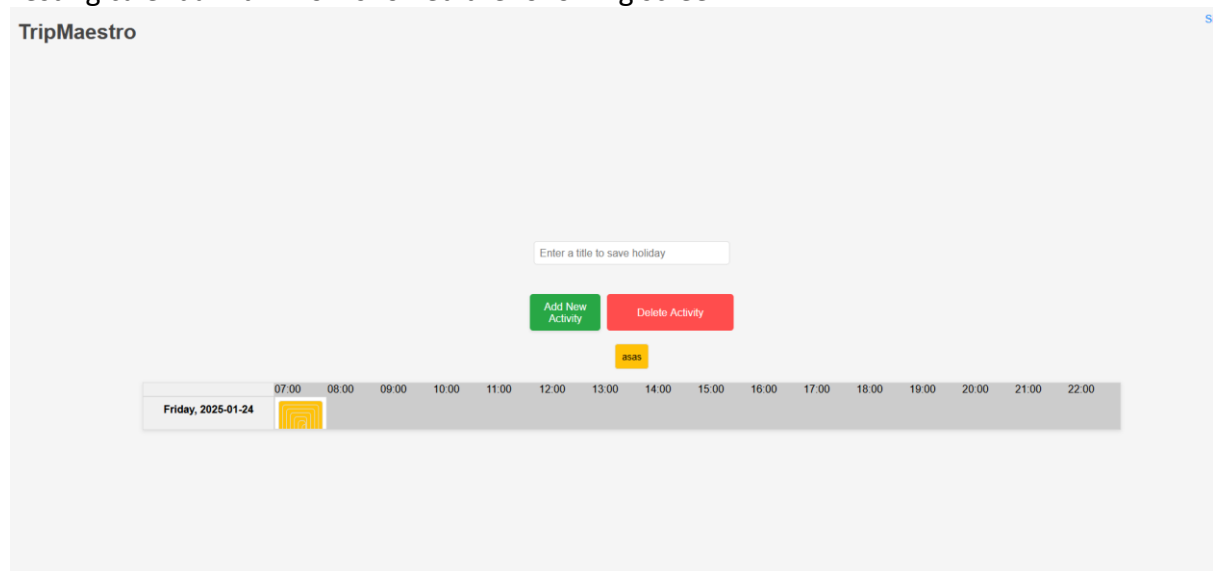
Changes to app.py:

Adding line 233 to get the activity name from JavaScript, and also adding to `calendar()` to use `selected_activity_name` to find the rest of the data about that activity from the database.

```
232 holidayID = request.args.get('holidayID', -1) # Get the holiday ID from query parameters
233 selected_activity_name = request.args.get('selected_activity') #Get the selected activity name from JS form
234 if 'user_id' not in session:
```

```
256 # Fetch activities and selected activity details if any
257 conn = sqlite3.connect('users.db')
258 selected_activity = None
259 try:
260     cursor = conn.cursor()
261     cursor.execute("SELECT activityName FROM activities WHERE holidayID = ?", (holidayID,))
262     activities = cursor.fetchall()
263
264     if selected_activity_name:
265         cursor.execute("""
266             SELECT activityName, startTime, location, bookingLink, additionalNotes
267             FROM activities
268             WHERE holidayID = ? AND activityName = ?
269             """, (holidayID, selected_activity_name))
270         activity_data = cursor.fetchone()
271         if activity_data:
272             selected_activity = {
273                 'activityName': activity_data[0],
274                 'startTime': activity_data[1],
275                 'location': activity_data[2],
276                 'bookingLink': activity_data[3],
277                 'additionalNotes': activity_data[4]
278             }
279     finally:
280         conn.close()
```

Testing `calendar.html` now showed the following screen:



The columns seemed to display properly, however the rows seemed to be overlapping or something. When investigating my HTML, I identified that each row needs its own div container around line 56.

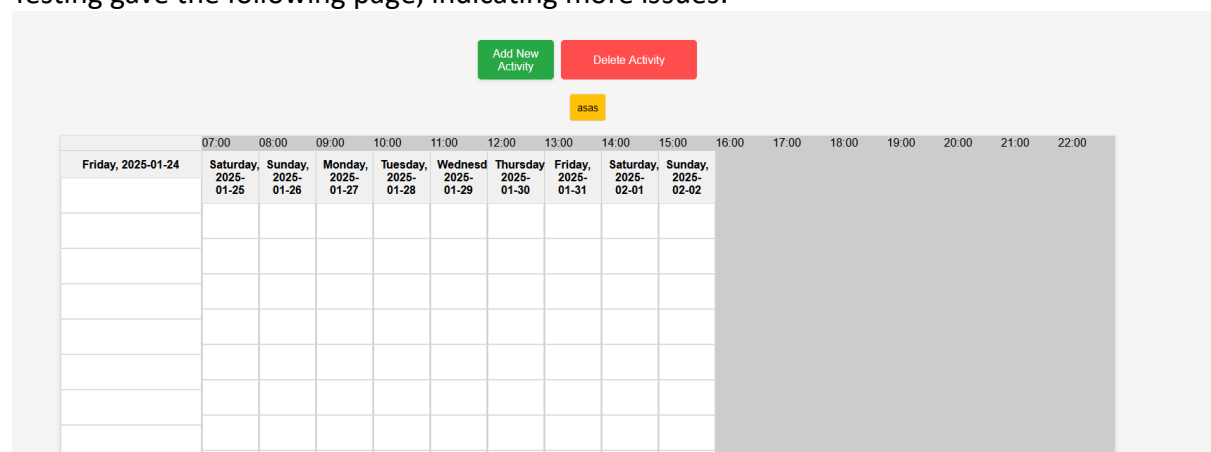
```

49
50 <div class="calendar-container">
51   <div class="day-row-header"></div>
52   {% for hour in range(7, 23) %}
53   |   <div class="time-slot-header">{{ "%02d:00" | format(hour) }}</div>
54   {% endfor %}
55
56   {% for date in all_dates %}
57   |   <div class="day-row-header">{{ date }}</div>
58   |   {% for hour in range(7, 23) %}
59   |   |   <div class="calendar-cell" data-date="{{ date }}" data-hour="{{ ho
60   |   |   {% endfor %}
61   |   {% endfor %}
62 </div>

```

To fix this, I added `<div class="day-row">` on line 59 so that for each day, there would be a new row.

Testing gave the following page, indicating more issues:



The screenshot shows a web application interface. At the top, there are two buttons: "Add New Activity" (green) and "Delete Activity" (red). Below these buttons is a yellow button labeled "asas". The main part of the interface is a calendar grid. The grid has columns for days of the week and dates, and rows for time slots. The first row of the grid shows dates from Friday, 2025-01-24 to Sunday, 2025-02-02. The time slots are labeled from 07:00 to 22:00. The grid is mostly empty, with some cells containing dates and times.

The issue wasn't obvious here so I decided to rollback to before I had edited calendar.html.

To work towards activity details appearing, I first edited the calendar container in the HTML to add metadata to each cell about its time as well as activityID. I also created a new container called selected-activity that holds the data of the activity being selected to display to the user. The ID labelled there allows it to be edited by the JavaScript directly, which will have access to the details through the metadata stored in the calendar cell.

```

33     <div class="selected-activity">
34         <h2>Currently Selected Activity</h2>
35         <p><strong>Name:</strong> <span id="activity-name">None</span></p>
36         <p><strong>Time:</strong> <span id="activity-time">None</span></p>
37         <p><strong>Location:</strong> <span id="activity-location">None</span></p>
38         <p><strong>Notes:</strong> <span id="activity-notes">None</span></p>
39     </div>
40
41     <div class="calendar-container">
42         <div class="day-row-header">
43             {% for hour in range(7, 23) %}
44                 <div class="time-slot-header">{{ "%02d:00" | format(hour) }}</div>
45             {% endfor %}
46         </div>
47
48         {% for date in all_dates %}
49             <div class="day-row-header">{{ date }}</div>
50             {% for hour in range(7, 23) %}
51                 <div
52                     class="calendar-cell"
53                     data-date="{{ date }}"
54                     data-hour="{{ hour }}"
55                     data-activity-id=""
56                     testText
57                 </div>
58             {% endfor %}
59         {% endfor %}
60     </div>

```

```

69 document.querySelectorAll('.activity-block').forEach(activity => {
70     activity.addEventListener('dragstart', event => {
71         event.dataTransfer.setData('activity-id', activity.getAttribute('data-id'));
72         event.dataTransfer.setData('activity-name', activity.getAttribute('data-name'));
73         event.dataTransfer.setData('activity-location', activity.getAttribute('data-location'));
74         event.dataTransfer.setData('activity-notes', activity.getAttribute('data-notes'));
75     });
76 });

```

I then edited the JavaScript that handles the beginning of the activity being dragged. The `event.dataTransfer.setData()` function makes it so that the 4 attributes I have obtained from the HTML document are stored under the key (activity-id, activity-name etc). This is so that they can be placed back into the calendar when the block is dropped.

```

83     cell.addEventListener('drop', event => {
84         event.preventDefault();
85
86         const activityId = event.dataTransfer.getData('activity-id');
87         const activityName = event.dataTransfer.getData('activity-name');
88         const activityLocation = event.dataTransfer.getData('activity-location');
89         const activityNotes = event.dataTransfer.getData('activity-notes');
90
91         cell.textContent = activityName;
92
93         cell.setAttribute('data-activity-id', activityId);
94         cell.setAttribute('data-name', activityName);
95         cell.setAttribute('data-location', activityLocation);
96         cell.setAttribute('data-notes', activityNotes);
97     });

```

I also added this JavaScript for when the activity block gets placed down in the calendar. This creates constants that are defined by the data that was transferred in the previous function. It then displays the activity name in the calendar for the user to see, but also holds attributes for the details of that activity. From there they will be displayed to the user should the user click on them.

```

98     cell.addEventListener('click', () => {
99         document.getElementById('activity-name').textContent = cell.getAttribute('data-name') || 'None';
100         document.getElementById('activity-time').textContent = `${cell.getAttribute('data-hour')}:00`; //makes 7 --> 7:00
101         document.getElementById('activity-location').textContent = cell.getAttribute('data-location') || 'None';
102         document.getElementById('activity-notes').textContent = cell.getAttribute('data-notes') || 'None';
103     });
104
105

```

I then made the code for what occurs when the cell is clicked, which writes to the HTML document to show the user the details about their activity.

Opening the webpage showed the following:

Add New Activity
Delete Activity

asas

Currently Selected Activity

Name: None
 Time: None
 Location: None
 Notes: None

07:00	Friday, 2025-01-24	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex
08:00		t	t	t	t	t	t	t	t	t	t	t	t	t	t	t
09:00																
10:00																
11:00																
12:00																
13:00																
14:00																
15:00																
16:00																
17:00																
18:00																
19:00																
20:00																
21:00																
22:00																
testText	Saturday, 2025-01-25	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex
		t	t	t	t	t	t	t	t	t	t	t	t	t	t	t
testText	Sunday, 2025-01-26	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex
		t	t	t	t	t	t	t	t	t	t	t	t	t	t	t

Key results that I identified from this normal test was that the testText was displaying in every cell in the calendar, meaning the attributes for each entry were also being stored. Dragging an activity and selecting on it showed the following values, meaning the attributes exist but are not being written to correctly.

Currently Selected Activity

Name: null

Time: 15:00

Location: null

Notes: null

Also, it seemed that there are two columns on the far left for the times and the dates, whereas one should be a column and the other should be a top row. Moreover, the currently selected activity is in a non-ideal location, and would be better on the far top left of the screenshot, rather than between the add activity button and the actual calendar.

The first issue I chose to work on was the double columns, because I figured that it would simply be a mislabelling of the top row as a column.

```
41 <div class="calendar-container">
42   <div class="day-row-header"> </div>
43   {% for hour in range(7, 23) %}
44     <div class="time-slot-header">{{ "%02d:00" | format(hour) }}</div>
45   {% endfor %}
46 <!--</div-->
```

Moving the ending div tag of the day-row-header class from the green to line 42 aimed to hold the structure of the columns, compared to before where every instance of the row became part of the structure. This fixed that problem with the results shown below:

TripMaestro

Enter a title to save holiday

Add New Activity

Delete Activity

asas

Eating

Currently Selected Activity

Name: None

Time: None

Location: None

Notes: None

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00	15:00	16:00	17:00	18:00	19:00	20:00	21:00	22:00
Friday, 2025-01-24	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t
Saturday, 2025-01-25	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t
Sunday, 2025-01-26	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t
Monday, 2025-01-27	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t
Tuesday, 2025-01-28	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t	testTex t

I then tried to move the “Currently Selected Activity” to the side, in the same vertical space as the Add and Delete Activity buttons. This first required all of it to be in the same div container:

```

22 <div class="vertical-strip">
23 <div class="activity-buttons">
24 <a href="{{ url_for('new_activity', holidayID=holidayID) }}" class="btn-add-activity">Add New Activity</a>
25 <button id="delete-activity-btn" class="btn-delete-activity">Delete Activity</button>
26 </div>
27
28 <div class="selected-activity">
29 <h2>Currently Selected Activity</h2>
30 <p><strong>Name:</strong> <span id="activity-name">None</span></p>
31 <p><strong>Time:</strong> <span id="activity-time">None</span></p>
32 <p><strong>Location:</strong> <span id="activity-location">None</span></p>
33 <p><strong>Notes:</strong> <span id="activity-notes">None</span></p>
34 </div>
35 </div>
36

```

When testing the web page now, the following result appeared, indicating a successful test:

TripMaestro

Enter a title to save holiday

Add New Activity
Delete Activity

Currently Selected Activity
Name: None
Time: None
Location: None
Notes: None

asas Eating

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00	15:00	16:00	17:00	18:00	19:00	20:00
Friday, 2025-01-24	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex
Saturday, 2025-01-25	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex
Sunday, 2025-01-26	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex
Monday, 2025-01-27	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex	testTex

Now to try and fix the last issue of attributes not being stored properly, I tried to trace back from where the attribute was defined, and realised that the document never received the data about the activities from app.py due to it not being returned from the calendar() function:

```

268 return render_template(
269     'calendar.html',
270     holidayID=holidayID,
271     all_dates=all_dates, #Sends the list of all dates
272     activity_names = activity_names #Sends the list of activity names
273 )
274

```

I updated it to the following:

```
267     conn = sqlite3.connect('users.db')
268     try:
269         cursor = conn.cursor()
270         cursor.execute("SELECT activityName, startTime, location, additionalNotes, activity
271         activities = cursor.fetchall()
272     finally:
273         conn.close()
274
275     activities_list = []
276     for activity in activities:
277         activities_list.append({
278             'id': activity[0], # rowid from the database
279             'name': activity[1],
280             'time': activity[2],
281             'location': activity[3],
282             'notes': activity[4]
283         })
284
285     #Passes the full list of dates to the template
286     ✨ return render_template(
287         'calendar.html',
288         holidayID=holidayID,
289         all_dates=all_dates, #Sends the list of all dates
290         activity_names = activity_names #Sends the list of activity names
291         activites = activities_list #Sends the list of activities' details
292     )
293
```

I experienced a syntax error here, and realised I missed a comma at the end of line 290.

When testing this, I realised the draggable activities disappeared.

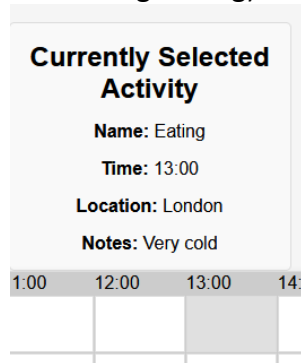
After reviewing the code, I suspected that it was because 'activities' was being passed into the HTML document which was trying to use "activities_list". To correct this, I renamed activities in the Python file to "activities_list".

Now testing gave me a successful result:

The screenshot shows a web application interface. At the top, there is a text input field with the placeholder text "Enter a title to save holiday". Below this, there are two buttons: a green button labeled "Add New Activity" and a red button labeled "Delete Activity". To the right of these buttons is a section titled "Currently Selected Activity". This section displays the following information: "Name: Eating", "Time: 13:00", "Location: London", and "Notes: Very cold".

At this point I was happy to remove the placeholder text in each box, in order to test the entire calendar's functionality.

It turned out that the activity name now wasn't displaying on the calendar, although the 'Currently Selected Activity' still worked as expected. (Grey box represents where my cursor was during testing)



After tracing the JavaScript, I realised I had created two different ways that the webpage implemented drag and drop, with the 'drop' event containing the attributes and data that it should have, but also `enableDragAndDrop()` which didn't contain the necessary data. As this was run as soon as the webpage was loaded, this meant that the calendar didn't receive the data. However, as the 'drop' event had the drag and drop functionality separately written with the correct data, the activity details were accurate and still able to work at the same time.

To correct this, I removed the `enableDragAndDrop()` procedure, removing the call for it at the start of the webpage loading. The other place in which it was called was when the "Delete Activity" button was clicked, so to tackle that, I rewrote that procedure so that it checked a flag "deleteMode" on whether to clear the attributes of a calendar cell or not. The corrected code is shown below:

```

114 let deleteMode = false; //Flag toggles whether it click should delete cells
115
116 document.getElementById('delete-activity-btn').addEventListener('click', () => {
117     deleteMode = !deleteMode; //Toggles delete mode
118     const calendarCells = document.querySelectorAll('.calendar-cell');
119
120     if (deleteMode) {
121         document.getElementById('delete-activity-btn').textContent = 'Exit Delete Mode';
122
123         calendarCells.forEach(cell => {
124             cell.addEventListener('click', function deleteActivity() {
125                 if (deleteMode) {
126                     if (cell.textContent.trim() !== '') {
127                         cell.textContent = ''; //Clears the contents of the cell
128                         cell.removeAttribute('data-activity-id');
129                         cell.removeAttribute('data-name');
130                         cell.removeAttribute('data-location');
131                         cell.removeAttribute('data-notes');
132                     }
133                 }
134             });
135         });
136     } else {
137         document.getElementById('delete-activity-btn').textContent = 'Delete Activity';
138     }
139 });

```

Testing this involved dragging the created activities onto the web page, and now displayed as it should:

The screenshot displays a web application interface for managing activities. At the top, there are two buttons: 'Add New Activity' (green) and 'Delete Activity' (red). To the right, a 'Currently Selected Activity' panel shows details for an activity named 'Eating' at 14:00 in London, with the note 'Very cold'. Below this, a calendar grid shows dates from Friday, 2025-01-24 to Tuesday, 2025-01-28. The grid has columns for each hour from 07:00 to 18:00. An activity 'Eating' is scheduled for Saturday, 2025-01-25, at 13:00. Above the calendar, there are several yellow tags: 'asas', 'Eating', 'time', 'ffd', 'zdasdasd', and 'aa'.

This completed the 5th success criteria where the user is able to write notes for a calendar entry and it can be displayed on the calendar page. It can be opened at any time by clicking on the calendar entry containing the required notes.

The final, 6th success criteria, involves the activities in the calendar being stored to be accessed at a later date. This would involve each activity being linked to a holiday, which is itself linked to a user. This would allow it so that when a calendar is opened, it is able to reconstruct the calendar based on the times stored of each activity.

To begin working on this, I made the save button work by taking the titles and timings of the activities, and giving them to the Python to add the relevant info to the database. This first required modifications to calendar.html, so that the 'Save Calendar' button now submitted a form when pressed with hidden data with the activity details.

This is achieved with the following HTML, and I also took in calendar-data instead of just the start time of the activity, as it will help future iterations of the program where it already covers for future data that the Python program may want rather than just limiting it to the starting time of the activity (which would suffice for this iteration).

```

70 <form id="save-form" action="/save_calendar" method="POST">
71   <input type="hidden" id="calendar-data" name="calendar-data" value="">
72   <input type="hidden" name="holiday-id" value="{{ holidayID }}">
73   <input type="hidden" id="holiday-title" name="holiday-title" value="">
74   <button type="submit" class="save-button">Save</button>
75 </form>
76

```

I then made the JavaScript to handle the submission of the form, which goes through every cell in the calendar, extracting the date, the hour, and the activity ID that will be used to find the relevant record in the database. All of this data is then passed back into the HTML as a JSON string, which is then passed into Python through being submitted in the form.

```

78     document.getElementById('save-form').addEventListener('submit' (e) => {
79         const activities = [];
80         document.querySelectorAll('.calendar-cell').forEach(cell => {
81             if (cell.getAttribute('data-activity-id')) {
82                 const date = cell.getAttribute('data-date')
83                 activities.push({
84                     activityId: cell.getAttribute('data-activity-id'),
85                     startTime: `${date} ${cell.getAttribute('data-hour')}:00`
86                 });
87             }
88         });

```

The Python for this takes in the data from the HTML and updates the database appropriately:

```

330 @app.route('/save_calendar', methods=['POST'])
331 def save_calendar():
332     if 'user_id' not in session:
333         return redirect(url_for('login'))
334
335     holiday_id = request.form.get('holiday-id')
336     holiday_title_list = request.form.getlist('holiday-title')
337     if holiday_title_list: #Makes a list of holiday titles to add to the database
338         holiday_title = holiday_title_list[0]
339     else:
340         holiday_title = ''
341     calendar_data = json.loads(request.form.get('calendar-data')) #Gets calendar data from the form
342
343     conn = sqlite3.connect('users.db')
344     try:
345         cursor = conn.cursor()
346         if holiday_title: #Sets holiday title to new title
347             cursor.execute("UPDATE holidays SET holidayTitle = ? WHERE holidayID = ?",
348                             (holiday_title, holiday_id))
349
350         for activity in calendar_data: #Goes through each activity to add the start time to the database
351             cursor.execute("UPDATE activities SET startTime = ? WHERE activityID = ?",
352                             (activity['startTime'], activity['activityId']))
353
354         conn.commit()
355     finally:
356         conn.close()
357
358     return redirect(url_for('calendar', holidayID=holiday_id)) #Opens calendar page back up

```

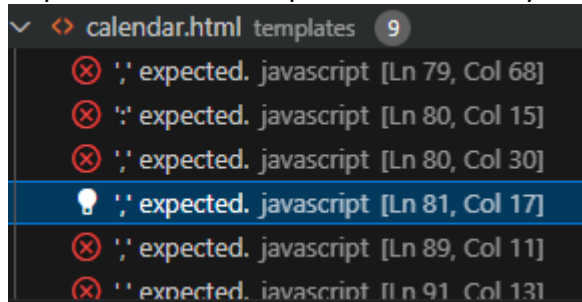
Testing the “Save Calendar” button resulting in the following error:

JSONDecodeError

`json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)`

Traceback (most recent call last)

A quick review of the problems tab in my IDE showed that a syntax error was likely:



This led me to realise that on line 78 of the JavaScript, I also need to pass in the event to be handled:

```
document.getElementById('save-form').addEventListener('submit', function(event) {  
    const activities = [];
```

I decided to test with a simple activity in a calendar, and pressing save loaded back onto the same webpage.

notes. note						
	07:00	08:00	09:00	10:00	11:00	12:00
Friday, 2025-01-17			qwsa			
Saturday, 2025-01-18						
Sunday, 2025-01-19						
Monday, 2025-01-20						
Tuesday, 2025-01-21						
Wednesday, 2025-01-22						
Thursday, 2025-01-23						
Friday, 2025-01-24						

Save

The database did update as expected:

16	16	24	3	qwsa	Friday, 2025-01-17 7:00		
----	----	----	---	------	-------------------------	--	--

I also realised here that the calendar didn't load back up with what was just saved, instead it cleared the calendar. However, I didn't think this was an immediate issue as I could trigger it to load back up with the relevant data when I had created the necessary code to load a holiday onto the calendar.

When I tried to test saving no activities to cover those bases, the expected result occurred with nothing being added in the database and the calendar still reopening.

To then work on the loading calendar process, I began making the container for the dashboard in which the user can select a holiday and its calendar to load:

```

17 <div class="holiday-selector">
18   <form action="/calendar" method="GET">
19     <select name="holidayID" required>
20       <option value="">Select a holiday to view</option>
21       {% for holiday in user_holidays %}
22         <option value="{{ holiday.id }}">{{ holiday.title }} ({{ holiday.destination }})</option>
23       {% endfor %}
24     </select>
25     <button type="submit" class="btn-view-holiday">View Holiday</button>
26   </form>
27 </div>

```

This makes a form out of a dropdown menu which displays all of the holidays that the user has created. This assumes (with Jinja2) that the data of the holidays is passed in with as a dictionary called 'holiday' so this will have to be accounted for in the Python.

```

168 holidayConnection = sqlite3.connect('users.db') #Creates dropdown list of holidays
169 try:
170     holidayCursor = holidayConnection.cursor()
171     holidayCursor.execute("""
172         SELECT holidayID, holidayTitle, countryName
173         FROM holidays
174         WHERE userID = ?,
175         """, (session['user_id'],))
176     user_holidays = [] #Creates an empty array to store the user's holidays
177     for row in holidayCursor.fetchall():
178         user_holidays.append({'id': row[0], 'title': row[1], 'destination': row[2]})
179
180 finally:
181     holidayConnection.close()

```

I added this to the dashboard route in app.py, and I also made sure to return user_holidays when the dashboard is rendering.

Testing the page gave the following error:

OperationalError

sqlite3.OperationalError: near ",": syntax error

Traceback (most recent call last)

```

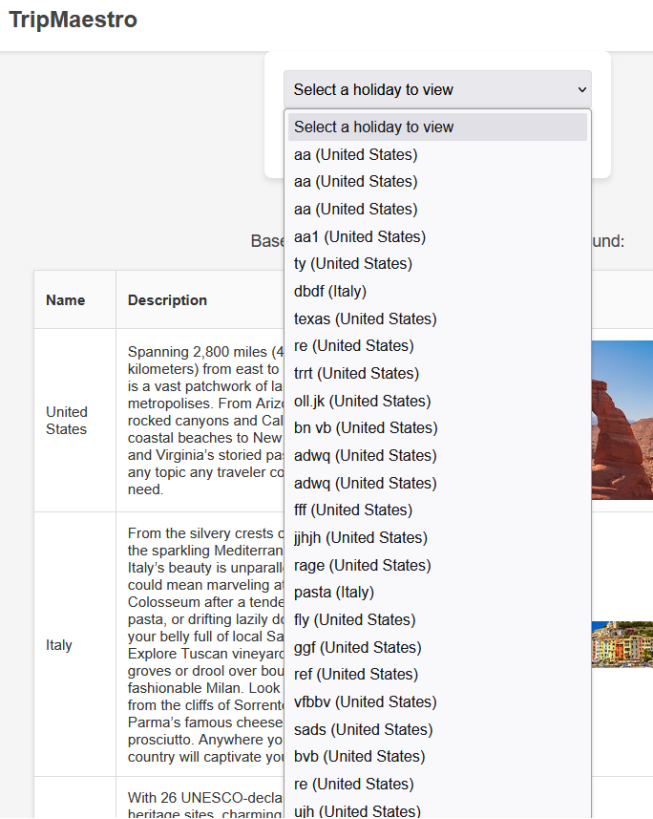
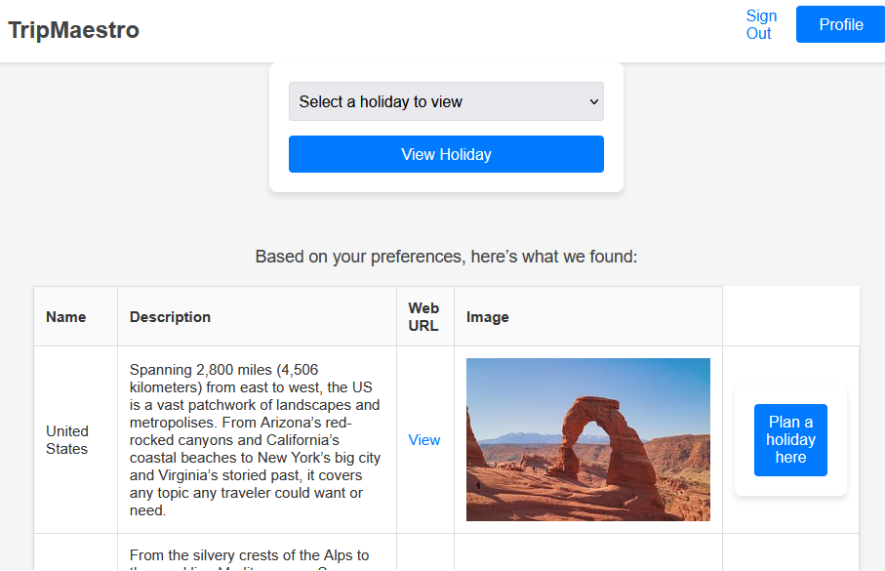
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1536, in __
    return self.wsgi_app(environ, start_response)
    ~~~~~^~~~~~
File "C:\Users\mabed\AppData\Roaming\Python\Python312\site-packages\flask\app.py", line 1514, in ws
    response = self.handle_exception(e)

```

```
174 WHERE userID = ?,  
175      "", (session['user_id'],))  
176 user.holidays = [] #Creates an empty
```

This turned out to be a comma left on line 174 causing the syntax error.

After fixing this, the dashboard updated to include the new dropdown:



To aid with testing as well as user experience, I made it so pressing the top left TripMaestro title returns you to the dashboard.

```
<a href="{{ url_for('dashboard') }}" class="title-link"><h1>TripMaestro</h1></a>
```

When I then went to test a saved holiday though, nothing loaded:

I checked the database to ensure the correct data was being added there and at first thought it was:

17	26	3	Sleep	Saturday, 2025-07-19 8:00
18	26	3	Visiting	Saturday, 2025-07-19 9:00
26	2025-07-19	2025-08-02	Mexico	3 Summer Holiday

However, I realised that this wasn't an acceptable format for the Python to process the time of the activity. It would be better to split it into date, and time for the Python to then handle them separately. This can be achieved by using the `.split()` in the JavaScript when giving the 'date' value to the Python:

```

79     document.getElementById('save-form').addEventListener('submit', function(event) {
80         const activities = [];
81         document.querySelectorAll('.calendar-cell').forEach(cell => {
82             if (cell.getAttribute('data-activity-id')) {
83                 const date = cell.getAttribute('data-date').split(', ')[1]; //Splits the date string to get the date
84                 activities.push({
85                     activityId: cell.getAttribute('data-activity-id'),
86                     startTime: `${date} ${cell.getAttribute('data-hour')}:00`
87                 });
88             }
89         });
90     });

```

Edited line 83 to add `.split`

Testing this did prove to be slightly challenging as the changes made affected what was written into the database from when the form was saved. This meant that loading the calendar didn't have the corrected dates to use in the database. To overcome this, I had to clear the database, and make new holidays and activities that now were saved in the correct format.

Now when going back to the dashboard to load the calendar up again, the same issue appeared, where nothing actually displayed.

I realised that the reason for this was that there was nothing that was actually writing to the calendar cell, so I made a new function in JavaScript that got the JSON of calendar data, and went through each cell to assign its attributes.

```

80 document.addEventListener('DOMContentLoaded', function() {
81     const savedData = JSON.parse(document.getElementById('saved-calendar-data').value); //Gets calendar data
82
83     savedData.forEach(activity => { //Iterates through each activity
84         const activityDate = activity.time.split(' ')[0]; // Get YYYY-MM-DD
85         const activityHour = activity.time.split(' ')[1].split(':')[0]; // Get HH
86
87         //Finds the corresponding cell for that date and time
88         const cells = document.querySelectorAll('.calendar-cell');
89         cells.forEach(cell => {
90             const cellDate = cell.getAttribute('data-date').split(', ')[1]; // Get YYYY-MM-DD
91             const cellHour = cell.getAttribute('data-hour');
92
93             //If this is the right cell, populate it
94             if (cellDate === activityDate && cellHour === activityHour) {
95                 cell.textContent = activity.name;
96                 cell.setAttribute('data-activity-id', activity.id);
97                 cell.setAttribute('data-name', activity.name);
98                 cell.setAttribute('data-location', activity.location);
99                 cell.setAttribute('data-notes', activity.notes);
100             }
101         });
102     });
103 });

```

This function took the calendar data as a JSON, and looked at each activity to find the cell in which it should go into. When it finds its correct one, it changes the text content of the cell to the activity name, and assigns all of its relevant attributes.

When testing this, I ensured a holiday was saved with activities as shown below:

TripMaestro

Enter a title to save holiday

Add New Activity

Delete Activity

Currently Selected Activity

Name: None

Time: None

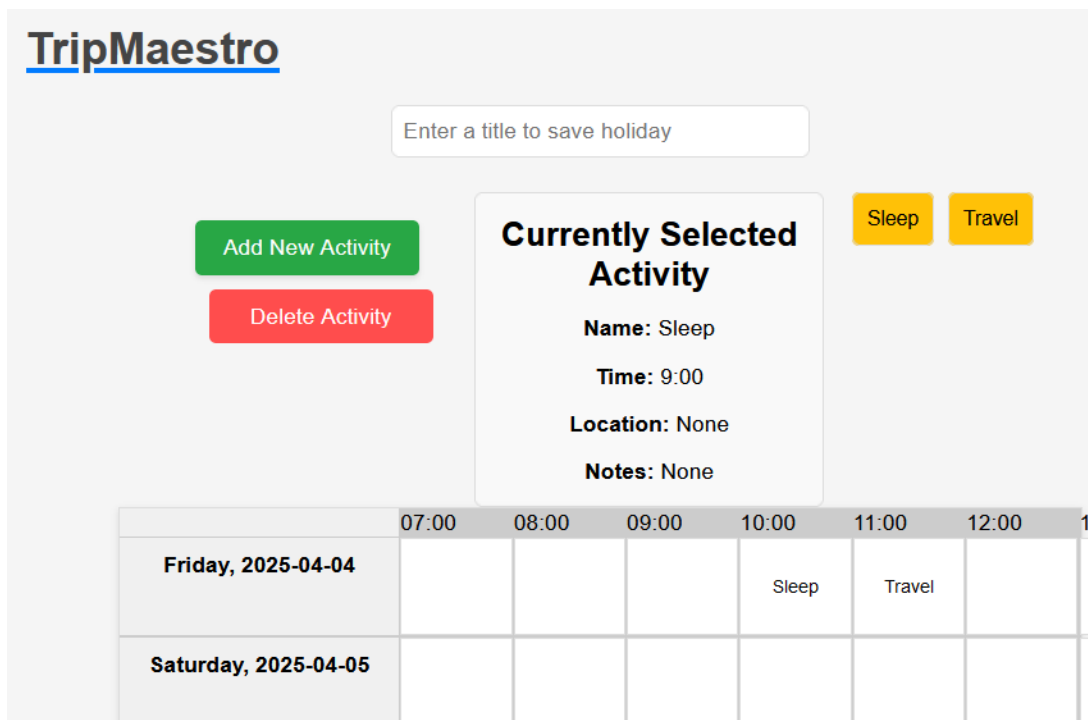
Location: None

Notes: None

Sleep **Travel**

	07:00	08:00	09:00	10:00	11:00	12:00
Friday, 2025-04-04	Sleep	Sleep	Sleep	Sleep	Travel	
Saturday, 2025-04-05						

However, when loading back onto the page, I saw the following calendar:



This makes sense given the approach taken to assigning the activities to the cell, where it assumes that each activity has a single, corresponding cell.

To fix this, I uniquely identified each activity through combining the day of the week the activity lies on, as well as the time of it.

```

80     document.addEventListener('DOMContentLoaded', function() {
81         const savedData = JSON.parse(document.getElementById('saved-calendar-data').value); //Gets
82
83         savedData.forEach(activity => { //Iterates through each activity
84             const parts = activity.time.split(' '); // Split into ["Friday,", "2025-04-04", "7:00"]
85             const activityDate = parts[1]; // "2025-04-04"
86             const activityHour = parts[2].split(':')[0]; // "7"
87
88             //Finds the corresponding cell for that date and time
89             const cells = document.querySelectorAll('.calendar-cell');
90             cells.forEach(cell => {
91                 const cellDate = cell.getAttribute('data-date').split(', ')[1]; // Get YYYY-MM-DD
92                 const cellHour = cell.getAttribute('data-hour');
93
94                 // If this is the right cell, populate it
95                 if (cellDate === activityDate && cellHour === activityHour) {
96                     cell.textContent = activity.name;
97                     cell.setAttribute('data-activity-id', activity.id);
98                     cell.setAttribute('data-name', activity.name);
99                     cell.setAttribute('data-location', activity.location);
100                    cell.setAttribute('data-notes', activity.notes);
101                }
102            });
103        });
104    });

```

This allowed the saving system to work fully, saving the below calendar with duplicated items as was required by the success criteria.

Add New Activity
Delete Activity

Currently Selected Activity

Name: sleep

Time: 9:00

Location: None

Notes: None

Sleep Travel sleep Sleep slepe aaa

	07:00	08:00	09:00	10:00	11:00	12:00
Friday, 2025-04-04	Sleep	Sleep	Sleep	Sleep	Sleep	
Saturday, 2025-04-05		Sleep	Sleep	Sleep	Travel	
Sunday, 2025-04-06			Travel			
Monday, 2025-04-07						
Tuesday, 2025-04-08						

It is also important to test the ability for the calendar, when saved, to retain additional details such as notes and location, below shows the successful results:

New Activity
Delete Activity

Currently Selected Activity

Name: Visits

Time: 8:00

Location: Hills

Notes: Great weather, take the camera

Sleep Travel sleep

	07:00	08:00	09:00	10:00
Friday, 2025-04-04			sleep	
Saturday, 2025-04-05		Visits		as
Sunday, 2025-04-06	Sleep		Travel	
Monday, 2025-04-07				
Tuesday, 2025-04-08	slepe			

Adding in the budget management feature:

This requires a new placeholder being added to the new activity page to hold the cost:

```

52
53
54 <div class="form-group">
55   <label for="activity-cost">Cost (£):</label>
56   <input type="number" id="activity-cost" name="activity_cost" min="0" step="0.01" placeholder="Enter activity cost">
57 </div>
58 <button type="submit">Save</button>

```

This adds a new box for the user to add the activity cost, and is validated using the attributes “min” and “step”, so the user can only type something to 2 decimal places, and can’t be negative.

Opening this page shows a successful implementation:

The screenshot shows a web form titled "Go back to holiday list" at the top. The form contains the following fields:

- Activity Name:** A text input field with the placeholder "Enter activity name".
- Date: (optional):** A date input field with the placeholder "dd/mm/yyyy" and a calendar icon.
- Location (optional):** A text input field with the placeholder "Enter location".
- Link to Booking (optional):** A text input field with the placeholder "Enter a link to booking, include https://www..".
- Notes:** A text area with the placeholder "Add any additional notes here".
- Cost (£):** A number input field with the placeholder "Enter activity cost".

A blue "Save" button is located at the bottom of the form.

This form data also needs to be taken into the Python so it can be added to the database. This is achieved by adding line 330 and line 338 where the database is added to:

```

328 booking_link = request.form.get('booking_link')
329 additional_notes = request.form.get('additional_notes') #Gets all d
330 activity_cost = request.form.get('activity_cost') or 0
331
332 conn = sqlite3.connect('users.db')

```

```

ngLink, additionalNotes, activityName, price)
ink, additional_notes, activity_name, activity_cost)])

```

This also required an update to the database, with additions to the activities and holidays tables, with price and cost being added respectively. To make the cost of the holiday synchronise with the sum of the activities, I've added a separate line of code that updates "cost" in the holidays table by taking the sum of the activities that have the relevant holidayID:

```

338     """ (holidayID, user_id, activity_date, activity_location, booking_link, additional_notes, activity_name, activity_cost))
339     #Inserts data into database
340     cursor.execute("SELECT SUM(price) FROM activities WHERE holidayID = ?", (holidayID,)) #Gets the total cost of all activities
341     total_cost = cursor.fetchone()[0] or 0 #Gives sum of all prices, or 0 if no activities
342     cursor.execute("UPDATE holidays SET cost = ? WHERE holidayID = ?", (total_cost, holidayID)) #Updates the total cost of the holiday for relevant holiday
343     conn.commit()
344     finally:
345         conn.close()

```

I then tested this like below, with the database result on the right:

[Go back to holiday list](#)

Activity Name:

Date: (optional)

Location (optional):

Link to Booking (optional):

Notes:

Cost (£):

[Save](#)

14	eherdgdgdf	Sunday, 2025-01-19 14:00				NULL
3	good times					123
3	bus tour					249.99

I also tested a cost of 3 decimal places to see if the validation worked:

[Go back to holiday list](#)

Activity Name:

Date: (optional)

Location (optional):

Link to Booking (optional):

Notes:

Cost (£):

[Save](#)

Cost (£):

Please select a valid value. The two nearest valid values are 154.34 and 154.35.

This shows a successful test as it isn't permitted to give a price that includes less than 1p.

To then show this cost to the user, I first updated calendar.html to hold a Jinja2 template for the cost of the holiday:

```

21
22     <div class="budget-info">
23         <h3>Budget Summary</h3>
24         <p>Total Cost: £{{ "%.2f"|format(total_cost|float) }}</p> <!-- Formats the total cost to 2 decimal places -->
25     </div>
26
27     <div class="vertical-strip">

```

I then edited the Python to pass the cost onto this HTML file:

```

259     dateCursor = dateConnection.cursor()
260     dateCursor.execute("SELECT startDate, endDate, cost FROM holidays WHERE holidayID =
261     holiday_data = dateCursor.fetchone()
262     dates = holiday_data[:2] #Extracts the start and end dates
263     total_cost = holiday_data[2] #Extracts the cost of the holiday
264     finally:
265         dateConnection.close()

```

I first had to also take the cost from the database along with the dates, and then separated it from the dates by using `holiday_data[:2]` and `holiday_data[2]`.

I also added the `total_cost` being returned when the template was displayed, along with bringing in the cost in another of the SQL queries.

Testing this was successful but had a slightly imperfect implementation as it doesn't take from the activities on the calendar. Instead, it adds all of the activities that have been made and their respective prices. This is shown below with the test that shows how it displays the total cost of the activities I circled in red compared to what's in the calendar.

The screenshot shows a web application interface. At the top, there is a text input field labeled "Enter a title to save holiday". Below it, there is a "Budget Summary" section showing "Total Cost: £372.90". To the left of the budget summary, there are two buttons: "Add New Activity" (green) and "Delete Activity" (red). To the right of the budget summary, there is a "Currently Selected Activity" section with fields for "Name: None", "Time: None", "Location: None", and "Notes: None". Below this, there are two buttons: "good times" and "bus tour", which are circled in red. At the bottom, there is a calendar grid showing dates from Saturday, 2025-01-11 to Monday, 2025-01-13. The calendar grid has columns for each hour from 07:00 to 22:00. On Saturday, 2025-01-11, the activities "good times" and "bus tour" are visible in the calendar cells.

This tested each of the attributes that the stakeholder wanted, also considering the events in which certain spots are left blank. However, it is difficult to cause this in the first place due to the nature of many of the input boxes having the `<required>` argument, meaning the user isn't allowed to put an empty value, or one of an incorrect datatype.

This completes the entire solution, meeting the requirements of the stakeholder. It requires the following libraries to be imported: flask, sqlite3, requests, datetime, json

The code is maintainable with containers holding more data than is immediately needed, such as a calendar cell containing basically every attribute that may be required in the future. This makes expanding the future capabilities of the website easier, and also means it is clearer where attributes originate from, with entire dictionaries of information being moved from Python to JavaScript as opposed to one string at a time.

Object Oriented Programming (OOP) has been utilised to make the usage of the API easier, which massively helped the collection of data for the holidays. The implementation of this seems like an interesting way of tackling the holiday management in Python, but the nature of Flask and specific functions being created for certain routes means there was less opportunity to use OOP.

Post development tests would largely involve the calendar due to the rest of the website being limited in how a user could input to it. On the other hand, the calendar can have arrangements of different activities, each with unique information, but also others with identical information. This would test how the database and website handles the repeating information.

On reflection, very few unit tests were possible in my opinion during the development of this project. The nature of a HTML project means that lots of information relies on previous stages, such as routes or JSON files from preceding JavaScript. This makes it unlikely that post development tests would uncover scenarios that haven't been covered already. I do believe that the fact that lots of "information relies on previous stages" does partly compromise on the modularity of the project, although with functions and templates being returned with predictable contents, I do believe that the project is simple to follow and logical, and is an effective solution for what the stakeholder is looking for.

Testing and Evaluation

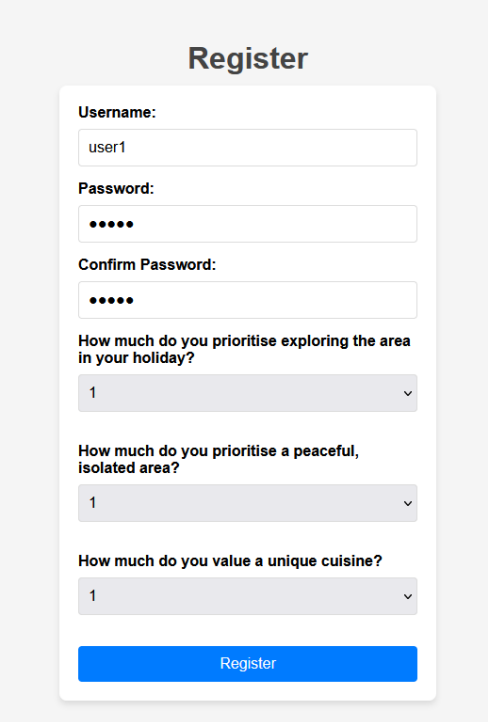
Testing for function:

Test number	What is being tested	Test data	Expected outcome	Test type	Success?
1	Registration system	"user1", "pass1", "pass1", 1,1,1	Opens login page, also makes a new entry in database.	Normal	Success
2	Registration system	"a", "a", "a", 1, 1, 1	Username and password is less than 3 characters so the user isn't allowed to make an account with that.	Invalid	Failure
3	Registration system	""", """, """, 2, 2, 2	There is no username and password so no account should be made; user to be prompted to sign in again.	Invalid	Success
4	Registration system	"user3", "pass3", "pass33", 1,1,1	Second and third box are supposed to be the password and confirmation of password respectively, and as they don't match, the user should be prompted to try again	Invalid	Success
5	Login system	"user1", "pass1"	Opens the dashboard for the user as the username and password is correct.	Normal	Success
6	Login system	"blahblah", "blahblah" (incorrect login)	The username and password doesn't match anything in the database so the user should be prompted to sign in again.	Invalid	Success
7	Holiday recommendations	Uses ratings from account, test ratings of 1,1,10	Dashboard opens to show holiday destinations that have a unique cuisine.	Normal	Success
8	Holiday recommendations	Uses ratings from account, test ratings of 10,1,1	Dashboard opens to show holiday destinations that are good to explore.	Normal	Success
9	Holiday recommendations	Pressing "Next Page" on holiday list using a profile with ratings of 1,1,10	Opens another 4 holiday destinations that match the user's preferences.	Normal	Success
10	Holiday recommendations	Pressing "View" on each holiday destination	Opens the TripAdvisor page about that destination	Normal	Success
11	Holiday recommendations	Pressing "Plan a holiday here"	Opens the page to make a new holiday for the user	Normal	Success
12	New holiday	""", null, null	User isn't allowed to create a new holiday without a start and end date so is prompted to try again	Invalid	Success
13	New holiday	"my holiday", 1/1/22, 1/2/22	User isn't allowed to create a holiday with dates that are in the past so is prompted to try again	Invalid	Success
14	New holiday	"my holiday", 1/6/25, 1/10/25	User isn't allowed to create a holiday with dates that are more	Invalid	Success

			than 90 days apart so is prompted to try again		
15	New holiday	""", 1/6/25, 1/7/25	User needs the holiday to have a name else they cannot recognise it when selecting which to open, so they are prompted to try again	Invalid	Success
16	New holiday	"my holiday", 1/6/25, 1/7/25	Calendar page opens for the user with a new record being created in the database for this holiday	Normal	Success
17	Calendar	Pressing "Add new activity"	Opens page to fill in details of a new activity	Normal	Success
18	New activity	All fields blank	User prompted to try again	Invalid	Success
19	New activity	"my activity", rest blank	Activity is created along with its record in the database, and user is taken to the calendar page	Normal	Success
20	New activity	"my activity", £34.95, rest blank	Activity is created along with its record in the database, and user is taken to the calendar page, with the total cost updated to reflect the cost of that activity	Normal	Success
21	Calendar	Dragging an activity onto calendar	The name of the activity is entered into the calendar	Normal	Success
22	Calendar	Clicking on an activity that is in the calendar	The details of that activity are displayed to the user in the "Currently Selected Activity"	Normal	Success
23	Calendar	Pressing "Delete Activity" and then clicking on activities in the calendar	The activity is removed from the calendar (but still exists outside of the schedule so can be dragged back on)	Normal	Success
24	Calendar	Blank title text box, and an activity in the schedule being saved using "Save" button	Makes the holiday loadable from the dashboard, and also amends the database so that the start times of the activities in its holiday are stored. Title in database isn't changed as blank box is to mean that old title is fine	Boundary	Success
25	Calendar	Blank title text box, with multiple instances of the same activity placed in calendar	User prevented from saving the calendar as there is only meant to be one instance of each activity	Invalid	Failure
26	Calendar	"New holiday title", activity in schedule being saved using "Save" button	Makes the holiday loadable from the dashboard, and also amends the database so that the start times of the activities in its holiday are stored. Title in database is changed to what was put in the text box.	Normal	Failure
27	Sign out button	Pressing on "Sign Out" on any page	Clears user's session and returns user to login page	Normal	Success
28	Opening holiday	Without selecting on a holiday, pressing on "View Holiday"	User is prompted to select a holiday from the list, they are not redirected to another page	Invalid	Success

Screenshots and results of the tests:

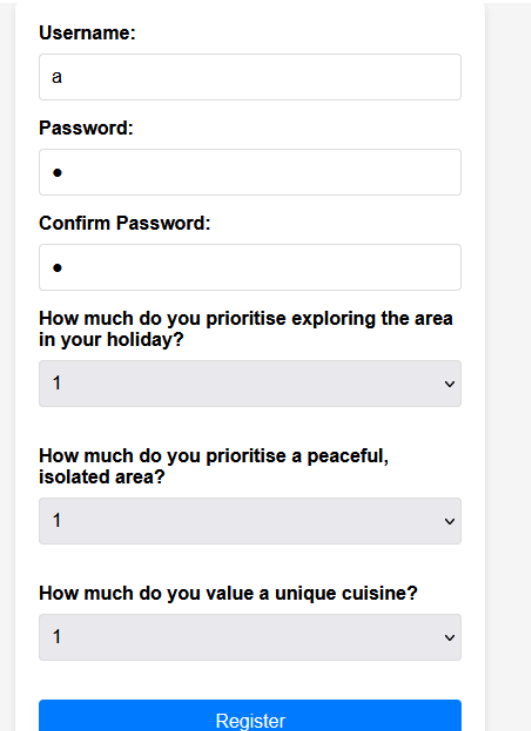
Test 1:



15	15	user1	pass1	1	1	1
----	----	-------	-------	---	---	---

This tested a normal username and password also with normal ratings. The ratings are automatically validated as they are a choice between 1 and 10.

Test 2:



This tested a username and password that is too small; it is meant to be validated to be between 3 and 24 characters and this is too short. However, this being added to the database shows this as being a permissible username and password meaning this was allowed when it shouldn't have.

3	3	a	a	1	1	1
---	---	---	---	---	---	---

Test 3:

The image displays two versions of a 'Register' form side-by-side. Both forms have the title 'Register' at the top. The left form contains the following fields: 'Username:' (empty text box), 'Password:' (empty text box), 'Confirm Password:' (empty text box), and three dropdown menus with the questions: 'How much do you prioritise exploring the area in your holiday?', 'How much do you prioritise a peaceful, isolated area?', and 'How much do you value a unique cuisine?'. Each dropdown menu has the value '2' selected. At the bottom is a blue 'Register' button. The right form is identical but has a browser password manager popup over the 'Username:' field. The popup text reads: 'Protect your email address: Use Firefox Relay email mask' and 'Manage Passwords'. The 'Confirm Password:' field and the three dropdown menus are also visible on the right form.

This test is for when the boxes are left blank, and this made my browser prompt me to fill in the box. This means that the code did not progress to save the username and password, showing a successful test.

Test 4:

The image shows a 'Register' form with an error message at the top: 'Passwords do not match!'. The 'Username:' field is filled with the text 'user3'. The 'Password:' and 'Confirm Password:' fields are filled with dots, indicating they are not visible. The three dropdown menus are also present, with the first two having the value '1' selected. The blue 'Register' button is at the bottom.

Top text shows that the passwords do not match, and no new entry is created in the database. The page is reloaded with empty text boxes for the user to then fill in.

Test 5:

Welcome to TripMaestro

Username:

Password:

Login

Don't have an account? [Sign up](#)

This test is a normal test using login data that should lead to an account. When pressing "Login" this opened the main dashboard as shown below:

TripMaestro

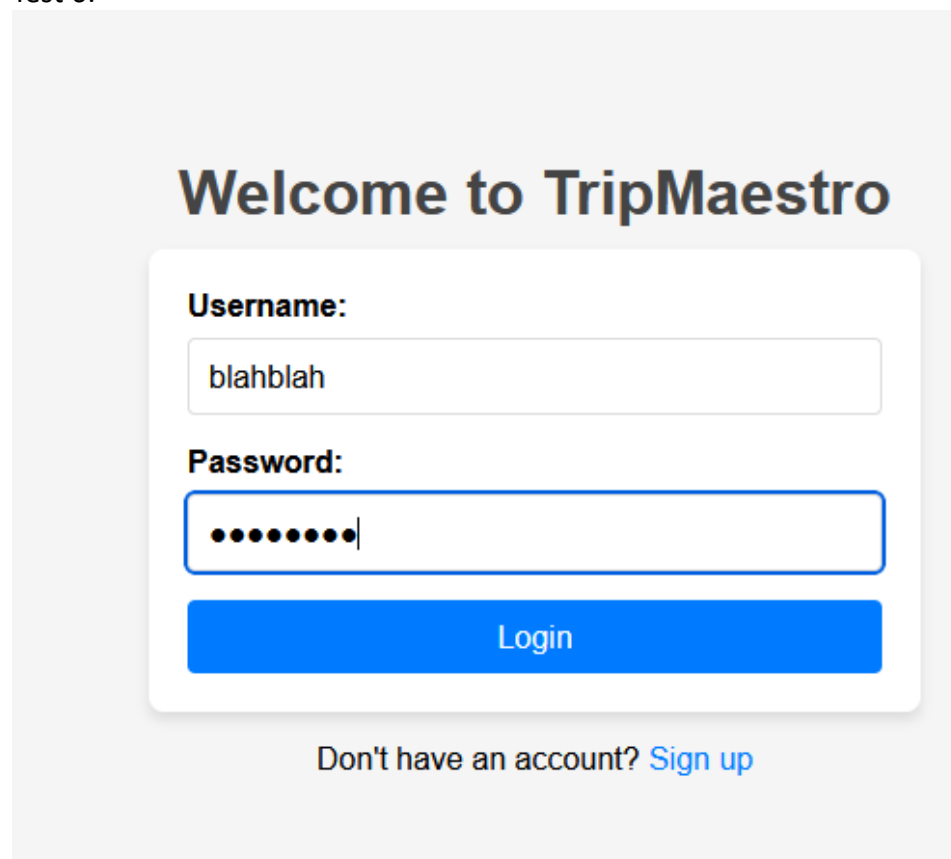
Sign OutProfile

Select a holiday to view▼
View Holiday

Based on your preferences, here's what we found:

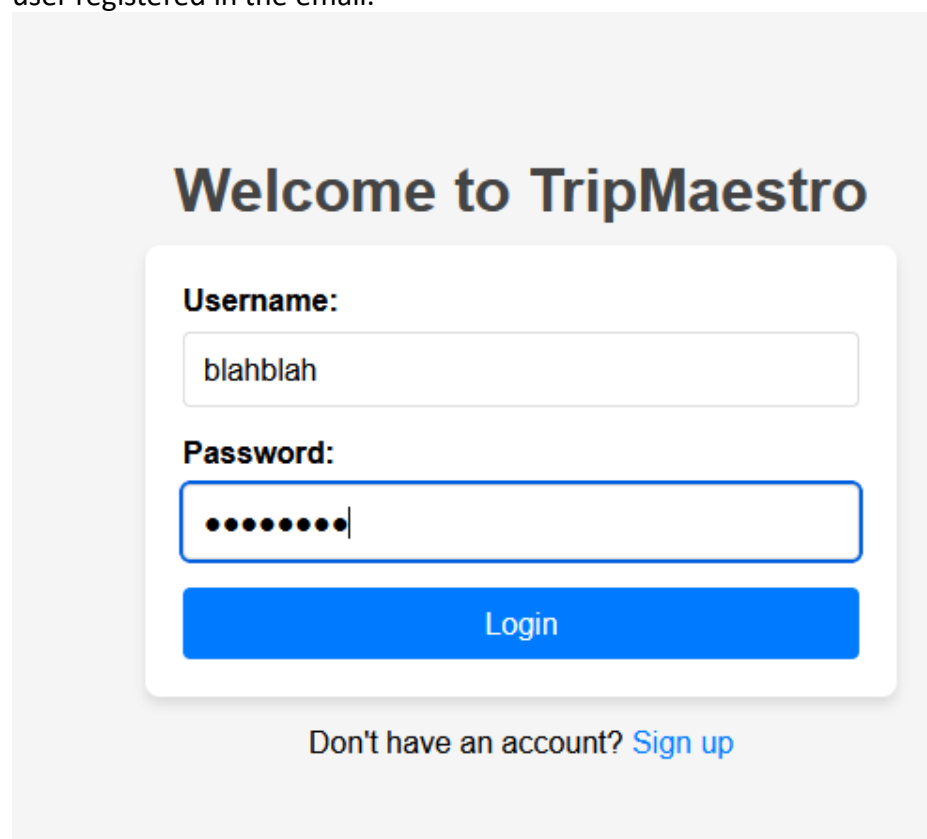
Name	Description	Web URL	Image	
Italy	From the silvery crests of the Alps to the sparkling Mediterranean Sea, Italy's beauty is unparalleled. A visit could mean marveling at the Colosseum after a tender plate of pasta, or drifting lazily down a canal, your belly full of local Sangiovese. Explore Tuscan vineyards and olive groves or drool over boutiques in fashionable Milan. Look for mermaids from the cliffs of Sorrento or nibble on Parma's famous cheese and prosciutto. Anywhere you choose, the country will captivate you.	View		Plan a holiday here
United States	Spanning 2,800 miles (4,506 kilometers) from east to west, the US is a vast patchwork of landscapes and metropolises. From Arizona's red-rocked canyons and California's coastal beaches to New York's big city and Virginia's storied past, it covers any topic any traveler could want or need.	View		Plan a holiday here
With 26 UNESCO-declared world				

Test 6:



The screenshot shows the TripMaestro login interface. At the top, the heading "Welcome to TripMaestro" is displayed in a large, bold, dark font. Below the heading is a white login form with rounded corners. Inside the form, the "Username:" label is followed by a text input field containing the text "blahblah". Below this, the "Password:" label is followed by a password input field with a blue border and a blue outline, containing ten black dots and a vertical cursor. A blue "Login" button is positioned below the password field. At the bottom of the form, the text "Don't have an account? [Sign up](#)" is displayed, with "Sign up" as a blue link.

This represents an incorrect username and password, and this is supposed to be rejected by the website. An incorrect login popup appears as below to represent the fact that this isn't a user registered in the email.



This screenshot is identical to the one above, showing the TripMaestro login page. It features the "Welcome to TripMaestro" heading, a login form with a username field containing "blahblah", a password field with ten dots and a cursor, a blue "Login" button, and a "Sign up" link at the bottom.

Test 7:

This test checks the functionality of the recommendation system, and specifically the cuisine ratings. The account currently logged in had ratings of 1,1 and 10 respectively with the 10 in cuisine rating. This is automatically validated through the dropdown, and the result is shown below. The cuisine ratings seem to be working as the holidays being displayed are of world renowned places for their cuisines, and also are roughly in order of the cuisine ratings that are stored in the database.

TripMaestro[Sign Out](#)[Profile](#)

Select a holiday to view

[View Holiday](#)

Based on your preferences, here's what we found:

Name	Description	Web URL	Image	
Italy	From the silvery crests of the Alps to the sparkling Mediterranean Sea, Italy's beauty is unparalleled. A visit could mean marveling at the Colosseum after a tender plate of pasta, or drifting lazily down a canal, your belly full of local Sangiovese. Explore Tuscan vineyards and olive groves or drool over boutiques in fashionable Milan. Look for mermaids from the cliffs of Sorrento or nibble on Parma's famous cheese and prosciutto. Anywhere you choose, the country will captivate you.	View		Plan a holiday here
Japan	In the Land of the Rising Sun, ancient temples sit alongside neon wonderlands and shinto shrines offer pockets of peace amid metropolises. Add tea ceremonies, snow monkeys, sushi, kimonos, and karaoke to the mix, and you've got one of the world's most fascinating countries.	View		Plan a holiday here
India	From the beaches of sun-soaked Goa to the frenetic bazaars of Mumbai, India offers wealth of vastly different, yet equally enthralling, experiences. Explore the sparkling lakes and palaces of Udaipur, watch traditional Indian dance in Mumbai, and enjoy the best of Indian cuisine.	View		Plan a holiday

Test 8:

This test is very similar to test 7 but tests another category: exploration. The result is shown below and roughly matches that category in the database. It isn't meant to be an exact mirror image of that database as the other factors still play a small part of the overall rating but is enough to imply that the rating system is working as intended.

This satisfies the larger requirement from the stakeholder of a recommendation system based on the user preferences, but doesn't over rely on that based on the fact that the other characteristics still play a (small but existing) part in determining which destinations are displayed to the user.

Select a holiday to view ▼

View Holiday

Based on your preferences, here's what we found:

Name	Description	Web URL	Image	
Italy	From the silvery crests of the Alps to the sparkling Mediterranean Sea, Italy's beauty is unparalleled. A visit could mean marveling at the Colosseum after a tender plate of pasta, or drifting lazily down a canal, your belly full of local Sangiovese. Explore Tuscan vineyards and olive groves or drool over boutiques in fashionable Milan. Look for mermaids from the cliffs of Sorrento or nibble on Parma's famous cheese and prosciutto. Anywhere you choose, the country will captivate you.	View		Plan a holiday here
United States	Spanning 2,800 miles (4,506 kilometers) from east to west, the US is a vast patchwork of landscapes and metropolises. From Arizona's red-rocked canyons and California's coastal beaches to New York's big city and Virginia's storied past, it covers any topic any traveler could want or need.	View		Plan a holiday here
Japan	In the Land of the Rising Sun, ancient temples sit alongside neon wonderlands and shinto shrines offer pockets of peace amid metropolises. Add tea ceremonies, snow monkeys, sushi, kimonos, and karaoke to the mix, and	View		Plan a holiday here

Test 9:

Using test 8 as a base point (so the destinations are sorted by explorative rating), pressing next page opens the next set of destinations that also line up with that rating:

Japan	and snow monkeys offer pockets of peace amid metropolises. Add tea ceremonies, snow monkeys, sushi, kimonos, and karaoke to the mix, and you've got one of the world's most fascinating countries.	View		Plan a holiday here
China	The grand story of China features a great wall, a forbidden city, and a few thousand Terracotta warriors. Settings include the sci-fi skyline of Shanghai, the jutting peaks of Zhangjiajie, and the tropical beaches of Hainan, but the tale of this ancient kingdom began long ago.	View		Plan a holiday here

[Previous Page](#) [Next Page](#)

127.0.0.1:5000/dashboard?page=2

Destinations that loaded:

Select a holiday to view View Holiday				
Based on your preferences, here's what we found:				
Name	Description	Web URL	Image	
France	There's much more to France than Paris—from the fairy-tale châteaux of the Loire Valley to the lavender fields of Provence, and the French Riviera's celebrity-studded beaches. World-famous gastronomy and fine wines provide the perfect complement to the country's alpine views and architectural masterpieces.	View		Plan a holiday here
Mexico	With 26 UNESCO-declared world heritage sites, charming colonial towns and dozens of thrilling cities, there's plenty to explore in this country of 109 million. Outside the cities, stunning Pacific beaches, stark deserts, mangrove swamps and swimming holes provide all you need for a relaxing, romantic or adventurous vacation. Captivating, cosmopolitan and chaotic Mexico City and the 32 states offer an incredible abundance of experiences, from laid-back and leisurely to upbeat to adrenalin-charging.	View		Plan a holiday here
	From sun-drenched archipelagos and bustling urban cities to snow-capped			

Test 10:

This test checks the “View” button in the destination table as shown below (mouse is hovering over the link):

TripMaestro [Sign Out](#) [Profile](#)

Select a holiday to view 

[View Holiday](#)

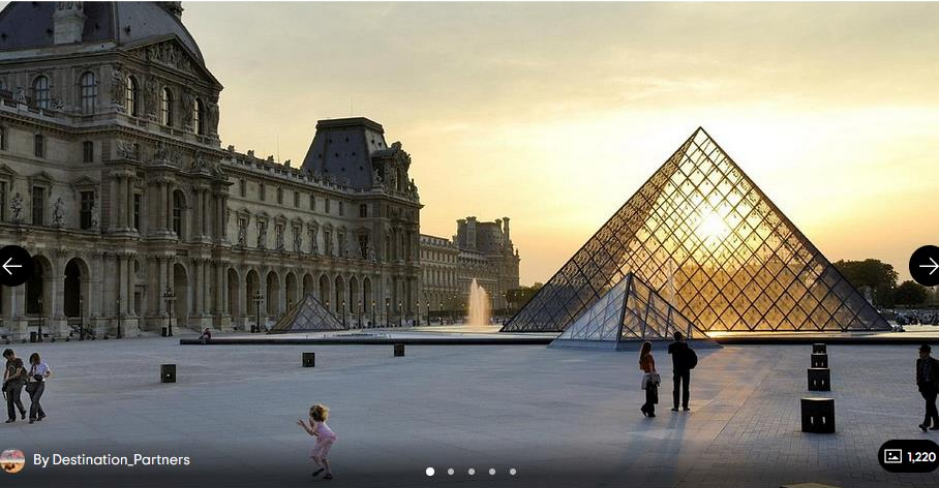
Based on your preferences, here's what we found:

Name	Description	Web URL	Image	
France	There's much more to France than Paris—from the fairy-tale châteaux of the Loire Valley to the lavender fields of Provence, and the French Riviera's celebrity-studded beaches. World-famous gastronomy and fine wines provide the perfect complement to the country's alpine views and architectural masterpieces.	View		Plan a holiday here
Mexico	With 26 UNESCO-declared world heritage sites, charming colonial towns and dozens of thrilling cities, there's plenty to explore in this country of 109 million. Outside the cities, stunning Pacific beaches, stark deserts, mangrove swamps and swimming holes provide all you need for a relaxing, romantic or adventurous vacation. Destination recommendations	View		Plan a holiday here

After completing the captcha the TripAdvisor opens as expected:

 **Tripadvisor**

Europe > France  [Plan Your Trip to France: Best of France Tourism](#)



By Destination_Partners  1,220

France

[Save](#)

There's much more to France than Paris—from the fairy-tale châteaux of the Loire Valley to the lavender fields of Provence, and the French Riviera's celebrity-studded beaches. World-famous gastronomy and fine wines provide the perfect complement to the country's alpine... [Read more](#) 

Essential France

Pick a category to filter your recs

Test 11:

Highlighting over "Plan a holiday here" as shown below:

Based on your preferences, here's what we found:				
Name	Description	Web URL	Image	
France	There's much more to France than Paris—from the fairy-tale châteaux of the Loire Valley to the lavender fields of Provence, and the French Riviera's celebrity-studded beaches. World-famous gastronomy and fine wines provide the perfect complement to the country's alpine views and architectural masterpieces.	View		Plan a holiday here
Mexico	With 26 UNESCO-declared world heritage sites, charming colonial towns and dozens of thrilling cities, there's plenty to explore in this country of 109 million. Outside the cities, stunning Pacific beaches, stark deserts, mangrove swamps and swimming holes provide all you need for a relaxing, romantic or adventurous vacation. Captivating, cosmopolitan and chaotic Mexico City and the 32	View		Plan a holiday here

This can only have a valid input as there is no other way to start a holiday, making it automatically validated as an input. Pressing the button leads to the new holiday page as shown below:

[Back to Dashboard](#)

Destination: France

Holiday Title:

Start Date:



End Date:



[Continue to Planner](#)

It also includes the destination above the input boxes carrying through from the destination that was clicked on in the last page.

Test 12:

Testing all blank fields should be denied by the website and that is shown in the test result below:

The screenshot shows a web form for planning a holiday. At the top, there is a blue link "Back to Dashboard". Below it, the text "Destination: France" is displayed in a large, bold, black font. The form itself is a white card with rounded corners. It contains a label "Holiday Title:" followed by a text input field with the placeholder text "Enter a title for your holiday". A dark grey tooltip with the text "Please fill in this field." is positioned over the input field. Below the title field is a date input field with the placeholder "dd / mm / yyyy" and a calendar icon. The label "End Date:" is positioned above this field. At the bottom of the form is a blue button with the text "Continue to Planner".

[Back to Dashboard](#)

Destination: France

Holiday Title:

Please fill in this field.

End Date:

[Continue to Planner](#)

Test 13:

This tests that the only dates that are valid are in the future:

[Back to Dashboard](#)

Destination: France

Holiday Title:

Start Date:



End Date:



[Continue to Planner](#)

The result is shown below and is a successful test as the user has been blocked from making the holiday:

[Back to Dashboard](#)

Destination: France

Start date must be today or in the future

Holiday Title:

Start Date:



End Date:

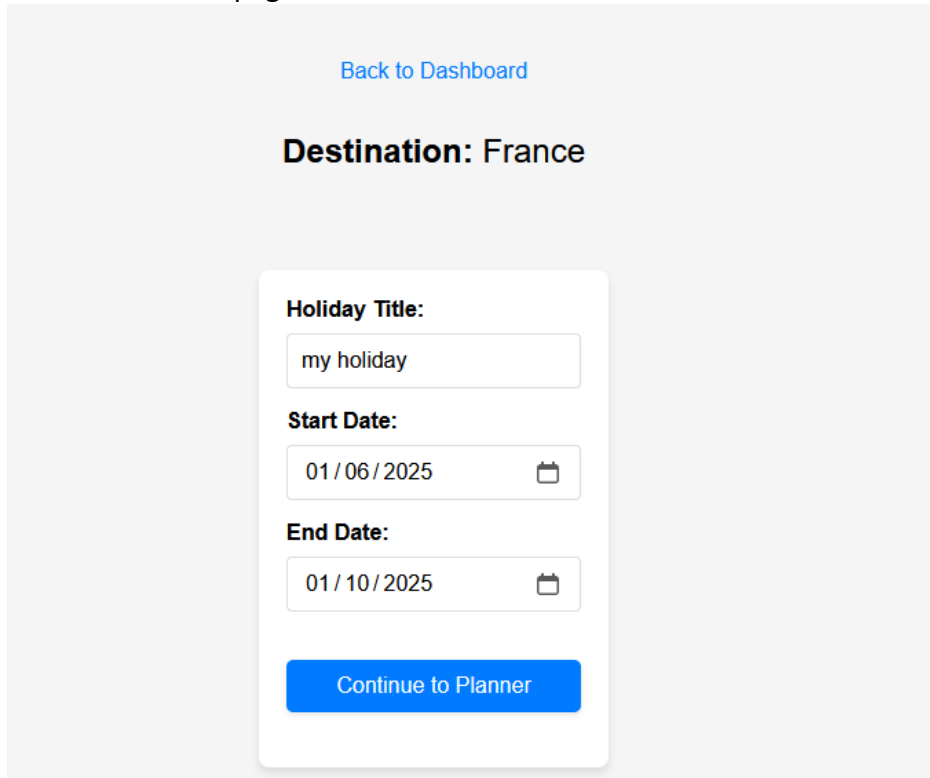


[Continue to Planner](#)

Test 14:

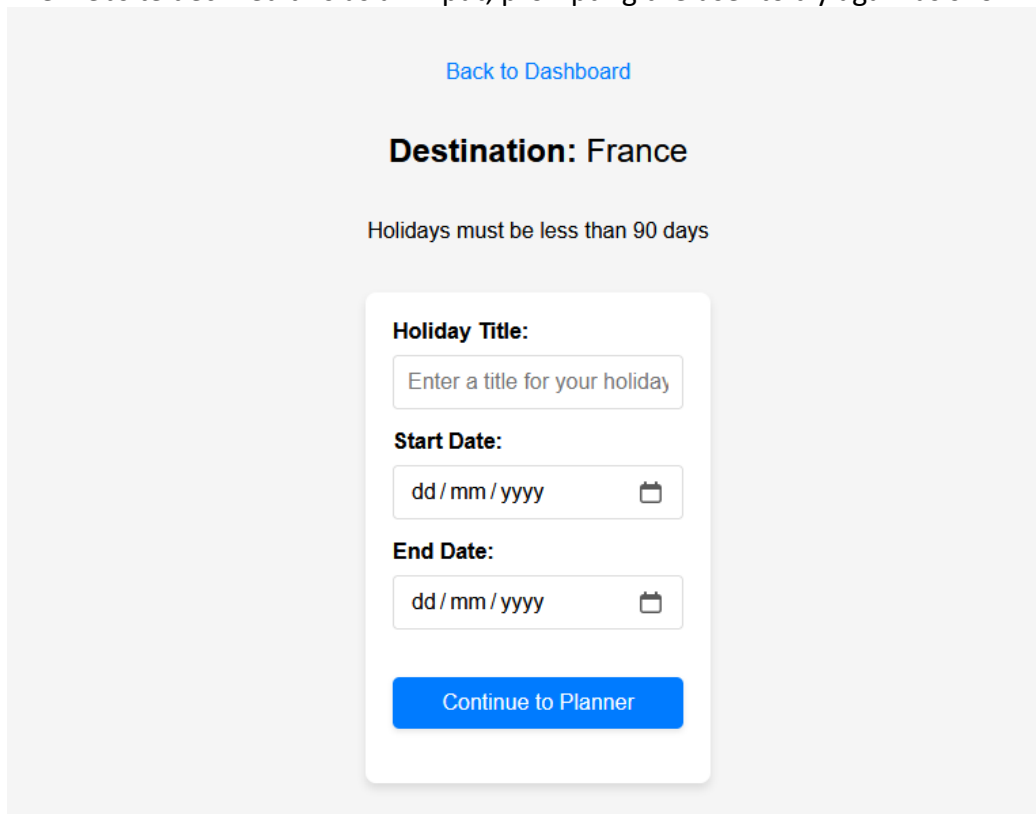
This test is to check that the length of the holiday isn't longer than 90 days. This is to satisfy the stakeholder's intended purpose of the website with how it is meant for recreational holidaymakers as opposed to people moving abroad.

Below shows the page with the test data to check for this validation:



The screenshot shows a web form titled "Destination: France" with a "Back to Dashboard" link. The form contains three input fields: "Holiday Title" with the value "my holiday", "Start Date" with the value "01/06/2025", and "End Date" with the value "01/10/2025". Each date field has a calendar icon. At the bottom is a blue button labeled "Continue to Planner".

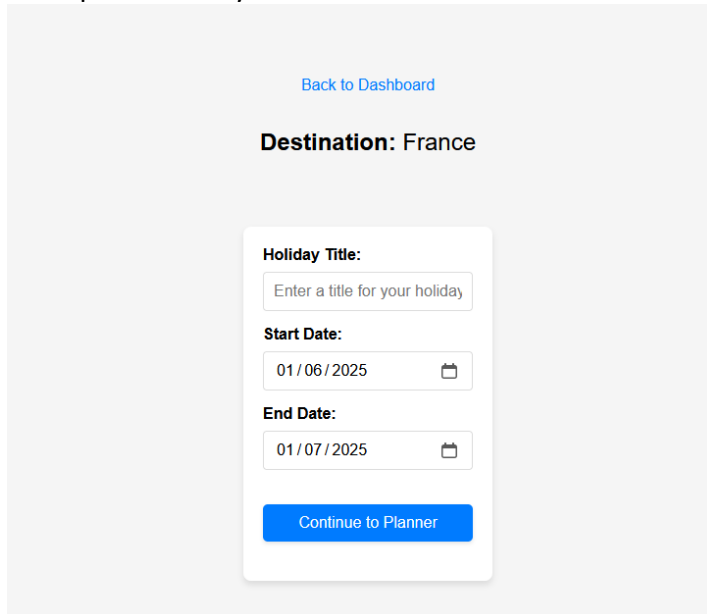
The website declined this as an input, prompting the user to try again as shown below:



The screenshot shows the same web form as above, but with an error message "Holidays must be less than 90 days" displayed below the "Destination: France" header. The input fields are now empty, with placeholder text: "Enter a title for your holiday" for the title, and "dd/mm/yyyy" for both the start and end dates. The "Continue to Planner" button remains at the bottom.

Test 15:

This tests for a holiday being made without a title as shown below, and is supposed to be declined as otherwise it would be difficult for the user to identify their old holidays. This goes against the intention of the stakeholder who wishes for the old holidays to be saved and opened at any time.



Back to Dashboard

Destination: France

Holiday Title:

Enter a title for your holiday

Start Date:

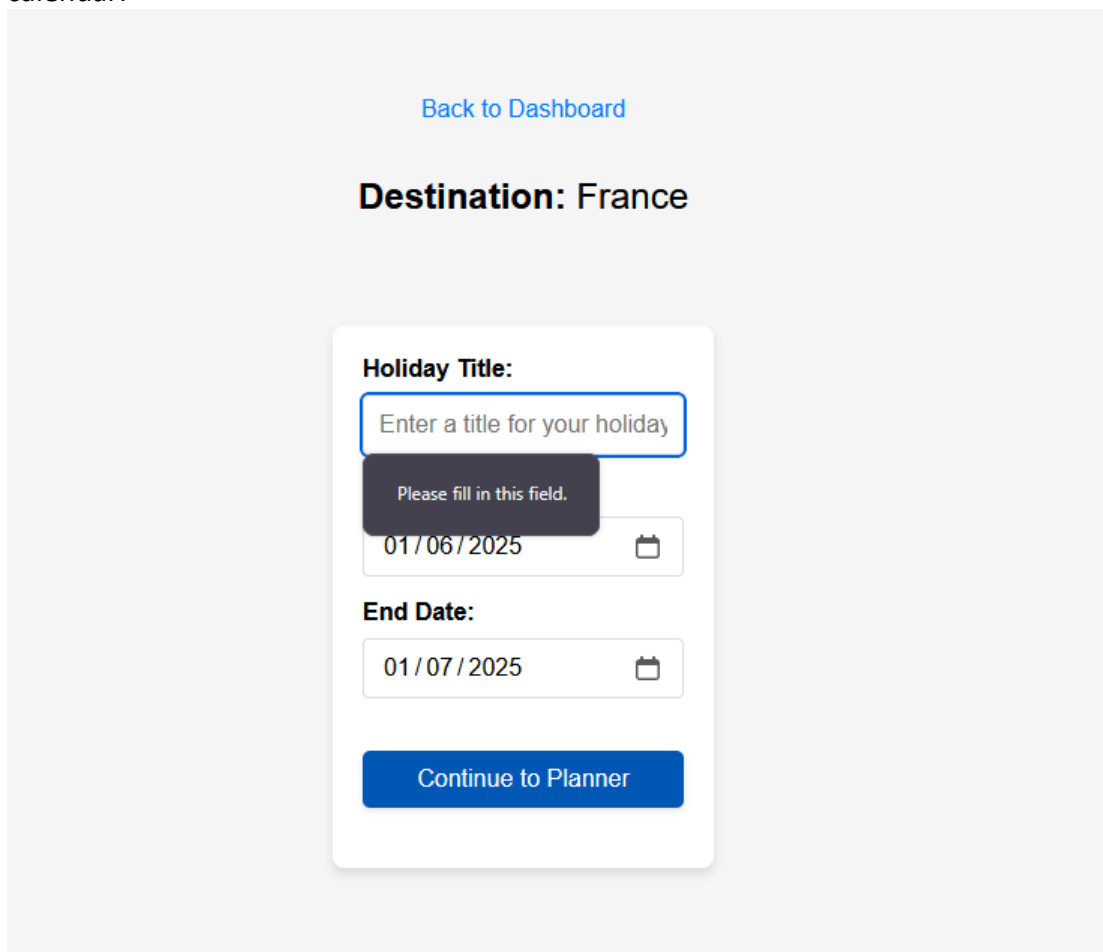
01 / 06 / 2025

End Date:

01 / 07 / 2025

Continue to Planner

Pressing “Continue to Planner” prompts the user to fill in the title field, not continuing to the calendar:



Back to Dashboard

Destination: France

Holiday Title:

Enter a title for your holiday

Please fill in this field.

01 / 06 / 2025

End Date:

01 / 07 / 2025

Continue to Planner

Test 16:

This tests for a normal input when making a holiday and is what the user is expected to enter. This meets the validation criteria of the title existing, the holiday dates being in the future, and also the holiday being less than 90 days in length:

[Back to Dashboard](#)

Destination: France

Holiday Title:

Start Date:

End Date:

[Continue to Planner](#)

The holiday is created successfully in the database, and also sends the user to the calendar page to modify their holiday as shown below.

ripMaestro

[Sign Out](#)

Budget Summary

Total Cost: £0.00

[Add New Activity](#)

[Delete Activity](#)

Currently Selected Activity

Name: None

Time: None

Location: None

Notes: None

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01								
Monday, 2025-06-02								
Tuesday, 2025-06-03								
Wednesday, 2025-06-04								
Thursday, 2025-06-05								

Test 17:

This tests the functionality of the “Add New Activity” button where it should open the page for the user to fill in the details of the activity:

The screenshot shows a user interface for holiday planning. At the top right, a 'Budget Summary' box displays 'Total Cost: £0.00'. Below this, there are two buttons: a green 'Add New Activity' button and a red 'Delete Activity' button. To the right of these buttons is a 'Currently Select Activity' box showing 'Name: None', 'Time: None', 'Location: None', and 'Notes: None'. At the bottom, there is a calendar view for 'Sunday, 2025-06-01' with time slots from 07:00 to 11:00.

Clicking on it opens the following page as expected:

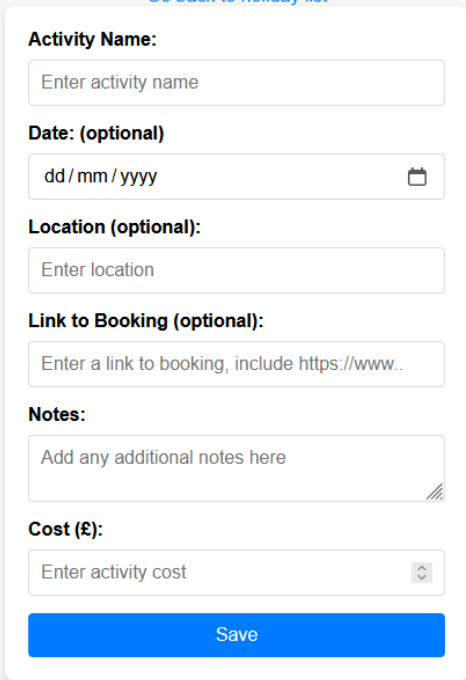
The screenshot shows the 'Add New Activity' form. At the top, there is a link 'Go back to holiday list'. The form contains the following fields:

- Activity Name:** A text input field with the placeholder 'Enter activity name'.
- Date: (optional):** A date input field with the placeholder 'dd / mm / yyyy' and a calendar icon.
- Location (optional):** A text input field with the placeholder 'Enter location'.
- Link to Booking (optional):** A text input field with the placeholder 'Enter a link to booking, include https://www..'.
- Notes:** A text area with the placeholder 'Add any additional notes here'.
- Cost (£):** A text input field with the placeholder 'Enter activity cost' and a dropdown arrow.

At the bottom of the form is a blue 'Save' button.

Test 18:

This tests that enough fields are filled in by the user to make the activity usable on the calendar, and that specifically a fully empty form isn't accepted by the website:

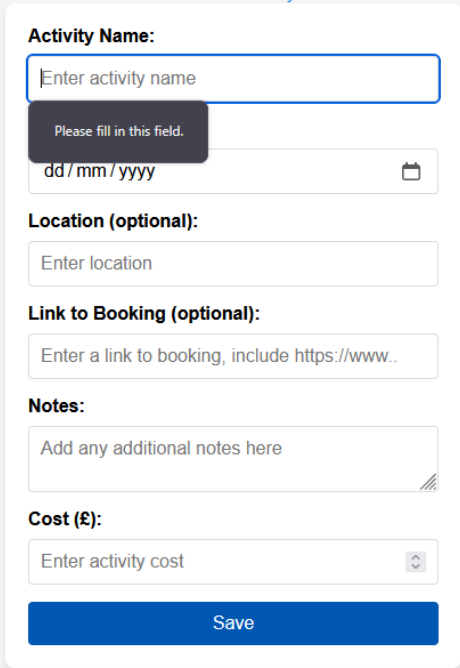


The screenshot shows a form titled "Add Activity" with a link "Go back to holiday list" at the top. The form contains the following fields:

- Activity Name:** A text input field with the placeholder "Enter activity name".
- Date: (optional):** A date input field with the placeholder "dd / mm / yyyy" and a calendar icon.
- Location (optional):** A text input field with the placeholder "Enter location".
- Link to Booking (optional):** A text input field with the placeholder "Enter a link to booking, include https://www..".
- Notes:** A text area with the placeholder "Add any additional notes here".
- Cost (£):** A text input field with the placeholder "Enter activity cost" and a dropdown arrow.

A blue "Save" button is located at the bottom of the form.

This is declined by the website with the browser prompting the name to be filled in. This prevents a blank activity being made, which would be meaningless to the user, and could accidentally take up slots in the calendar when it is just meant to be left blank.



The screenshot shows the same "Add Activity" form, but with a validation error on the "Activity Name" field. The error message "Please fill in this field." is displayed in a dark grey box over the input field. The other fields remain empty, and the "Save" button is still visible at the bottom.

Test 19:

This tests that only the activity name is filled is as shown below:

Go back to holiday list

Activity Name:
my activity

Date: (optional)
dd / mm / yyyy

Location (optional):
Enter location

Link to Booking (optional):
Enter a link to booking, include https://www..

Notes:
Add any additional notes here

Cost (£):
Enter activity cost

Save

This is permitted and is also expected by the program given that this is now useful to the user. This could be used for something generic like sleep, which the user might not want to fill all of the details for. This results in the expected output of the activity being brought onto the calendar page:

Budget Summary
Total Cost: £0.00

Add New Activity

Delete Activity

Currently Selected Activity
Name: None
Time: None
Location: None
Notes: None

my activity

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01								
Monday, 2025-06-02								
Tuesday, 2025-06-03								

Test 20:

This test adds a price to the activity to check that the total cost displayed to the user on the calendar page is accurate. This is tested with the following data entered:

[Go back to holiday list](#)

Activity Name:

Date: (optional)

Location (optional):

Link to Booking (optional):

Notes:

Cost (£):

Save

This test is successful with the calendar page displaying £34.95 (shown below) as the total cost which it should due to it costing that, and the other activity not having a cost. To add to this, the validation input box for the cost when adding a new activity has a minimum of £0 and goes up in increments of 1p meaning that it is self-validating where the user cannot enter a negative price.

Enter a title to save holiday

Budget Summary

Total Cost: £34.95

Add New Activity

Delete Activity

Currently Selected Activity

Name: None

Time: None

Location: None

Notes: None

my activity my activity2

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01								
Monday, 2025-06-02								
Tuesdav. 2025-06-03								

Test 21:

This is to test that the activity can be dragged onto the calendar. This is successful where I had dragged the yellow activity boxes onto the calendar:

The screenshot shows a web application interface. At the top, a 'Budget Summary' box displays 'Total Cost: £34.95'. Below this, on the left, are two buttons: 'Add New Activity' (green) and 'Delete Activity' (red). In the center is a 'Currently Selected Activity' panel with fields for Name, Time, Location, and Notes, all currently set to 'None'. On the right are two yellow activity boxes labeled 'my activity' and 'my activity2'. The bottom section is a calendar grid. The header shows time slots from 07:00 to 14:00. The first row is for 'Sunday, 2025-06-01', and the second row is for 'Monday, 2025-06-02'. In the Sunday row, the 07:00 slot contains 'my activity' and the 08:00 slot contains 'my activity2'. All other slots are empty.

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01	my activity	my activity2						
Monday, 2025-06-02								

This doesn't really require validation as it is either that the user does this or they do not, there is no real variety in what the user can do in terms of dragging an activity on. However it is also worth testing dragging another, different activity into an already occupied time slot, and this results in the following change (my activity was dragged onto the 08:00 slot):

This screenshot shows the same application after a second drag. The 'my activity' box has been moved from the 07:00 slot to the 08:00 slot on Sunday, 2025-06-01. The 07:00 slot is now empty, and the 08:00 slot contains 'my activity'. The 'my activity2' box remains in the 08:00 slot. The rest of the interface, including the budget summary, buttons, and the 'Currently Selected Activity' panel, remains unchanged.

	07:00	08:00	09:00	10:00	11:00	12:00	13:00
Sunday, 2025-06-01		my activity					
Monday, 2025-06-02							
Tuesday, 2025-06-03							

Test 22:

This is to test for clicking on an activity in the calendar and either occurs or it doesn't, meaning validation isn't required. However, to test all the different ways that an activity's details can be found from, this includes testing a click from "No activity selected" → "my activity3", and then also "my activity3" → "my activity4".

The results of these are shown below representing successful tests:

Total Cost: £63.82

Add New Activity

Delete Activity

Currently Selected Activity

Name: my activity3
Time: 7:00
Location: Mostar
Notes: Bring raincoats

my activity

my activity2

my activity3

my activity4

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01	my activity3	my activity4						
Monday, 2025-06-02								
Tuesday, 2025-06-03								

Add New Activity

Delete Activity

Currently Selected Activity

Name: my activity4
Time: 8:00
Location: Sarajevo
Notes: Have uber downloaded

my activity

my activity2

my activity3

my activity4

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01	my activity3	my activity4						
Monday, 2025-06-02								

Test 23:

This checks for the functionality of the “Delete Activity” button, and the toggling of the cursor being able to delete activities. This is tested by enabling the toggle and then clicking on the activity. This is further tested by then turning the toggle off and then on again to verify it is a reproducible process.

(Initial calendar)

Total Cost: £63.82

Add New Activity

Delete Activity

Currently Selected Activity

Name: my activity4
Time: 8:00
Location: Sarajevo
Notes: Have uber downloaded

my activity my activity2 my activity3 my activity4

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01	my activity3	my activity4	my activity	my activity2				
Monday, 2025-06-02								

After first two activities deleted:

Total Cost: £63.82

Add New Activity

Exit Delete Mode

Currently Selected Activity

Name: my activity4
Time: 8:00
Location: Sarajevo
Notes: Have uber downloaded

my activity my activity2 my activity3 my activity4

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01			my activity	my activity2				
Monday, 2025-06-02								

After delete mode toggled off then on again, final two activities deleted:

Total Cost: £63.82

Add New Activity

Exit Delete Mode

Currently Selected Activity

Name: my activity2
Time: 10:00
Location: None
Notes: None

my activity my activity2 my activity3 my activity4

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01								
Monday, 2025-06-02								

Test 24:

This tests that the calendar can save and then be reloaded. Below is the test data of the calendar, with an empty holiday title field along with two activities in the calendar. The empty holiday title field is meant to tell the website to not override the existing holiday title.

Budget Summary
Total Cost: £63.82

Add New Activity

Exit Delete Mode

Currently Selected Activity
Name: my activity2
Time: 10:00
Location: None
Notes: None

my activitymy activity2my activity3my activity4

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01	my activity	my activity2						
Monday, 2025-06-02								

Then returning to the dashboard, the holiday that was just saved is shown under the dropdown as it would've from the previous test, and loading it up gives the following result showing a successful test:

TripMaestro

Sign OutProfile

Select a holiday to view
Select a holiday to view
my holiday (France)

Budget Summary
Total Cost: £63.82

Add New Activity

Delete Activity

Currently Selected Activity
Name: None
Time: None
Location: None
Notes: None

my activitymy activity2my activity3my activity4

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01	my activity	my activity2						

Test 25:

This test checks that the calendar prevents multiple instances of one activity being placed at once and then saved. The calendar to be saved is shown below:

Budget Summary

Total Cost: £63.82

Add New Activity

Delete Activity

Currently Selected Activity

Name: None

Time: None

Location: None

Notes: None

my activity

my activity2

my activity3

my activity

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01	my activity	my activity2	my activity	my activity	my activity	my activity	my activity	
Monday, 2025-06-02								

Saving it and reloading it gave this calendar with only the final instance of “my activity” appearing, showing an unsuccessful test where it was meant to prioritise the first appearance of “my activity”:

Budget Summary

Total Cost: £63.82

Add New Activity

Delete Activity

Currently Selected Activity

Name: None

Time: None

Location: None

Notes: None

my activity

my activity2

my activity3

my activity4

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01		my activity2					my activity	
Monday, 2025-06-02								
Tuesday, 2025-06-03								

In hindsight, it would make more help if the user was never able to drag multiple of the same activity onto the calendar in the first place and this should be validated before the activity is dragged onto the calendar.

Test 26:

This tests the full functionality of the save calendar process which includes a new title and activities:

The dashboard features a text input at the top containing "newest holiday title". Below it is a "Budget Summary" box showing "Total Cost: £63.82". On the left, there are two buttons: "Add New Activity" (green) and "Delete Activity" (red). In the center, a "Currently Selected Activity" panel displays fields for Name, Time, Location, and Notes, all currently set to "None". To the right of this panel are four yellow buttons labeled "my activity", "my activity2", "my activity3", and "my activity4". At the bottom is a calendar grid for the week of June 1st to 4th, 2025. The grid has columns for hours from 07:00 to 14:00. Activities are scheduled as follows: "my activity2" on Sunday (08:00-09:00), "my activity" on Sunday (13:00-14:00), and "my activity3" on Monday (10:00-11:00).

	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00
Sunday, 2025-06-01		my activity2					my activity	
Monday, 2025-06-02				my activity3				
Tuesday, 2025-06-03								
Wednesday, 2025-06-04								

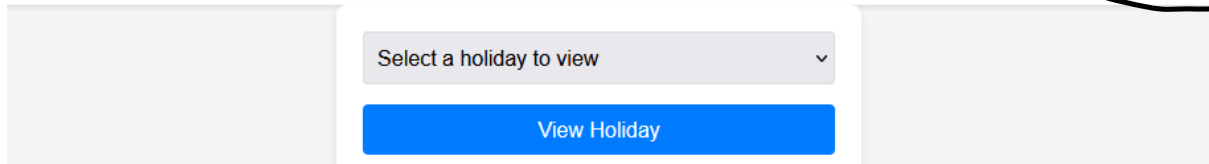
Pressing save showed me the contents of the actual calendar remaining as expected, and checking on the dropdown on the dashboard showed the new title of the holiday also as expected, indicating a successful result:

The screenshot shows a dropdown menu with the text "Select a holiday to view" and a downward arrow. The dropdown is open, showing the same text and an additional option: "newest holiday title (France)". Below the dropdown, the text "Based on your preferences, here's what we found:" is displayed. At the bottom, a table header is visible with columns: "Name", "Description", "Web URL", and "Image".

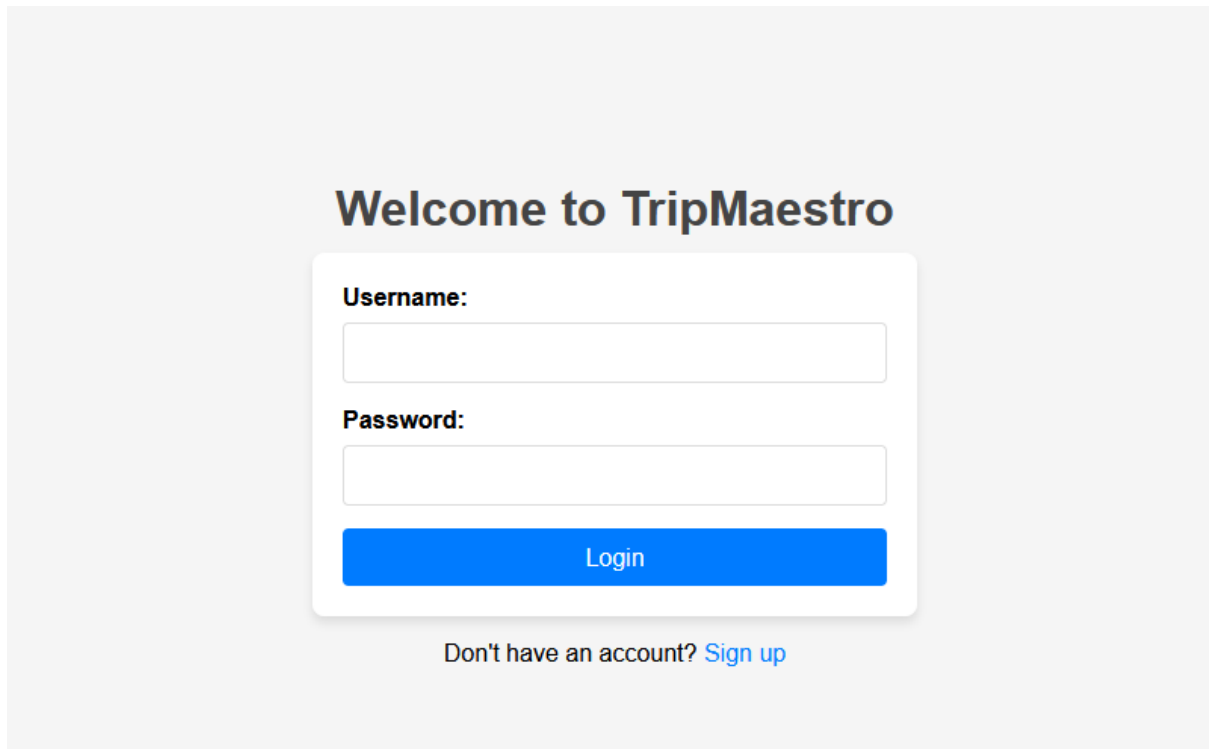
Name	Description	Web URL	Image
------	-------------	---------	-------

Test 27:

This is to test that the “Sign Out” button at the top right of each page clears the session and takes the user to the original page.

TripMaestro[Sign Out](#)A screenshot of the TripMaestro dashboard. It features a light gray background. In the center, there is a white box containing a dropdown menu with the text "Select a holiday to view" and a downward arrow. Below the dropdown is a blue button with the text "View Holiday".

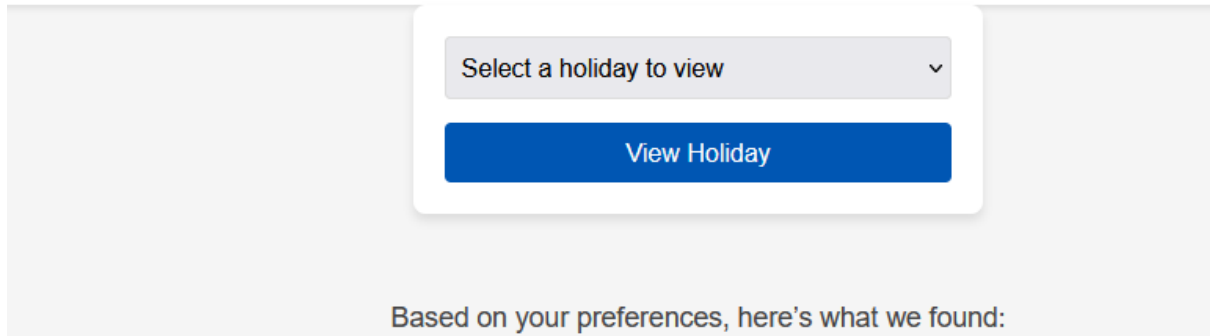
Pressing this button takes the user to the login page, and to check that it actually clears the session, the URL can be changed so that the route is /dashboard. When this is opened, the Python only allows the user to the webpage if the session is connected to a user, and testing this after pressing sign out, I am returned to the login page as expected showing a successful test:

A screenshot of the TripMaestro login page. It has a light gray background. At the top, the text "Welcome to TripMaestro" is displayed in a large, bold, dark font. Below this, there is a white login form with a shadow. The form contains two input fields: "Username:" and "Password:". Below the password field is a blue "Login" button. At the bottom of the form, there is a link that says "Don't have an account? Sign up".

Test 28:

This test is to check the event that the user presses “View Holiday” on the dashboard without selecting a holiday to view. This covers every possible input as it automatically validates itself otherwise as the user can only pick from those existing, saved holidays.

Test input:

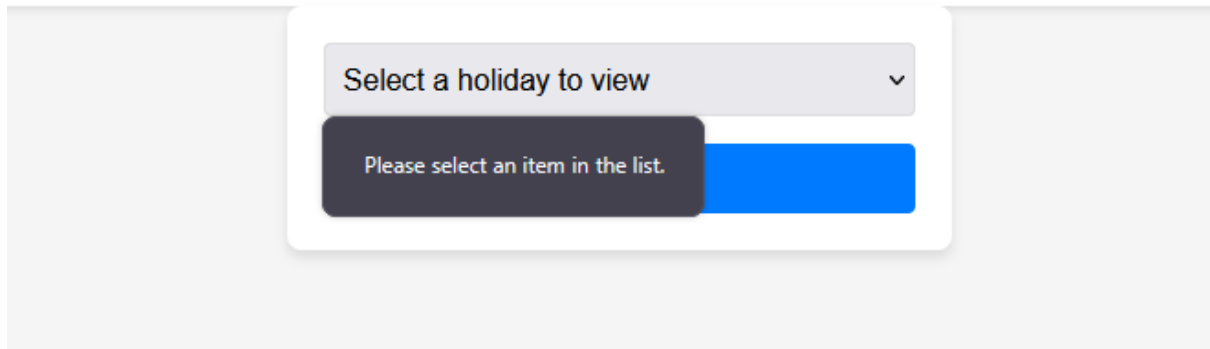


Select a holiday to view ▼

View Holiday

Based on your preferences, here's what we found:

The website declines this as an input, with the browser being made to prompt the user to select a holiday:



Select a holiday to view ▼

Please select an item in the list.

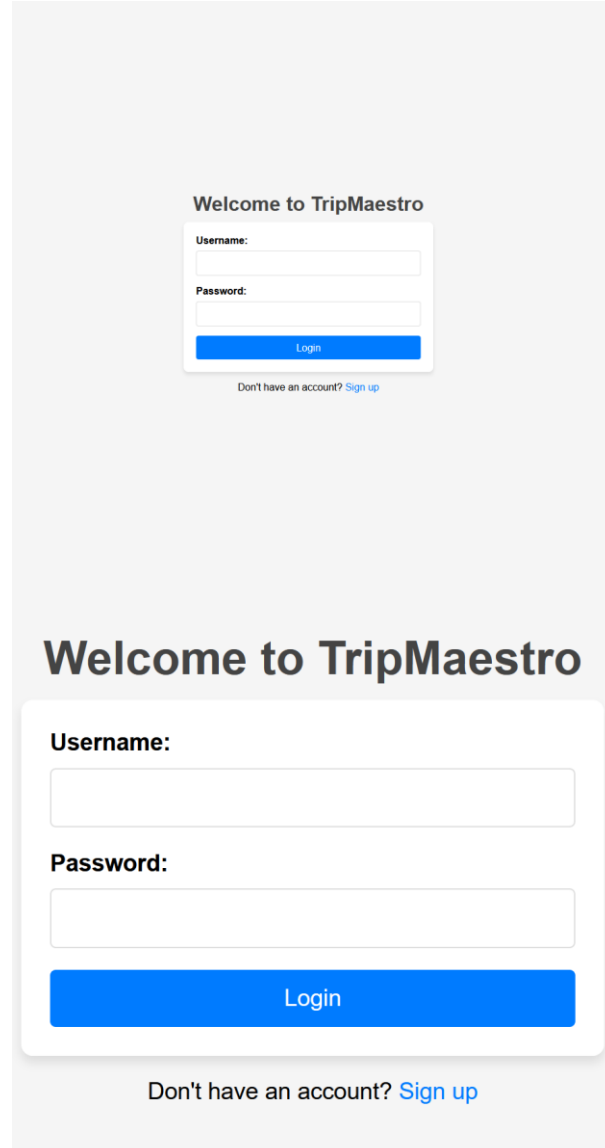
This is the end of the testing for functionality, with good success across the board with minor weaknesses in how it permits multiple instances of the same activity. This causes a misalignment in how it is saved, and would be better to just have a validation on the calendar's contents every time a new activity is put on the calendar, to check that it doesn't already exist.

Usability features and testing

Feature number	Usability feature	Description	Success?
1	Readable text/button size	This would describe a website where the text is/can be large enough for visually impaired users to read. This also can apply to buttons where the size of buttons on the webpage can vary to what the user requires.	Partial success
2	Scroll bars	This allows the vertical space on the website to be traversed quickly where the scroll bars on the side of the website exist on practically every browser.	Success
3	Button hover effects	This is where as the mouse hovers over a button or something the user can interact with, it slightly changes colour to show the user that it is something that can be engaged with.	Success
4	Placeholder values	A text box that is blank will show slightly greyed out text that the user can use to guide their input.	Success
5	Hover descriptions	Where the mouse hovers over a button, the functionality of it is described to the user so they know exactly what will happen after clicking on something.	Failure
6	Coloured hyperlinks	Text that links to something else is made a different colour to show it can be interacted with and will take the user to a different page.	Success
7	Snap to grid alignment	How the user isn't expected to exactly align an activity with the calendar box, but instead can just drag the cursor to somewhere in the box and it occupies the whole space.	Success
8	Auto formatting	This is, where possible, the user is forced to use a framework to make it suitable to the required data type of the input.	Success

Feature 1: Readable text/appropriate button size

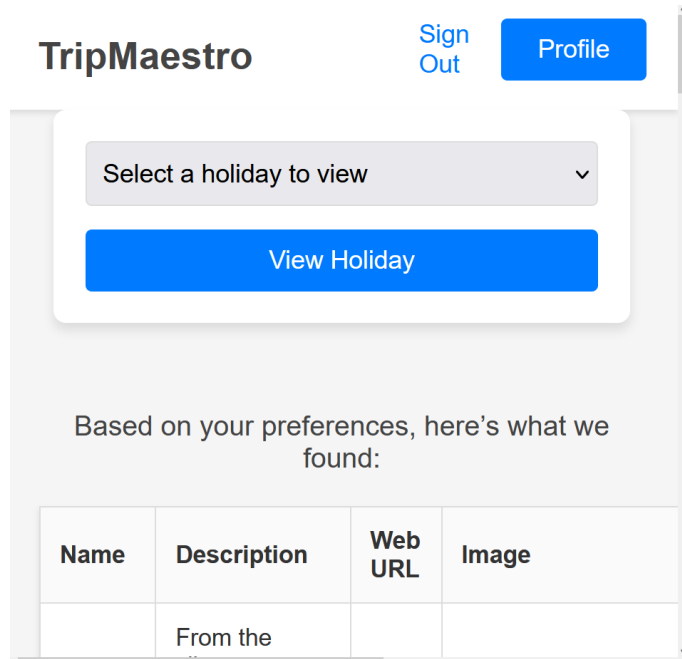
This is achieved largely through the nature of TripMaestro and how it is a website. This enables the user to use Ctrl + ScrollUp/Down to zoom into/out of text and buttons. This is shown on the login page below, with first showing the default, then a zoomed in version.



These aren't the exact same scale as is shown on my webpage, but has been scaled down equally to fit in the document so they both represent my entire screen at once. It is clear that the second image has larger text and buttons, but can look less aesthetically pleasing.

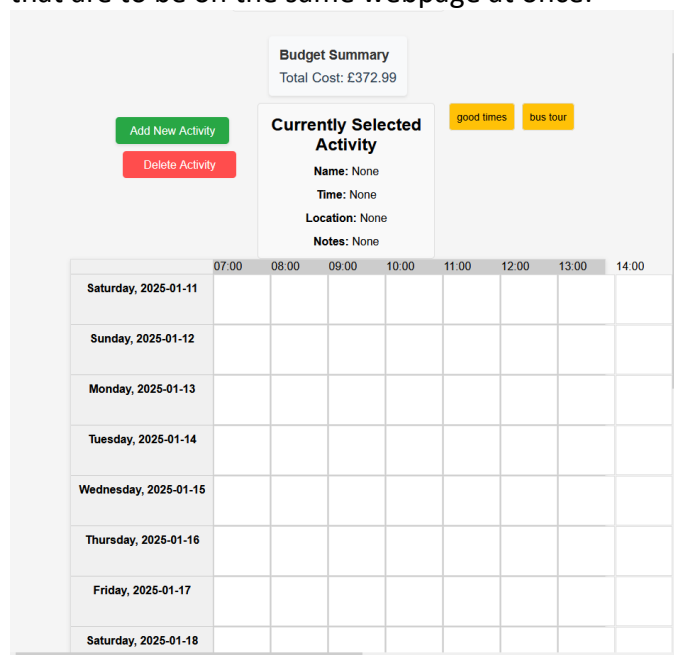
Feature 2: Scroll bars

This is a feature that would be especially important on a page with a lot of vertical space. This can go in tandem with the first usability feature, as when zoomed in, it would take longer to get to the end of the page. This is shown well on the dashboard, where the same 220% zoom from the last screenshot results in this being the displayed page:



The length of the vertical scroll bar (on the right side) shows that it is very useful here where it allows much more vertical space to be covered as opposed to having to use the arrow keys.

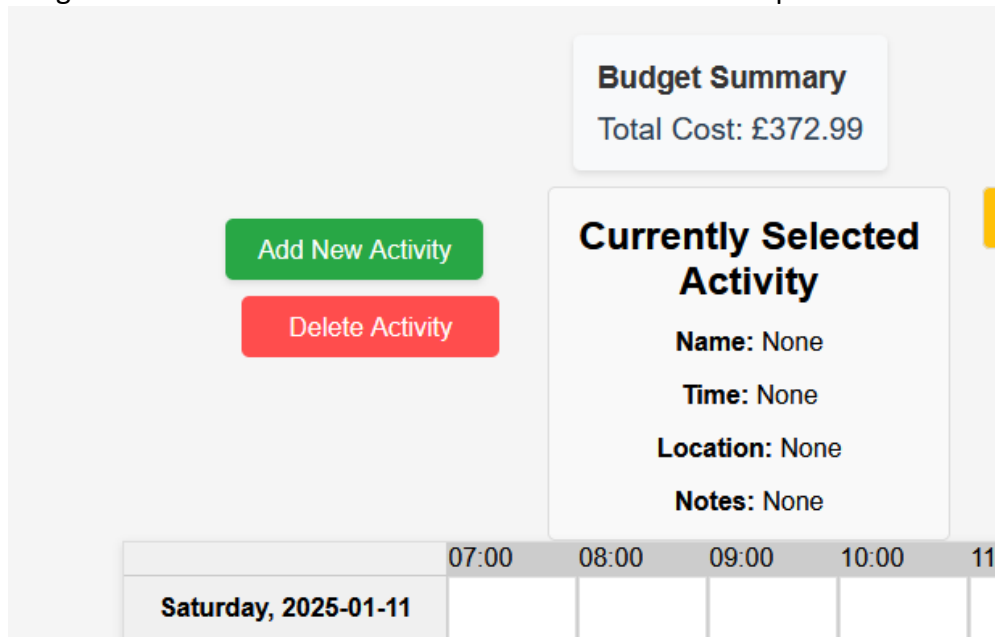
This also can be useful outside of zoom when viewing the calendar and there are lots of days that are to be on the same webpage at once:



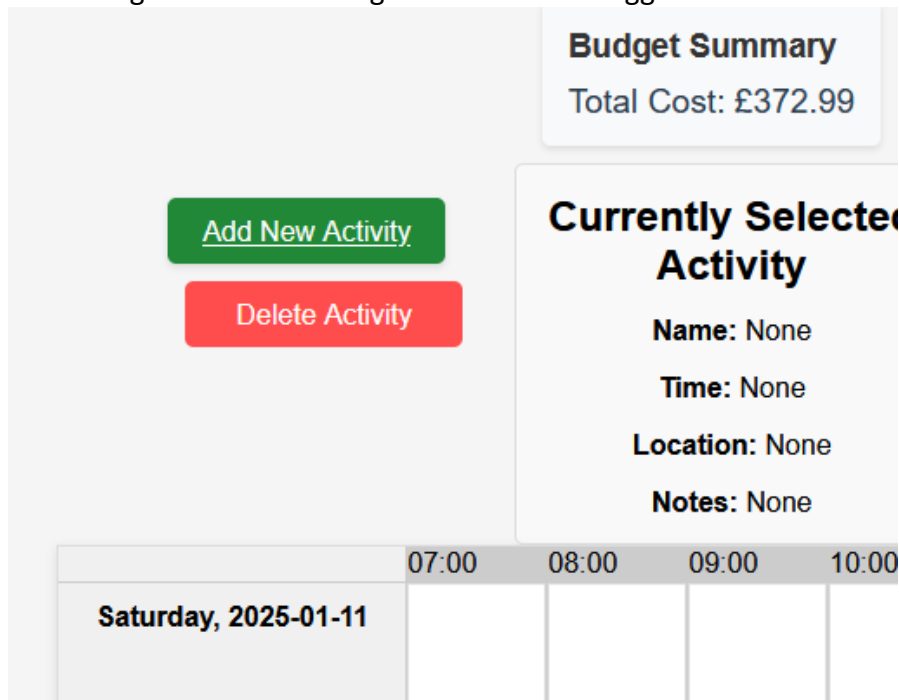
This represents a successful implementation of scroll bars, but partly not due to choice, and is rather an implied feature of using a website for TripMaestro.

Feature 3: Button Hover Effects

This feature is implemented across the website, such as on this calendar page. The below image shows the colours of the buttons without the cursor placed on one of the buttons:



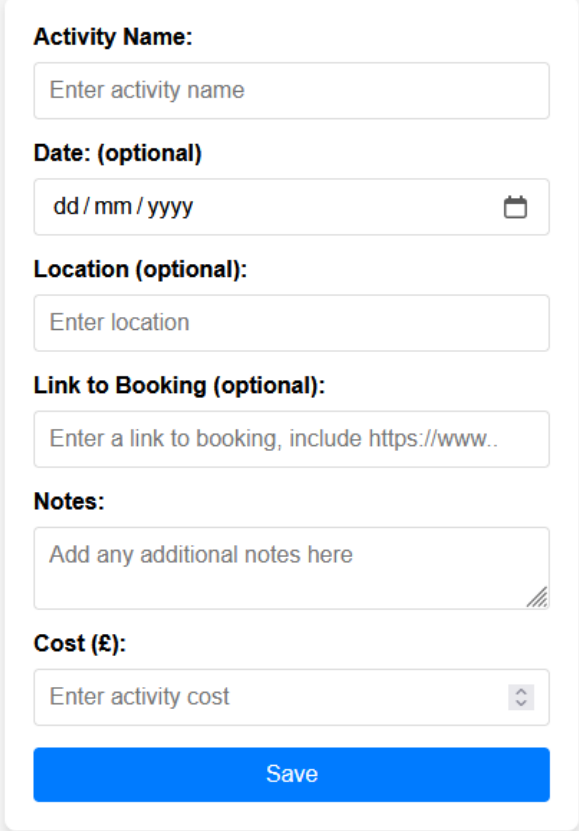
This changes to the following if the cursor is dragged over the “Add New Activity” box:



This is implemented across the site right from the login page, and guides the user through the interface, which is key for the stakeholder given they wouldn't want the website to be limited to those are skilful with technology.

Feature 4: Placeholder values

This feature enables the user to get an idea about what each text box is meant to include when filling in a new activity form for example. This is shown below in that form:



The screenshot shows a web form for adding a new holiday activity. At the top, there is a blue link that says "Go back to holiday list". The form itself is a white card with a light gray border. It contains several labeled input fields: "Activity Name:" with a placeholder "Enter activity name"; "Date: (optional)" with a placeholder "dd / mm / yyyy" and a calendar icon; "Location (optional):" with a placeholder "Enter location"; "Link to Booking (optional):" with a placeholder "Enter a link to booking, include https://www.."; "Notes:" with a placeholder "Add any additional notes here" and a diagonal line icon; and "Cost (£):" with a placeholder "Enter activity cost" and a small up/down arrow icon. At the bottom of the form is a blue "Save" button.

The inclusion of translucent text such as “Enter activity name” and “Enter a link to booking, include https://www..” act as prompts for the user to help them use the website. This benefits the stakeholder where ease of use is important to help keep most of the people on the original blog also able to use TripMaestro, and not restricting it to knowledge of technology.

This is also implemented across the website, including in the login, registration, new holiday forms and others, helping the website to be consistent and predictable so the user isn’t intimidated if they find a new feature that they want to use.

Feature 5: Hover descriptions

This feature would have it so that when the user drags their cursor over a button or an activity, it would display a little bit of text to the user about how and when to use it. This would help the ease of use but hasn’t been implemented in TripMaestro due to users generally not checking for this anyways, instead just clicking through until an end result, which makes the translucent placeholders good for textboxes, but obviously the same can’t be used when guiding the user through buttons.

Feature 6: Coloured Hyperlinks

The differentiation between the blue and regular black text helps the user to know what they can progress to help build their holiday. This is implemented by default with HTML, and its use is clear to see on the dashboard:

TripMaestro

Sign Out Profile

Select a holiday to view

View Holiday

Based on your preferences, here's what we found:

Name	Description	Web URL	Image
Italy	From the silvery crests of the Alps to the sparkling Mediterranean Sea, Italy's beauty is unparalleled. A visit could mean marveling at the Colosseum after a tender plate of pasta, or drifting lazily down a canal, your belly full of local Sangiovese. Explore Tuscan vineyards and olive groves or drool over boutiques in fashionable Milan. Look for mermaids from the cliffs of Sorrento or nibble on Parma's famous cheese and prosciutto. Anywhere you choose, the country will captivate you.	View	
United States	Spanning 2,800 miles (4,506 kilometers) from east to west, the US is a vast patchwork of landscapes and metropolises. From Arizona's red-rocked canyons and California's coastal beaches to New York's big city and Virginia's storied past, it covers any topic any traveler could want or need.	View	
	With 26 UNESCO-declared world heritage sites, charming colonial towns and dozens of thrilling cities, there's plenty to explore in this country of 400		

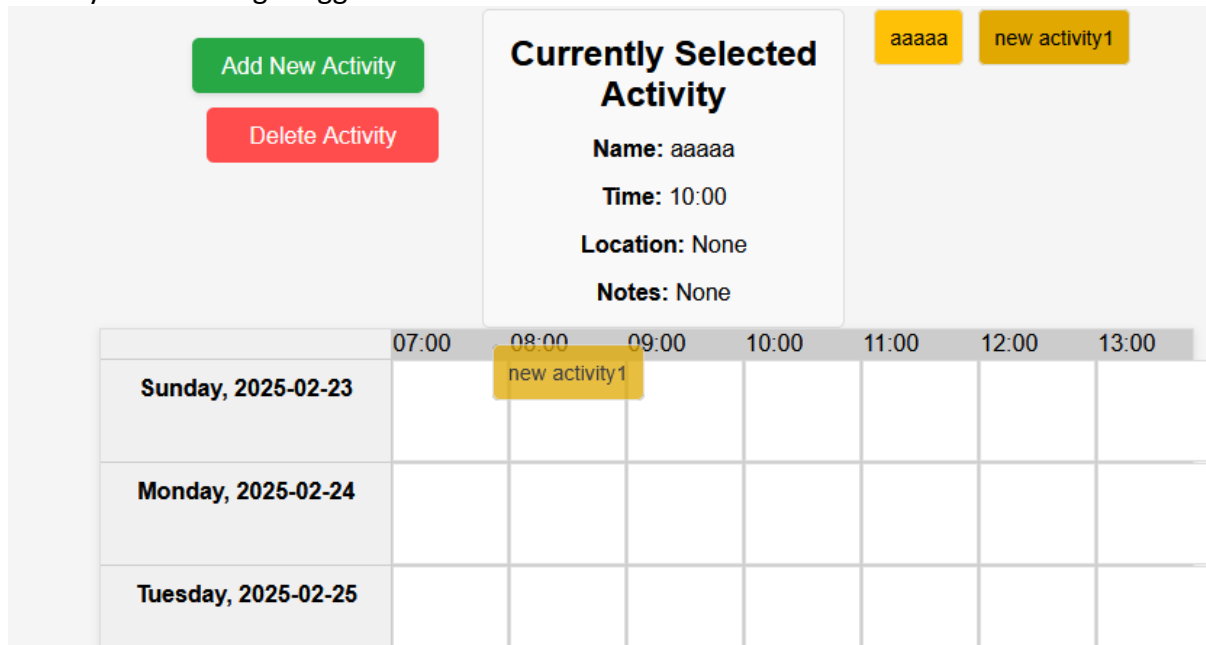
Plan a holiday here

Plan a holiday here

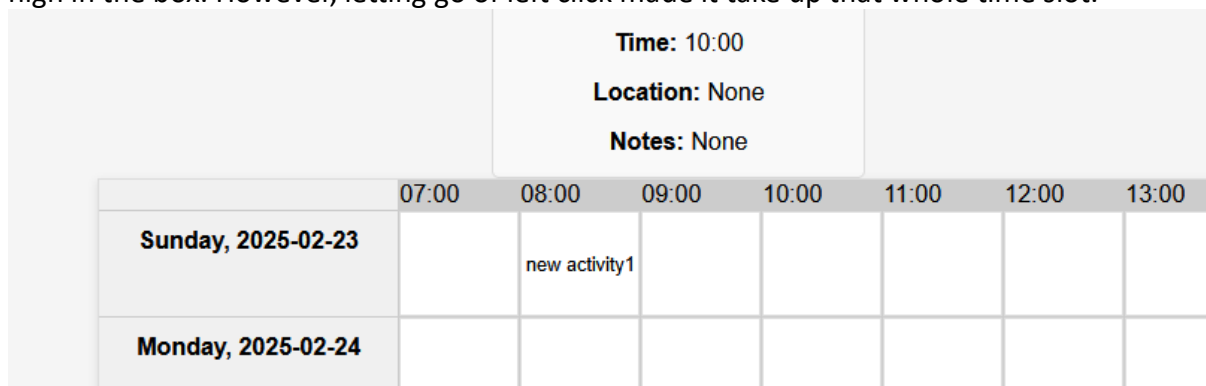
I've circled the 3 blue hyperlinks on the webpage here, and they very clearly differentiate itself from the description text and the country name. This enables the user to know that the words in black text are for reading only, but there is more to discover/use by using the blue links.

Feature 7: Snap to grid alignment

This is only implemented in TripMaestro in the calendar, where dragging an activity onto the calendar doesn't require the activity to be perfectly aligned, but holding the cursor anywhere in the box and then letting go of left click makes the entire box be taken up by that activity. This is shown in the screenshot below which was taken as the activity "new activity1" was being dragged.



The cursor was roughly in the middle of that yellow box around 08:00, showing it was very high in the box. However, letting go of left click made it take up that whole time slot.

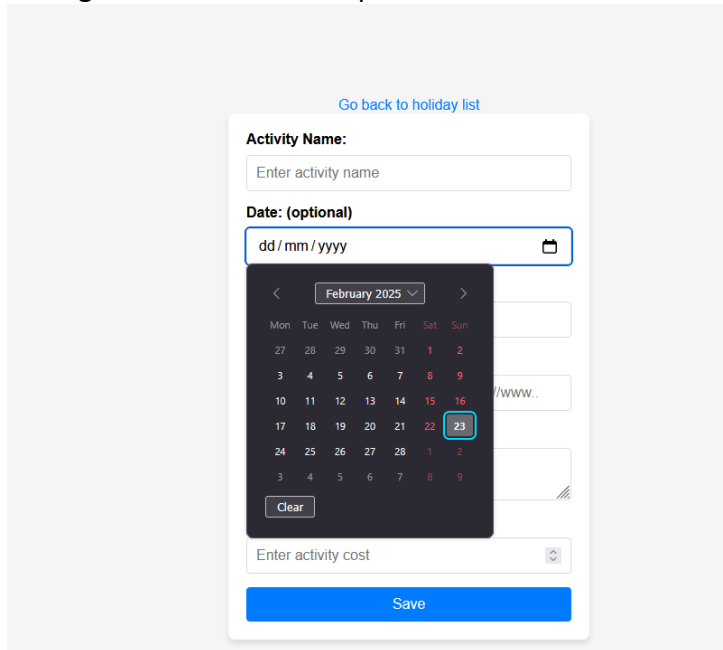


This means that it is easier to use where the user requires less accuracy with their cursor. This especially can help if the user is more zoomed out, and also keeps the contents of the calendar aligned making more a visually satisfying interface.

Feature 8: Auto formatting

This mainly applies for two types of data that have been used in TripMaestro, dates, and prices.

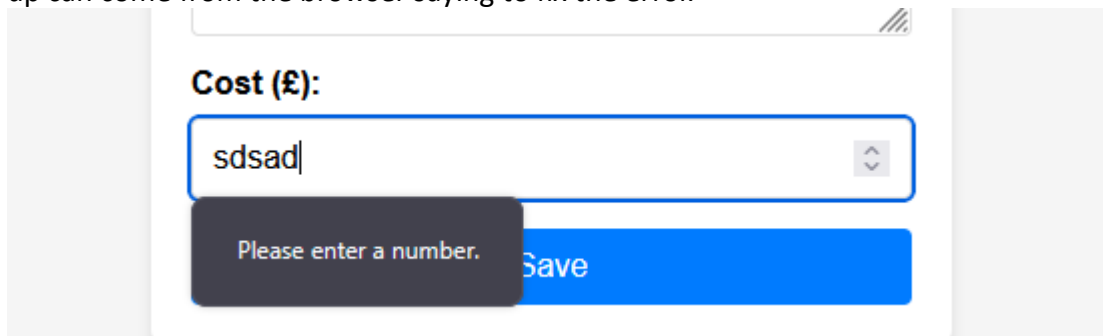
For dates, the user is helped through a pop-up allowing it to select a specific date. This avoids the user typing "22.01.25" which may not be in the required format for the website. This way prevents user error as the browser and its pop-up essentially guides the user through the date selection process.



The screenshot shows a form titled "Go back to holiday list" with the following fields:

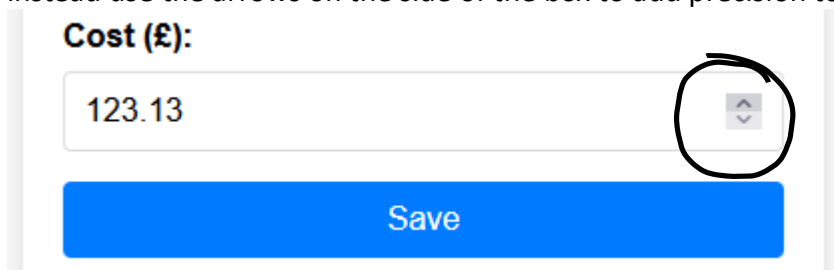
- Activity Name:** A text input field with the placeholder "Enter activity name".
- Date: (optional)** A date input field with the placeholder "dd/mm/yyyy" and a calendar icon. A date picker is open, showing February 2025. The date 23 is selected.
- Enter activity cost:** A text input field with a dropdown arrow.
- Save:** A blue button.

For prices, if the data type isn't correct, the page isn't required to refresh, and a small pop-up can come from the browser saying to fix the error.



The screenshot shows the "Cost (£):" field with the text "sdsad" entered. A blue border highlights the input field. Below the field, a dark grey error message box says "Please enter a number." A blue "Save" button is visible to the right of the error message.

Moreover, the user is also prevented from including too many decimal points, and can instead use the arrows on the side of the box to add precision to the inputted value.



The screenshot shows the "Cost (£):" field with the value "123.13" entered. A blue border highlights the input field. To the right of the field, a small grey button with up and down arrows is circled in black. Below the field, a blue "Save" button is visible.

Solution Evaluation

Re-iterating the success criteria: “

1. The website has drop down menus that points the user to a list of destinations, and clicking on one of those destinations takes you to another external website to find out more. An external website is used as they will be able to provide specialist information from people who have visited that destination and given reviews, unlike the main website where, when it is launched, no reviews would be on there.
2. The main website can take inputs that indicate preferences about holidays, with a set of buttons ranging from “never” to “very likely” about their tendencies to get involved with different types of activities such as hiking, exploring historical landmarks and visiting scenic areas. This can help provide a more complete user experience as the website can tailor what is displayed based on their previous responses, helping them to find the destination they want to go.
3. When the user first opens the website, there is a login prompt for text boxes for a username and password. There is also another option below the textboxes to register, and these are saved in a database for later use. This enables holiday plans to be stored for later use and modification, so the user is in no pressure to finish planning the holiday in one go.
4. When the user decides on their destination, they are prompted with different categories of activities to select from. When opening these categories, the user is given a prompt to select how much time should be allocated for that. This time is then represented on a block on a calendar for the user to drag for when they want it. The calendar is meant to save those activities, and display the details of those activities when prompted.
5. The website has a textbox below each day that has been planned for the user to add their own specific notes about things that are not included in the original set of activities. For example, a large group may use this to indicate who is participating in certain activities and who is staying back in the accommodation.
6. When the user clicks on an activity it gives the user a form to fill in regarding that activity. This asks for various information that can be useful later, such as a time duration, when it starts/finishes, whether food is available there, what transport will be used, as well as contact information for that place if it is necessary. This is helpful as it can make the user feel that the website is more useful than just a list of activities written on paper, because it can provide information that the user may not immediately need but might want to know later in the trip.

“

Success criteria number	Test number	What was tested	Success?	Page number for test evidence
1	7	List of destinations displayed to the user, not as a dropdown but as a page scroll	Mostly a success	130
1	9	Ability to scroll through pages of holiday destinations	Success	132
1	10	External website linked so that the user can click on it to find more information about the destination and reviews	Success	133
2	1	Ability to make a new account with ratings for each of the characteristics described	Success	126
2	7	The list of destinations given to the user matched the ratings that were defined at the account's creation, where destinations listed had a priority for a unique cuisine	Success	130
2	8	The list of destinations given to the user matched the ratings that were defined at the account's creation, where destinations listed had a priority for exploration	Success	131
3	5	Using valid login credentials lets you into your account	Success	128
3	6	Using invalid login credentials doesn't let you into your account	Success	129
3	1	Registering an account is possible	Success	126
3	27	Signing out to switch account is possible	Success	150
3	26	An existing holiday is loadable from the dashboard using the dropdown menu	Success	149
4	-	"Different categories to choose from" This wasn't achieved, all activities follow the same format	Failure	-
4	25	Length of an activity doesn't need to be described as it runs until next activity. However, users may try and cover 3 hours of one activity by placing one activity into 3 time slots which wouldn't save	Partial failure	148
4	21	Activity draggable onto calendar	Success	144
4	24	Contents of calendar save and can be opened at any time	Success	147
4	22	Clicking on an activity in the calendar shows its details	Success	145
5	20	Filling in the notes box in the new activity page displays itself when clicked on in the calendar	Success	143

6	18	New activity page requires detail to be used later	Success	141
6	19	New activity page can work with just an activity name to then be added to the calendar	Success	142
6	20	New activity page can hold more details that may be needed by a user in varying amounts during a holiday	Success	143

The solution has met the majority of the success criteria but doesn't entirely meet it in regard to specifically the calendar functionality. The usage of the calendar isn't made entirely clear to the user, and the user isn't really guided with how to keep each activity to only have one occurrence in the calendar.

- This could either be solved by making each activity only be able to exist once in the calendar, or allow it to exist multiple times and then make the budget and other features only depend on what is in the calendar.
- The dropdown of destinations isn't entirely met, however I don't think this initial requirement was visually appealing as it would almost tuck too much information away from the user as you want them to want to engage with the website and the ability to recommend destinations is such an integral part.
- There is also room to improve in regard to the nature of the activities added to the calendar, where the user isn't able to choose the type of activity they want. This would also help another success criteria where the user is able to add more detail such as modes of transport for the activity. The more details that the stakeholder wishes to be added here, the more of a need there is to distinguish between different types of activities. This is because it would become too cluttered for a seamless user experience, as if someone is trying to make a simple restaurant activity, they wouldn't want to be asked to fill in a prompt about what protective equipment they would want.
- The holiday planner is also limited in how it only provides one format for the user to view the holiday and its details, on the website, which doesn't make it that superior to a paper plan. On the other hand, the versatility of the internet can mean that in theory the calendar in TripMaestro could be exported to a PDF list, or imported into Google Calendar as a .ical file. The stakeholder expressed a genuine desire for TripMaestro to replace paper plans of holidays, and this would be the step that would begin that takeover, as right now TripMaestro holidays only being accessible on the website doesn't make it that appealing.

Usability feature 1: Readable text/appropriate button size

This wasn't a success particular to TripMaestro, rather just a side effect of being a website. This is with how websites can be zoomed into using Ctrl + ScrollUp/Down as opposed to any specific feature being implemented into TripMaestro. This also can apply for the mobile experience, but this wasn't a large priority for the stakeholder, and this group of users is let down in other aspects of using TripMaestro.

However, this all is still an advantage over a traditional application experience where the scaling of it is rarely modifiable unlike a website which can scale in size, and also use browser extensions for other desirable features.

Usability feature 2: Scroll bars

Again this is a usability feature that came about due to the nature of TripMaestro and how essentially all web browsers allow for scrolling down the contents of a page. This also doesn't encompass every part of the user's experience, with the notes for each activity not having a scroll feature during the activity's creation. This can be useful if the user wants to divide the notes they are adding to the activity into different lines to separate their text into parts, causing a lot of vertical space. I do think this is only a minor inconvenience though as the text box does scale in size based on how much text is added to it, a scroll bar would only be necessary if the text box took up the whole page, which isn't really feasible if the user is just describing a footnote of one activity in one holiday.

Usability feature 3: Button Hover Effects

Button hover effects enable the user to recognise when there is functionality behind a button and it isn't just one static component of the web page. This is achieved with the slight change of colour when the user's cursor goes over a button, and also can be useful in recognising when the website is responding to the user's inputs and that the browser hasn't crashed. It also helps to the visual appeal by making the user more immersed in the holiday planning process where they aren't clicking on images (which the buttons otherwise would look like) but actual buttons that clearly initiate processes. The implementation of this also improves accessibility where new users to the site who are unfamiliar with how to navigate can interact with the buttons and quickly learn their way around as they become learn the button hover effects to look out for when trying to do something. They have been implemented with standardised colours across the website making it effective for desktop users, but does let down users of mobile devices who don't have a cursor so are less likely to pick up on which parts of the websites are buttons.

Usability feature 4: Placeholder values

The presence of lighter coloured text that exists before a user enters into a text box can help to guide the user to understand what should be entered into the text box. This is especially important for pages such as the new activity form, where it isn't immediately obvious in how to break down their activity's details into components, but the placeholder text guides the user through to tell them that the location has its own box and additional notes has a

separate box. It also is essential when trying to help the user keep data they are entering in the right format, such as the booking link. To make the booking link be a clickable hyperlink on the main calendar page, the user needs to add “https://www.” to the start of the link, and this is told to the user through the placeholder text. The implementation of this isn’t standard across the site though, with some text boxes using the imperative “Enter” whereas other boxes saying “Add”. These two imply varying degrees of necessity, whereas in the actual website the choice between these words don’t correlate with whether a specific box in a form is required for the user to progress. This limits the effectiveness of this usability feature where the user better understands what to include due to the existence of the placeholder, but can’t always know whether something is mandatory to fill in. To add to this, lots of users don’t actually read the placeholders before filling in the text boxes, rendering the usability feature useless in that situation, although the colour of the placeholder text arguably is dark enough to be noticed at the minimum by most users.

Usability feature 5: Hover descriptions

This feature hasn’t been implemented in TripMaestro so has no effectiveness at all, but would have been good to help guide users through the functionality of each button on the page. Most buttons are self-describing on the page such as “Add New Activity” but aren’t clear on what exactly they lead to. For example, it isn’t immediately made clear that “Add New Activity” will create a draggable box representing that activity that the user can place on the calendar. This kind of ambiguity can make the user not engaged with the website as they feel that they are first discovering what things do instead of actually designing their holiday when they first use the website. This lack of retention can make be a disadvantage from the perspective of the stakeholder where the user doesn’t feel the need to stay on TripMaestro when they could go to a competing site. On the other hand, it can be argued that very few users actually take the time to hover their cursor over a button, instead preferring to click on it and simply seeing what happens to learn for next time. This can make the omission of hover descriptions not as important as it may first seem, although may still pose some small disadvantage to users who rely on it given their experience on other sites.

Usability feature 6: Coloured Hyperlinks

Coloured hyperlinks offer similar usability benefits to button hover effects, where the user can clearly see that clicking on the text will lead them to another page. It arguably is more effective given that the user’s cursor doesn’t need to be over the link for it to be displayed differently to the user. To add to this, blue hyperlinks by default have button hover effects so those benefits still apply. Mobile users also can benefit from this where it doesn’t rely on a cursor for hyperlinks to be distinguished from regular text. It is effective at guiding the user through the page, but can be a problem where the stakeholder desired a visually appealing interface, and using the standard blue coloured hyperlinks without an actual button for the user to interact with can be said to look old-fashioned.

Usability feature 7: Snap to grid alignment

This is a usability feature is very specific to the calendar functionality of the website, where it helps the user place the activity into their calendar, not requiring an exact placement of the box into the calendar grid. This keeps the user's activities in a uniform placement in the calendar, and not having a vertical misalignment between days, meaning the user can clearly separate the activities of different days. The same benefit also applies with horizontal misalignment where users may not want an activity to lie between multiple time slots. However, this can be a disadvantage where the user might want the activity slot to cover multiple slots at once and would feel that this is more accurately represented if the title of the activity lied between two time slots. In the grand scheme, it would just be better to remedy this by allowing the user to choose the length of each activity and for the horizontal length of the activity in the calendar to be dependent on that. Overall, the implementation of snap to grid alignment is effective based on the current state of the website, but could be more useful if the grid could be setup to be aligned based on the timings that the user wanted.

Usability feature 8: Auto formatting

Auto formatting is implemented when entering in dates throughout the website, where the user is given a menu to click through to find the exact date they are looking for. This ensures that the backend is always able to understand the date the user is trying to input given the menu keeps it in a comprehensible format. It avoids the situation where the user has to take time to understand the need for DD/MM/YYYY as other people may use a different symbol instead of "/". It is also implemented when entering in the prices of the activities, where the input box forces the user to enter a number with an increment of 0.01, preventing the user from entering in prices smaller than 1p which wouldn't make sense as a price.

Both of these scenarios improves the usability of the website, keeping it accessible to people who otherwise would not know to use the date format, or those who may use commas instead of decimal points as is seen in Europe. However, the effectiveness of this is limited where the prices are limited to only being in GBP (£); there are other currencies which have different denominations or may not have any at all, making the validation useless where there is no point showing the user 0.1 of a currency where the currency only exists in whole numbers. This is especially important for TripMaestro where the blog of the stakeholder's site is intended for an international audience, with international destinations, although this can be fixed without too much difficulty where the user is given either a dropdown or a text box containing the ISO 4217 currency code to then have corresponding changes across the page.

Improvements to existing usability features have already been outlined, but those that have been omitted entirely include:

1. Colour accessibility

This mainly refers to colour blindness where people are unable to see the difference between certain colours, and can pose a problem on the main calendar page. For example, the “Add New Activity” and “Delete Activity” buttons are green and red respectively, and would be difficult to distinguish for people with deuteranopia. This could be remedied with an accessibility page in which users are able to identify any colour blindness they have, and changing the colours so that those that the user may traditionally be unable to see are changed to move into their perceivable colour range and are distinguishable again. For features like this however, it is important to maintain a minimal amount of data collection where the user’s privacy should be respected. It can be made that so the user has to expressly say whether they are happy with storing this data every time they log in, or if the data should not be stored and the user should re-enter their colour blindness each time they create a new session.

2. Text-to-speech

For the visually impaired, text-to-speech enables users to know what is on screen without actually being able to see it. This functionality could be included in the accessibility options when they first create the account, where a sound is displayed to the user to indicate that the option is available. This would work similarly to how Cortana starts speaking during Windows setup to keep the setup accessible to users who need that. However, the implementation of this isn’t too important due to TripMaestro being a website and not an application. This means that visually impaired users likely already have a browser extension that reads the text on the screen, so there would be no need to do the same functionality twice. It is still important to consider how the screen reader reads the page. For example, on the calendar page, the user would want to hear “Add New Activity button in green”, but without preparing for this, it may hear “HTML button Add New Activity button” which is much less easily comprehensible.

Solution Maintenance:

- One maintenance issue arises with the API key and how it is authorised. For example, every time the API key is refreshed, it will need to be changed else the dashboard will be unusable. To add to this, with the developer build, each time there is a new IP address that is trying to use the API, it has to be added to the whitelist to avoid accidental API usage. However, this can be remedied in the build that is launched to users, where the domain name can be whitelisted instead.
- The actual reliance on the API also may need maintenance in the long term where the image format may change to something like a .webp and would need further testing and modification to see if new formats would still work. This can possibly be remedied to be made “future-proof” where an “iframe” is generated that takes an extract of the images as they appear on the website. However, it can be argued that a greater risk is that the API is taken entirely offline, for maintenance or just isn’t profitable for TripAdvisor anymore. In this scenario, time would have to be spent to implement a new API extracting its relevant information, which may also lose some functionality that may have only existed through the TripAdvisor API.
- Another maintenance issue can arise in how the database is managed, where the different ratings for the destinations may change over time due to changing circumstances in the country. This may only require minor changes in the short term where a country may only go up or down a few spots in the rankings within a few years so isn’t a great issue in terms of keeping the website operational. If in the future the website is synchronised with an external database managed by another organisation, the website has to be ready for their database to go offline and have some failsafe.
- Security is also something that needs to be maintained, with login details being encrypted with the latest protocols. However, as it stands there is no effort put on security with hashing not being implemented yet along with encryption, but this forecasts that being implemented before roll out to the general public, as otherwise the reputation of TripMaestro can be damaged in the short term, and consequently have a damaging impact on future users’ willingness to stay using the platform. This also may harm users’ view on the stakeholder’s original blog where they may build a stigma of distrust to the owner.
- Different platforms need to have maintenance unique to them as well, mobile users already have been neglected compared to desktop users in the development of the program so that’s not relevant to maintenance, but different web browsers also need to be maintained uniquely. For example, Firefox has native uBlock Origin which blocks ads and pop-ups, which sometimes can hinder the functionality of the webpage. Currently, nothing on TripMaestro is caught by its blocker, but in the future it may falsely flag something as not desired by the user when it is a feature on the website. This means that parts of the website may need to be labelled differently in the future, with testing done over the course of the website’s existence to check this.

Solution Limitations:

- The database can have another issue where the current implementation refers to a file held locally called users.db. When it is deployed to a larger audience, this will need to be modified so that it can refer to a centralised server where all user data is stored, and kept private from the user's browser to ensure security. This helps to eliminate multiple local databases trying to be synchronised, and also the problems that may arise when multiple database transactions are trying to happen at once. It helps to maintain atomicity in transactions where they either are performed entirely or not at all, with the single, unified database that can allow record-locking to occur.
- This poses another problem where going over 5,000 API calls in a month incurs a charge to TripAdvisor, and there is no current way to limit the number of calls per user. As a result, a user that makes many accounts and scrolls through many pages of destinations could cause limitless charge being incurred by the owner of the API key. To add to this, the charge itself is an issue that the stakeholder has to be ready to take on where the website heavily relies on the API to be able to give suggestions to users. Whilst there is no immediate way to circumvent the charge, if an alternative API is found with similar functionality at a lower price, TripMaestro can be less of a financial burden to the owner of the original blog. This is beneficial long-term for the commercial success of TripMaestro where revenue could theoretically be generated through affiliate links for hotel bookings, or through payments to promote certain services such as car rentals.
- Security is a major limitation in this website, with major shortcomings in how hashing hasn't been used to store user data. This results in people who are able to take data from the database being able to access plaintext usernames and passwords which can be used on this site and on others due to people generally reusing the same login details on multiple sites. Furthermore, with how 'POST' and 'GET' methods have been used throughout the project, there are details that can often be modified in the URL that allow users to get unauthorised access into holidays that aren't theirs. To fix this, verification would need to be implemented before loading each page to check that only the planner of that holiday is able to see and modify that holiday.
- Mobile support is also a real shortcoming for the website, with no testing done at all to check for functionality with just touch inputs. It would be necessary to test that the items on the page scale appropriately for vertical-based aspect ratios such as 16:9 to 22.5:9. This compares to computer monitors which are horizontal-based, normally around 16:9 to 3:2 (13.5:9) on some laptop screens. This may actually be beneficial when using the calendar as any additional dates in the calendar add another row to the table, that is only displayed with more vertical space. On the other hand, the dashboard and its recommendations to the user would be very annoying to use due to the description of each destination taking up horizontal space in the table. As the browser will still try to fit it on the page, the text will appear squished, and the image will look very out of place due to it being stacked directly below the previous text. A short-term fix would be to zoom out, but this would result in the text on the page being impossibly small to read, as well as the buttons being too small to accurately press. An entire separate GUI layout would need to be designed, rearranging the contents of the table to fit a more vertical format when a mobile browser is detected.

Code broken down into each file:

app.py

```
from flask import *
import sqlite3
import requests
from datetime import *
import json

app = Flask(__name__)
app.secret_key = 'TemporaryFlaskKey123'

class TripAdvisorAPI: #defines class
    def __init__(self): #initializes class
        self.api_key = "0F0E9E89119248B2AFEF0832BFE031E7" #API key attribute
        self.base_url = "https://api.content.tripadvisor.com/api/v1/location" #Base URL for API
        self.headers = {"accept": "application/json"} #Common headers used for API calls

    def search_destinations(self, country_name): #Method to search for destinations
        search_url =
f"{self.base_url}/search?key={self.api_key}&searchQuery={country_name}&category=geos&language=en" #URL
for search endpoint
        response = requests.get(search_url, headers=self.headers) # Sends request to API
        if response.status_code == 200: #If the request is successful
            data = response.json() #Converts response to JSON
            first_item = data['data'][0] #Looks into the data variable, for the data section of
response, for first item
            location_id = first_item.get('location_id') #Extract the location ID
            name = first_item.get('name') #Extract the name
            return location_id, name #Returns location ID and name
        else: #If request is unsuccessful
            print(f"Error fetching destinations: {response.status_code}") #Prints error message
            return None, None #Returns nothing for both location ID and name

    def get_location_details(self, location_id): #Method to get location details
        details_url = f"{self.base_url}/{location_id}/details?key={self.api_key}&language=en" #URL for
details endpoint
        response = requests.get(details_url, headers=self.headers) #Sends request to API
        if response.status_code == 200:
            data = response.json()
            description = data.get('description', 'No description available') #Extracts description
from data
            web_url = data.get('web_url', 'No web URL available') #Extracts web URL from data
            return description, web_url
        else:
            print(f"Error fetching location details: {response.status_code}")
            return None, None
```

```
def get_location_image(self, location_id): #Method to get location image
    images_url = f"{self.base_url}/{location_id}/photos?key={self.api_key}&language=en"
    response = requests.get(images_url, headers=self.headers)
    if response.status_code == 200:
        data = response.json()
        images = data.get('data', []) #Extracts images from data
        first_image = images[0] #Extracts first image from images
        medium_image = first_image.get('images', {}).get('medium', {}) #Extracts all types of the
first image
        # then the middle image from there
        image_url = medium_image.get('url', 'No image URL available') #Extracts image URL from
medium image
        return image_url
    else:
        print(f"Error fetching location images: {response.status_code}")
        return []

def fetch_and_display_details(self, country_name): #Method to fetch and display details
    location_id, name = self.search_destinations(country_name) #Searches for destinations
    description, web_url = self.get_location_details(location_id) #Gets location details
    image_url = self.get_location_image(location_id) #Gets location image
    return name, description, web_url, image_url #Returns name, description, web URL, and image
URL

def determinePreferences():
    userConnection = sqlite3.connect('users.db') #Same old database connection
    try:
        userCursor = userConnection.cursor()
        userCursor.execute("SELECT explorativeRating, isolatedRating, uniqueCuisineRating FROM users
WHERE id = ?",
                            (session['user_id'],)) #Gets user preferences from database
        user_prefs = userCursor.fetchone() #Takes the user's record from the database
    finally:
        userConnection.close() #Ends connection between program and database to avoid locking

    explore_pref = user_prefs[0] * 0.1 #Converts user preferences to a multiplier of each
destinations score
    peaceful_pref = user_prefs[1] * 0.05
    cuisine_pref = user_prefs[2] * 0.1

    destConnection = sqlite3.connect('users.db')
    try:
        destCursor = destConnection.cursor()
        destCursor.execute("SELECT * FROM countries")
        countries = destCursor.fetchall() #Makes a complete array of every country in the database
    finally:
        destConnection.close()

    ranking = [] #Makes an empty array to store the rankings of the numerical scores
```

```
destRanking = [] #Makes an empty array to store the rankings of the countries
scoreDict = {} #Makes a dictionary to hold scores and their corresponding countries
for each in countries:
    score = round(each[1] * cuisine_pref + each[2] * peaceful_pref + each[3] * explore_pref)
    #Calculates the score of each country based on user preferences
    if score not in scoreDict: #If the score is not already in the dictionary
        scoreDict[score] = each[0] #Adds the score and country to the dictionary
    else: #If the score is already in the dictionary
        while score in scoreDict: #Makes sure that it doesn't increment to another score
            score += 0.01 #Increments the score by a small amount
        scoreDict[score] = each[0] #Adds the score and country to the dictionary
    ranking.append(score) #Adds that country's scr to the ranking array
    ranking.sort() #Sorts the ranking array
    ranking.reverse() #Reverses the ranking array so that the highest score is first
for each in ranking: #Goes through each score in the ranking array to rank the countries
    destRanking.append(scoreDict[each]) #Adds the country to the destRanking array based on the
score
return destRanking #Returns the array of countries ranked by user preferences

@app.route('/') #Run for a blank URL, e.g: www.tripmaestro.com/
def baseRoute():
    return render_template('login.html') #Returns user to login page

@app.route('/login', methods=['GET', 'POST']) #Run for what would be www.tripmaestro.com/login
#These specify what HTTP methods this route can accept, GET meaning it can request data, and POST
meaning it can send data for processing

def login():
    if request.method == 'POST': #Checks that a form has actually been submitted where data has been
sent, rather than just changing the URL
        username = request.form['username'] #Defines 'username' with what was inputted in with the
label of 'username' in the form
        password = request.form['password'] #Same as above but for the variable 'passowrd' and the
label of 'password' in the HTML
        userConnection = sqlite3.connect('users.db') #Creates an instance of the database
        try:
            cursor = userConnection.cursor() #Creates a cursor that the program can use to interact
with the database
            cursor.execute("SELECT * FROM users WHERE username = ?", (username,)) #Gets all of the
data from a matching username in the database
            user = cursor.fetchone() #This takes a record from the database and assigns it to user.
        finally:
            userConnection.close() #Ends userConnection between program and database

        if user and user[2] == password: #Checks if user exists and inputted password matches what is
in the database
            session['user_id'] = user[0] #Takes ID from database and enters it into the session to
make future requests easier
```

```
        session['username'] = user[1] #Does same thing with username as some things are easier to
request with username
        return redirect(url_for('dashboard')) #Redirects user to their dashboard
(www.tripmaestro.com/dashboard) now logged in
    else:
        flash('Invalid username or password. Please try again.') #Password doesn't match, that is
indicated to the user with a pop-up

    return render_template('login.html') #Makes a default point for if the URL was typed in manually
for www.../login or another error

# Register route
@app.route('/register', methods=['GET', 'POST']) #Enables GET and POST methods for Python to send and
receive data from the webpage
def register():
    if request.method == 'POST': #Checks that data has been received
        username = request.form['username'] #Defines username with what was entered in 'username' on
the webpage
        password = request.form['password'] #Defines password with what was entered in 'password' on
the webpage
        confirm_password = request.form['confirm_password'] #Defines confirm_password with what was
entered as the 2nd password
        explorative_rating = request.form['explorativeRating'] #Defines explorative_rating with value
from webpage
        isolated_rating = request.form['isolatedRating'] #Defines isolated_rating with value from
webpage
        unique_cuisine_rating = request.form['uniqueCuisineRating'] #Defines unique_cuisine_rating
with value from webpage

        if password != confirm_password: #Checks that the passwords are the same
            flash('Passwords do not match!') #Sends a pop-up to the user saying passwords do not match
            return redirect(url_for('register')) #Sends the user back to the registration page to try
again
        try:
            userConnection = sqlite3.connect('users.db') #Same database userConnection as login system
            cursor = userConnection.cursor()
            cursor.execute(f"INSERT INTO users (username, password, explorativeRating, isolatedRating,
uniqueCuisineRating) VALUES ('{username}', '{password}', {explorative_rating}, {isolated_rating},
{unique_cuisine_rating})") #Uses SQL to add new user to database
            userConnection.commit()
            userConnection.close()
            return redirect(url_for('login')) #Takes user to login
        except sqlite3.IntegrityError: #Considers the error of a duplicate username
            flash('Username already exists. Please choose a different one.') #Gives a pop-up to the
user indicating an error
        finally:
            userConnection.close()
```

```
    return render_template('register.html') #If webpage opened directly (with no form filled in), user
    redirected to that webpage

@app.route('/logout')
def logout():
    session.clear() #Clears all details held about user
    return redirect(url_for('login'))

@app.route('/dashboard', methods=['GET', 'POST'])
def dashboard():
    if 'user_id' not in session: #If the user is not logged in
        return redirect(url_for('login')) #Redirects user to login page

    holidayConnection = sqlite3.connect('users.db') #Creates dropdown list of holidays
    try:
        holidayCursor = holidayConnection.cursor()
        holidayCursor.execute("""
            SELECT holidayID, holidayTitle, countryName
            FROM holidays
            WHERE userID = ?
            """, (session['user_id'],))
        user_holidays = [] #Creates an empty array to store the user's holidays
        for row in holidayCursor.fetchall():
            user_holidays.append({'id': row[0], 'title': row[1], 'destination': row[2]})

    finally:
        holidayConnection.close()

    tripadvisorapi = TripAdvisorAPI() #Creates an instance of the TripAdvisorAPI class
    ranked_destinations = determinePreferences() #Gets array of ranked destinations
    page = request.args.get('page', 1, type=int) #Gets the page number from the URL, defaults to 1
    prev_page, next_page = page, page #Initializes variables for previous and next page
    per_page = 4 #Constant used to determine how many destinations are shown on each page
    start_index = (page - 1) * per_page #Finds first destination to be shown on a page
    end_index = start_index + per_page #Finds last destination to be shown on a page
    page_items = [] #Creates an array to store dictionaries of destination data
    for each in range(start_index, end_index): #Goes through each destination in the range of
    destinations to be shown on the page
        name, description, web_url, image_url =
tripadvisorapi.fetch_and_display_details(ranked_destinations[each])
        #Gets the name, description, web URL, and image URL of each destination
        page_items.append({'name': name, 'description': description, 'web_url': web_url, 'image_url':
image_url})
        #Adds the name, description, web URL, and image URL to the page_items array
    if end_index < len(ranked_destinations): #If the end index is less than the total number of
    destinations
        next_page = page + 1 #Sets the next page to the current page plus 1
    if page > 1: #If the page is greater than 1
```

```
prev_page = page - 1 #There has to be a page before it so set it to be that

return render_template("dashboard.html",
                       page_items=page_items,
                       page=page,
                       next_page=next_page,
                       prev_page=prev_page,
                       user_holidays=user_holidays)

@app.route('/new_holiday', methods=['GET', 'POST']) #Expects form submission so needs both GET and
POST methods
def new_holiday():
    if 'user_id' not in session: #If the user is not logged in
        return redirect(url_for('login')) #Redirects user to login page
    if request.method == 'POST': #If the form has been submitted
        destination = request.form['destination']
        startDate = request.form['start-date']
        endDate = request.form['end-date']
        holiday_title = request.form['holiday-title']
        userID = session['user_id'] #Gets all data to be stored in database
        if not destination or not startDate or not endDate or not holiday_title: #If any fields are
empty
            flash('Please fill in all fields')
            return render_template('new_holiday.html', destination=destination) #Sends user back to
the form
        if endDate < startDate: #If the end date is before the start date
            flash('End date must be after start date')
            return render_template('new_holiday.html', destination=destination) #Sends user back to
the form
        tempStartDate = datetime.strptime(startDate, '%Y-%m-%d') #Takes start date and converts it to
a datetime object
        tempEndDate = datetime.strptime(endDate, '%Y-%m-%d') #"" with end date
        if (tempEndDate - tempStartDate).days > 90: #If the holiday is longer than 30 days
            flash('Holidays must be less than 90 days')
            return render_template('new_holiday.html', destination=destination) #Sends user back to
the form
        if tempStartDate < datetime.today():
            flash('Start date must be today or in the future')
            return render_template('new_holiday.html', destination=destination)

        holidayConnection = sqlite3.connect('users.db')
        try:
            holidayCursor = holidayConnection.cursor()
            holidayCursor.execute("INSERT INTO holidays (countryName, startDate, endDate, userID,
holidayTitle ) VALUES (?, ?, ?, ?, ?)",
                                (destination, startDate, endDate, userID, holiday_title)) #Inserts data
into database
            holidayID = holidayCursor.lastrowid #Gets the primary key of the user's holiday
```

```
        holidayConnection.commit()
    finally:
        holidayConnection.close()
    return redirect(url_for('calendar', holidayID = holidayID)) #Redirects user to the calendar
page
    destination = request.args.get('destination') #Gets the destination from the URL if form not
submitted
    if not destination: #If no destination is in the URL
        return redirect(url_for('dashboard')) #Redirects user to the dashboard
    return render_template('new_holiday.html', destination=destination) #If destination is in the URL,
renders the new holiday page

@app.route('/calendar', methods=['GET', 'POST'])
def calendar():
    holidayID = request.args.get('holidayID', -1) # Get the holiday ID from query parameters
    if 'user_id' not in session:
        return redirect(url_for('login'))
    if holidayID == -1:
        return redirect(url_for('dashboard'))

    # Fetch holiday start and end dates
    dateConnection = sqlite3.connect('users.db')
    try:
        dateCursor = dateConnection.cursor()
        dateCursor.execute("SELECT startDate, endDate, cost FROM holidays WHERE holidayID = ?",
(holidayID,))
        holiday_data = dateCursor.fetchone()
        dates = holiday_data[:2] #Extracts the start and end dates
        total_cost = holiday_data[2] #Extracts the cost of the holiday
    finally:
        dateConnection.close()

    start_date = datetime.strptime(dates[0], '%Y-%m-%d') #Takes start date and converts it to a
datetime object
    end_date = datetime.strptime(dates[1], '%Y-%m-%d') #"" with end date

    #Generates a list of all dates for the holiday
    all_dates = []
    for i in range((end_date - start_date).days + 1):
        all_dates.append((start_date + timedelta(days=i)).strftime('%A, %Y-%m-%d'))

    conn = sqlite3.connect('users.db')
    try:
        cursor = conn.cursor()
        cursor.execute("SELECT activityName FROM activities WHERE holidayID = ?", (holidayID,))
        activity_names_db = cursor.fetchall() # Returns a tuple of all activities for the holiday
    finally:
        conn.close()
```

```
activity_names = []
for activity in activity_names_db:
    activity_names.append(activity[0]) #Converts tuple into a list of activity names

conn = sqlite3.connect('users.db')
try:
    cursor = conn.cursor()
    cursor.execute("SELECT activityID, activityName, startTime, location, additionalNotes, price
FROM activities WHERE holidayID = ?", (holidayID,))
    activities = cursor.fetchall()
finally:
    conn.close()

activities_list = []
for activity in activities:
    activities_list.append({
        'id': activity[0], # rowid from the database
        'name': activity[1],
        'time': activity[2],
        'location': activity[3],
        'notes': activity[4],
        'cost' : activity[5]
    }) #Converts tuple into a list of dictionaries with activity details

#Passes the full list of dates to the template
return render_template(
    'calendar.html',
    holidayID=holidayID,
    all_dates=all_dates, #Sends the list of all dates
    activity_names = activity_names, #Sends the list of activity names
    calendar_data = json.dumps(activities_list), #Sends the list of activities' details
    activities_list = activities_list, #Sends the list of activities' details
    total_cost = total_cost #Sends the total cost of the holiday
)

@app.route('/new_activity', methods=['GET', 'POST'])
def new_activity(): #Defines the new_activity function
    if 'user_id' not in session: #Checks if the user is logged in
        return redirect(url_for('login'))
    if request.method == 'POST':
        user_id = session['user_id']
        holidayID = request.form.get('holidayID') #Gets the holiday ID from the hidden input field
        if not holidayID:
            return redirect(url_for('dashboard')) #If no holiday ID is found, user is redirected to
the dashboard
        activity_name = request.form.get('activity_name')
        if len(activity_name) > 16:
```

```
        flash('Activity name must be 16 characters or less')
        return redirect(url_for('new_activity', holidayID=holidayID))
    activity_date = request.form.get('activity_date')
    activity_location = request.form.get('activity_location')
    booking_link = request.form.get('booking_link')
    additional_notes = request.form.get('additional_notes') #Gets all data to be stored in
database
    activity_cost = request.form.get('activity_cost') or 0

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    try:
        cursor.execute("""
            INSERT INTO activities (holidayID, userID, startTime, location, bookingLink,
additionalNotes, activityName, price)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            """, (holidayID, user_id, activity_date, activity_location, booking_link,
additional_notes, activity_name, activity_cost))
        #Inserts data into database
        cursor.execute("SELECT SUM(price) FROM activities WHERE holidayID = ?", (holidayID,))
#Gets the total cost of all activities
        total_cost = cursor.fetchone()[0] or 0 #Gives sum of all prices, or 0 if no activities
        cursor.execute("UPDATE holidays SET cost = ? WHERE holidayID = ?", (total_cost,
holidayID)) #Updates the total cost of the holiday for relevant holiday
        conn.commit()
    finally:
        conn.close()
    return redirect(url_for('calendar', holidayID=holidayID))
    holidayID = request.args.get('holidayID') #If form isn't filled in, holiday ID is taken from URL
    if holidayID:
        return render_template('new_activity.html', holidayID=holidayID)
    return redirect(url_for('dashboard'))

@app.route('/save_calendar', methods=['POST'])
def save_calendar():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    holiday_id = request.form.get('holiday-id')
    holiday_title_list = request.form.getlist('holiday-title')
    if holiday_title_list: #Makes a list of holiday titles to add to the database
        holiday_title = holiday_title_list[0]
    else:
        holiday_title = ''
    calendar_data = json.loads(request.form.get('calendar-data')) #Gets calendar data from the form

    conn = sqlite3.connect('users.db')
    try:
```

```
        cursor = conn.cursor()
        if holiday_title: #Sets holiday title to new title
            cursor.execute("UPDATE holidays SET holidayTitle = ? WHERE holidayID = ?",
                            (holiday_title, holiday_id))

        for activity in calendar_data: #Goes through each activity to add the start time to the
database
            cursor.execute("UPDATE activities SET startTime = ? WHERE activityID = ?",
                            (activity['startTime'], activity['activityId']))

        conn.commit()
    finally:
        conn.close()

    return redirect(url_for('calendar', holidayID=holiday_id)) #Opens calendar page back up

if __name__ == '__main__':
    app.run(debug=True)
```

login.html

```
<html>
<head>
  <link rel="stylesheet" href="{{ url_for('static', filename='styles.css')
  }}">
  <title>Login - TripMaestro</title>
</head>
<body>
  <h1>Welcome to TripMaestro</h1>
  <div>
    {% for message in get_flashed_messages() %}
      <div>{{ message }}</div>
    {% endfor %}
  </div>
  <form action="/login" method="post">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>
    <br>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>
    <br>
    <button type="submit">Login</button>
  </form>
  <p>Don't have an account? <a href="/register">Sign up</a></p>
</body>
</html>
```

plan.html

```
<html>
<head>
  <link rel="stylesheet" href="{{ url_for('static', filename='plan.css')
  }}">
  <title>Holiday Planner</title>
</head>
<body>
  <header class="planner-header">
    <h1>Holiday Planner</h1>
    <a href="/logout" class="small-text">Sign out</a>
  </header>

  <div class="settings">
    <h2>
      <label for="holiday-title">Title:</label>
      <input type="text" id="holiday-title" name="holiday-title"
value="My Holiday" placeholder="Enter a title">
    </h2>
```

```
<p class="small-text">
  First day of the week:
  <select name="first-day" id="first-day">
    <option value="Monday">Monday (Default)</option>
    <option value="Sunday">Sunday</option>
  </select>
</p>
</div>

<div class="calendar-container">
  <div class="time-row-header"></div>
  <div class="day-header">Monday</div>
  <div class="day-header">Tuesday</div>
  <div class="day-header">Wednesday</div>
  <div class="day-header">Thursday</div>
  <div class="day-header">Friday</div>
  <div class="day-header">Saturday</div>
  <div class="day-header">Sunday</div>

  {% for hour in range(7, 23) %}
    <div class="time-row-header">{{ "%02d:00" | format(hour) }}</div>
    {% for day in range(7) %}
      <div class="calendar-cell" data-day="{{ day }}" data-hour="{{ hour
    }}"></div>
    {% endfor %}
  {% endfor %}
</div>

<form action="/save_holiday" method="POST">
  <input type="hidden" id="calendar-data" name="calendar-data" value="">
  <button type="submit" class="save-button">Save</button>
</form>

<p class="footer-note">Toggle hidden hours: 11 PM → 7 AM</p>

<script>
  // Minimal JavaScript to capture calendar data
  document.querySelector('.save-button').addEventListener('click', () =>
{
  const calendarData = [];
  document.querySelectorAll('.calendar-cell').forEach(cell => {
    const day = cell.dataset.day;
    const hour = cell.dataset.hour;
    const activity = cell.textContent.trim();
    if (activity) {
      calendarData.push({ day, hour, activity });
    }
  });
});
```

```
        document.getElementById('calendar-data').value =
JSON.stringify(calendarData);
    });
</script>
</body>
</html>
```

register.html

```
<html>
<head>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css')
    }}">
    <title>Register for TripMaestro</title>
</head>
<body>
    <h1>Register</h1>

    <div>
        {% for message in get_flashed_messages() %}
            <div>{{ message }}</div>
        {% endfor %}
    </div>

    <form action="{{ url_for('register') }}" method="POST">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username" required>

        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required>

        <label for="confirm_password">Confirm Password:</label>
        <input type="password" id="confirm_password" name="confirm_password"
required>

        <label for="explorativeRating">How much do you prioritise exploring
the area in your holiday?</label>
        <select id="explorativeRating" name="explorativeRating" required>
            <option value="1">1</option>
            <option value="2">2</option>
            <option value="3">3</option>
            <option value="4">4</option>
            <option value="5">5</option>
            <option value="6">6</option>
            <option value="7">7</option>
            <option value="8">8</option>
            <option value="9">9</option>
            <option value="10">10</option>
```

```
</select>
<br><br>
<label for="isolatedRating">How much do you prioritise a peaceful,
isolated area?</label>
<select id="isolatedRating" name="isolatedRating" required>
  <option value="1">1</option>
  <option value="2">2</option>
  <option value="3">3</option>
  <option value="4">4</option>
  <option value="5">5</option>
  <option value="6">6</option>
  <option value="7">7</option>
  <option value="8">8</option>
  <option value="9">9</option>
  <option value="10">10</option>
</select>
<br>
<br>
<label for="uniqueCuisineRating">How much do you value a unique
cuisine?</label>
<select id="uniqueCuisineRating" name="uniqueCuisineRating" required>
  <option value="1">1</option>
  <option value="2">2</option>
  <option value="3">3</option>
  <option value="4">4</option>
  <option value="5">5</option>
  <option value="6">6</option>
  <option value="7">7</option>
  <option value="8">8</option>
  <option value="9">9</option>
  <option value="10">10</option>
</select>
<br><br>
<button type="submit">Register</button>
</form>
</body>
</html>
```

dashboard.html

```
<html>
<head>
  <link rel="stylesheet" href="{{ url_for('static', filename='styles.css')
  }}">
  <title>Register for TripMaestro</title>
</head>
<body>
  <h1>Register</h1>

  <div>
    {% for message in get_flashed_messages() %}
      <div>{{ message }}</div>
    {% endfor %}
  </div>

  <form action="{{ url_for('register') }}" method="POST">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>

    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>

    <label for="confirm_password">Confirm Password:</label>
    <input type="password" id="confirm_password" name="confirm_password"
required>

    <label for="explorativeRating">How much do you prioritise exploring
the area in your holiday?</label>
    <select id="explorativeRating" name="explorativeRating" required>
      <option value="1">1</option>
      <option value="2">2</option>
      <option value="3">3</option>
      <option value="4">4</option>
      <option value="5">5</option>
      <option value="6">6</option>
      <option value="7">7</option>
      <option value="8">8</option>
      <option value="9">9</option>
      <option value="10">10</option>
    </select>
    <br><br>
    <label for="isolatedRating">How much do you prioritise a peaceful,
isolated area?</label>
    <select id="isolatedRating" name="isolatedRating" required>
      <option value="1">1</option>
      <option value="2">2</option>
      <option value="3">3</option>
```

```
<option value="4">4</option>
<option value="5">5</option>
<option value="6">6</option>
<option value="7">7</option>
<option value="8">8</option>
<option value="9">9</option>
<option value="10">10</option>
</select>
<br>
<br>
<label for="uniqueCuisineRating">How much do you value a unique
cuisine?</label>
<select id="uniqueCuisineRating" name="uniqueCuisineRating" required>
  <option value="1">1</option>
  <option value="2">2</option>
  <option value="3">3</option>
  <option value="4">4</option>
  <option value="5">5</option>
  <option value="6">6</option>
  <option value="7">7</option>
  <option value="8">8</option>
  <option value="9">9</option>
  <option value="10">10</option>
</select>
<br><br>
<button type="submit">Register</button>
</form>
</body>
</html>
```

dashboard.html

```
<html>
<head>
  <link rel="stylesheet" href="{{ url_for('static', filename='styles.css')
  }}">
  <title>Register for TripMaestro</title>
</head>
<body>
  <h1>Register</h1>

  <div>
    {% for message in get_flashed_messages() %}
      <div>{{ message }}</div>
    {% endfor %}
  </div>

  <form action="{{ url_for('register') }}" method="POST">
```

```
<label for="username">Username:</label>
<input type="text" id="username" name="username" required>

<label for="password">Password:</label>
<input type="password" id="password" name="password" required>

<label for="confirm_password">Confirm Password:</label>
<input type="password" id="confirm_password" name="confirm_password"
required>

<label for="explorativeRating">How much do you prioritise exploring
the area in your holiday?</label>
<select id="explorativeRating" name="explorativeRating" required>
  <option value="1">1</option>
  <option value="2">2</option>
  <option value="3">3</option>
  <option value="4">4</option>
  <option value="5">5</option>
  <option value="6">6</option>
  <option value="7">7</option>
  <option value="8">8</option>
  <option value="9">9</option>
  <option value="10">10</option>
</select>
<br><br>
<label for="isolatedRating">How much do you prioritise a peaceful,
isolated area?</label>
<select id="isolatedRating" name="isolatedRating" required>
  <option value="1">1</option>
  <option value="2">2</option>
  <option value="3">3</option>
  <option value="4">4</option>
  <option value="5">5</option>
  <option value="6">6</option>
  <option value="7">7</option>
  <option value="8">8</option>
  <option value="9">9</option>
  <option value="10">10</option>
</select>
<br>
<br>
<label for="uniqueCuisineRating">How much do you value a unique
cuisine?</label>
<select id="uniqueCuisineRating" name="uniqueCuisineRating" required>
  <option value="1">1</option>
  <option value="2">2</option>
  <option value="3">3</option>
  <option value="4">4</option>
```

```
        <option value="5">5</option>
        <option value="6">6</option>
        <option value="7">7</option>
        <option value="8">8</option>
        <option value="9">9</option>
        <option value="10">10</option>
    </select>
    <br><br>
    <button type="submit">Register</button>
</form>
</body>
</html>
```

calendar.html

```
<html>
<head>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css')
    }}">
    <title>Holiday Planner</title>
</head>
<body>
    <header>
        <div class="planner-header">
            <a href="{{ url_for('dashboard') }}" class="title-
link"><h1>TripMaestro</h1></a>
        </div>
        <div class="header-right">
            <a href="/logout" class="btn-signout">Sign Out</a>
        </div>
    </header>

    <div class="settings">
        <h2>
            <input type="text" id="holiday-title" name="holiday-title"
value="" placeholder="Enter a title to save holiday">
        </h2>
    </div>

    <div class="budget-info">
        <h3>Budget Summary</h3>
        <p>Total Cost: £{{ "%.2f"|format(total_cost|float) }}</p> <!-- Formats
the total cost to 2 decimal places -->
    </div>

    <div class="vertical-strip">
        <div class="activity-buttons">
```

```

        <a href="{ url_for('new_activity', holidayID=holidayID) }"
class="btn-add-activity">Add New Activity</a>
        <button id="delete-activity-btn" class="btn-delete-
activity">Delete Activity</button>
    </div>

    <div class="selected-activity">
        <h2>Currently Selected Activity</h2>
        <p><strong>Name:</strong> <span id="activity-name">None</span></p>
        <p><strong>Time:</strong> <span id="activity-time">None</span></p>
        <p><strong>Location:</strong> <span id="activity-
location">None</span></p>
        <p><strong>Notes:</strong> <span id="activity-
notes">None</span></p>
    </div>

    <div class="activity-palette">
    {% for activity in activities_list %}
        <div
            class="activity-block"
            draggable="true"
            data-id="{ activity['id'] }"
            data-name="{ activity['name'] }"
            data-location="{ activity['location'] }"
            data-notes="{ activity['notes'] }">
                {{ activity['name'] }}
            </div>
        {% endfor %}
    </div>
</div>

<div class="calendar-container">
    <div class="day-row-header"> </div>
    {% for hour in range(7, 23) %}
        <div class="time-slot-header">{{ "%02d:00" | format(hour) }}</div>
    {% endfor %}

    {% for date in all_dates %}
        <div class="day-row-header">{{ date }}</div>
        {% for hour in range(7, 23) %}
            <div
                class="calendar-cell"
                data-date="{ date }"
                data-hour="{ hour }"
                data-activity-id="">
            </div>
        {% endfor %}
    {% endfor %}

```

```
</div>

<form id="save-form" action="/save_calendar" method="POST">
  <input type="hidden" id="saved-calendar-data" value='{{ calendar_data
}}'>
  <input type="hidden" id="calendar-data" name="calendar-data" value="">
  <input type="hidden" name="holiday-id" value="{{ holidayID }}">
  <input type="hidden" id="holiday-title" name="holiday-title" value="">
  <button type="submit" class="save-button">Save</button>
</form>

<script>

document.addEventListener('DOMContentLoaded', function() {
  const savedData = JSON.parse(document.getElementById('saved-calendar-
data').value); //Gets calendar data

  savedData.forEach(activity => { //Iterates through each activity
    const parts = activity.time.split(' '); // Split into ["Friday,",
"2025-04-04", "7:00"]
    const activityDate = parts[1]; // "2025-04-04"
    const activityHour = parts[2].split(':')[0]; // "7"

    //Finds the corresponding cell for that date and time
    const cells = document.querySelectorAll('.calendar-cell');
    cells.forEach(cell => {
      const cellDate = cell.getAttribute('data-date').split(',
')[1]; // Get YYYY-MM-DD
      const cellHour = cell.getAttribute('data-hour');

      // If this is the right cell, populate it
      if (cellDate === activityDate && cellHour === activityHour) {
        cell.textContent = activity.name;
        cell.setAttribute('data-activity-id', activity.id);
        cell.setAttribute('data-name', activity.name);
        cell.setAttribute('data-location', activity.location);
        cell.setAttribute('data-notes', activity.notes);
      }
    });
  });

  document.getElementById('save-form').addEventListener('submit',
function(event) {
  event.preventDefault();

  const activities = [];
  document.querySelectorAll('.calendar-cell').forEach(cell => {
```

```
const activityId = cell.getAttribute('data-activity-id');
if (activityId) {
    activities.push({
        activityId: activityId,
        startTime: `${cell.getAttribute('data-date')}${cell.getAttribute('data-hour')}:00`
    });
}
});

// Set the form values
this.querySelector('#calendar-data').value = JSON.stringify(activities);
const titleInput = document.querySelector('input[name="holiday-title"]');
if (titleInput) {
    this.querySelector('#holiday-title').value = titleInput.value;
}

// Submit the form
this.submit();
});

document.querySelectorAll('.activity-block').forEach(activity => {
    activity.addEventListener('dragstart', event => {
        event.dataTransfer.setData('activity-id',
activity.getAttribute('data-id'));
        event.dataTransfer.setData('activity-name',
activity.getAttribute('data-name'));
        event.dataTransfer.setData('activity-location',
activity.getAttribute('data-location'));
        event.dataTransfer.setData('activity-notes',
activity.getAttribute('data-notes'));
    });
});

document.querySelectorAll('.calendar-cell').forEach(cell => { //Select all
calendar cells
    cell.addEventListener('dragover', event => { //Add dragover event
listener
        event.preventDefault(); //Prevents text from being draggable
    });

    cell.addEventListener('drop', event => {
        event.preventDefault();

        const activityId = event.dataTransfer.getData('activity-id');
        const activityName = event.dataTransfer.getData('activity-name');
        const activityLocation = event.dataTransfer.getData('activity-
location');
```

```
const activityNotes = event.dataTransfer.getData('activity-
notes');

cell.textContent = activityName;

cell.setAttribute('data-activity-id', activityId);
cell.setAttribute('data-name', activityName);
cell.setAttribute('data-location', activityLocation);
cell.setAttribute('data-notes', activityNotes);
});
cell.addEventListener('click', () => {
    document.getElementById('activity-name').textContent =
cell.getAttribute('data-name') || 'None';
    document.getElementById('activity-time').textContent =
`${cell.getAttribute('data-hour')}:00`; // $ makes 7 --> 7:00
    document.getElementById('activity-location').textContent =
cell.getAttribute('data-location') || 'None';
    document.getElementById('activity-notes').textContent =
cell.getAttribute('data-notes') || 'None';
});
});

let deleteMode = false; //Flag toggles whether it click should delete cells

document.getElementById('delete-activity-btn').addEventListener('click', () =>
{
    deleteMode = !deleteMode; //Toggles delete mode
    const calendarCells = document.querySelectorAll('.calendar-cell');

    if (deleteMode) {
        document.getElementById('delete-activity-btn').textContent = 'Exit
Delete Mode';

        calendarCells.forEach(cell => {
            cell.addEventListener('click', function deleteActivity() {
                if (deleteMode) {
                    if (cell.textContent.trim() !== '') {
                        cell.textContent = ''; //Clears the contents of the
cell

                        cell.removeAttribute('data-activity-id');
                        cell.removeAttribute('data-name');
                        cell.removeAttribute('data-location');
                        cell.removeAttribute('data-notes');
                    }
                }
            });
        });
    } else {
```

```
        document.getElementById('delete-activity-btn').textContent = 'Delete
Activity';
    }
});

</script>
</body>
</html>
```

new_activity.html

```
<html>
<head>
    <title>Add Activity</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css')
}}">
</head>
<body>
    <header>
        <div class="planner-header">
            <a href="{{ url_for('dashboard') }}" class="title-
link"><h1>TripMaestro</h1></a>
        </div>
        <div class="header-right">
            <a href="/logout" class="btn-signout">Sign Out</a>
        </div>
    </header>

    <a href="/calendar?holidayID={{ holidayID }}" class="back-link">Go back to
holiday list</a>

    <div>
        {% for message in get_flashed_messages() %}
            <div>{{ message }}</div>
        {% endfor %}
    </div>

    <form action="/new_activity" method="POST">
        <input type="hidden" name="holidayID" value="{{ holidayID }}">

        <!-- Fields to create a new activity -->
        <div class="form-group">
            <label for="activity-name">Activity Name:</label>
            <input type="text" id="activity-name" name="activity_name"
placeholder="Enter activity name" required>
        </div>

        <div class="form-group inline">
```

```

        <label for="activity-date">Date: (optional)</label>
        <input type="date" id="activity-date" name="activity_date">
    </div>

    <div class="form-group">
        <label for="activity-location">Location (optional):</label>
        <input type="text" id="activity-location" name="activity_location"
placeholder="Enter location">
    </div>

    <div class="form-group">
        <label for="booking-link">Link to Booking (optional):</label>
        <input type="url" id="booking-link" name="booking_link"
placeholder="Enter a link to booking, include https://www..">
    </div>

    <div class="form-group">
        <label for="additional-notes">Notes:</label>
        <textarea id="additional-notes" name="additional_notes"
placeholder="Add any additional notes here"></textarea>
    </div>

    <div class="form-group">
        <label for="activity-cost">Cost (£):</label>
        <input type="number" id="activity-cost" name="activity_cost"
min="0" step="0.01" placeholder="Enter activity cost">
    </div>

    <button type="submit">Save</button>
</form>
</body>
</html>

```

new_holiday.html

```

<html>
<head>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css')
}}">
    <title>Create Holiday</title>
    <div class="header-right">
        <a href="/logout" class="btn-signout">Sign Out</a>
    </div>
</head>
<body>
    <header>
        <a href="{{ url_for('dashboard') }}" class="planner-
header"><h1>TripMaestro</h1></a>

```

```
<div class="back-to-dashboard">
  <a href="/dashboard" class="small-text">Back to Dashboard</a>
</div>
</header>

<main class="setup-form">
  <p class="selected-destination">
    <strong>Destination:</strong> {{ destination }}
  </p>
  <br>
  <br>
  <div class="flash-messages">
    {% for message in get_flashed_messages() %}
      <div>{{ message }}</div>
    {% endfor %}
  </div>
  <br>
  <br>
  <form action="/new_holiday" method="POST">
    <!-- Hidden input to pass the destination -->
    <input type="hidden" name="destination" value="{{ destination }}">

    <div class="form-group">
      <label for="holiday-title">Holiday Title:</label>
      <input type="text" id="holiday-title" name="holiday-title"
placeholder="Enter a title for your holiday" required>
    </div>

    <div class="form-group">
      <label for="start-date">Start Date:</label>
      <input type="date" id="start-date" name="start-date" required>
    </div>

    <div class="form-group">
      <label for="end-date">End Date:</label>
      <input type="date" id="end-date" name="end-date" required>
    </div>

    <button type="submit" class="save-button">Continue to
Planner</button>
  </form>
</main>
</body>
</html>
```

styles.css

```
/* Global Styles */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: Arial, sans-serif;
}

body {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background-color: #f5f5f5;
  margin: 0;
}

h1 {
  color: #444;
  margin-bottom: 20px;
  font-size: 2rem;
  text-align: center;
}

h1, h2 {
  text-align: center;
  margin-bottom: 10px;
}

form {
  background: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  width: 100%;
  max-width: 400px;
}

label {
  display: block;
  margin-bottom: 8px;
  font-weight: bold;
}

input, select, textarea {
  width: 100%;
```

```
padding: 10px;
margin-bottom: 15px;
border: 1px solid #ddd;
border-radius: 4px;
font-size: 1rem;
}

button {
  width: 100%;
  padding: 10px;
  background-color: #007BFF;
  color: #fff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 1rem;
}

button:hover {
  background-color: #0056b3;
}

p {
  margin-top: 15px;
  text-align: center;
}

a {
  color: #007BFF;
  text-decoration: none;
}

a:hover {
  text-decoration: underline;
}

.back-to-dashboard {
  text-align: center;
  display: block;
  margin-bottom: 20px;
  color: #007BFF;
  text-decoration: none;
}

.back-to-dashboard:hover {
  text-decoration: underline;
}
```

```
.flash-messages {
  text-align: center;
}

/* Header Styles */
.header-right {
  position: absolute;
  top: 0;
  right: 0;
  display: flex;
  justify-content: flex-end;
  align-items: center;
  padding: 10px 20px;
  z-index: 1000;
}

.planner-header {
  position: absolute;
  top: 20px; /* Add vertical margin */
  left: 20px; /* Add horizontal margin */
  font-size: 2rem;
  color: #444;
  font-weight: bold;
}

/* Settings for "Title" */
.settings {
  display: flex;
  align-items: center;
  justify-content: center; /* Center horizontally */
  margin-top: 80px; /* Add space below "Trip Maestro" */
}

.settings label {
  margin-right: 10px; /* Spacing between the label and input box */
  font-size: 1.2rem;
  font-weight: bold;
}

.settings input {
  width: 300px; /* Increase the width */
  font-size: 1rem; /* Ensure font size is readable */
  padding: 8px; /* Add padding for better appearance */
  border: 1px solid #ddd;
  border-radius: 5px;
}

/* Dashboard-Specific Styles */
```

```
.dashboard-header {
  background: #fff;
  width: 100%;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.header-left,
.header-right {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.header-left h1 {
  font-size: 1.5rem;
  margin: 0;
  color: #444;
}

.dashboard-main {
  width: 90%;
  max-width: 1200px;
  margin: 40px auto;
}

.dashboard-main p {
  font-size: 1.1rem;
  margin-bottom: 20px;
  color: #444;
}

.destination-table {
  width: 100%;
  border-collapse: collapse;
  background-color: #fff;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.destination-table thead {
  background-color: #fafafa;
}

.destination-table th,
.destination-table td {
```

```
border: 1px solid #ddd;
padding: 12px;
text-align: left;
font-size: 0.95rem;
color: #333;
}

.destination-table th {
  font-weight: bold;
}

/* Calendar-Specific Styles */
.calendar-container {
  display: grid;
  grid-template-columns: 200px repeat(16, 80px);
  gap: 1px;
  background-color: #ccc;
  max-width: 80%;
  margin: 0 auto;
  border: 1px solid #ddd;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}

.time-row-header {
  font-weight: bold;
  text-align: center;
  background-color: #f0f0f0;
  font-size: 0.8rem;
  padding: 5px;
}

.day-header {
  text-align: center;
  background-color: #007BFF;
  color: white;
  font-weight: bold;
  font-size: 0.9rem;
  padding: 10px;
}

.calendar-cell {
  background-color: white;
  min-height: 70px; /* Increased height for better text visibility */
  border: 1px solid #ddd;
  position: relative;
  overflow: visible; /* Allow text overflow if necessary */
  text-align: center;
  display: flex; /* Flexbox for centering content */
}
```

```
align-items: center; /* Vertical centering */
justify-content: center; /* Horizontal centering */
font-size: clamp(0.8rem, 1vw, 1.2rem); /* Adjust font size for readability
*/
white-space: nowrap; /* Prevent text from wrapping */
text-overflow: ellipsis; /* Show ellipsis if text overflows */
}

.calendar-cell div {
  display: none; /* Ensure no child div overrides the textContent */
}

.save-button {
  display: block;
  margin: 20px auto;
  padding: 10px 20px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 1rem;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.save-button:hover {
  background-color: #0056b3;
}

.activity-palette {
  display: flex;
  justify-content: center;
  margin-bottom: 20px;
  gap: 10px;
}

.activity-block {
  background-color: #ffc107;
  border: 1px solid #ddd;
  padding: 10px;
  border-radius: 5px;
  text-align: center;
  cursor: grab;
  font-size: 0.9rem;
}

.activity-block:hover {
```

```
        background-color: #e0a800;
    }

    .calendar-cell:hover {
        background-color: #e0e0e0;
    }

    .week-info {
        text-align: center;
        font-size: 1.5rem;
        font-weight: bold;
        margin-bottom: 20px;
        color: #333;
    }

    .activity-buttons {
        display: flex;
        justify-content: center;
        margin: 20px 0;
        text-align: center;
    }

    .btn-add-activity {
        display: inline-block;
        padding: 10px 20px;
        background-color: #28a745;
        color: white;
        text-decoration: none;
        border-radius: 5px;
        font-size: 1rem;
        text-align: center;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
    }

    .btn-add-activity:hover {
        background-color: #218838;
    }

    .selected-destination{
        font-size: 1.5rem
    }

    .day-row-header {
        font-weight: bold;
        text-align: center;
        background-color: #f0f0f0;
        font-size: 1rem; /* Adjust font size for better readability */
        padding: 10px; /* Add padding for more space */
    }
```

```
}

.btn-delete-activity {
  background-color: #ff4d4d;
  color: white;
  border: none;
  border-radius: 5px;
  padding: 10px 20px;
  cursor: pointer;
  font-size: 1rem;
  margin-left: 10px; /* Space between buttons */
}

.btn-delete-activity:hover {
  background-color: #e60000;
}

/* Parent container for alignment */
.vertical-strip {
  display: flex; /* Align items in a row */
  flex-direction: row; /* Default direction for flex */
  align-items: flex-start; /* Align items to the top */
  gap: 20px; /* Add spacing between items */
  margin-left: 20px; /* Add left margin */
}

/* Style for the buttons container */
.activity-buttons {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

/* Style for the "Currently Selected Activity" container */
.selected-activity {
  border: 1px solid #ddd;
  padding: 15px;
  background-color: #f9f9f9;
  border-radius: 5px;
  width: 250px; /* Set a fixed width for consistency */
  margin-left: 20px; /* Add margin to move it slightly */
}

.budget-info {
  background-color: #f8f9fa;
  padding: 15px;
  border-radius: 5px;
  margin: 10px;
}
```

```
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.budget-info h3 {
    margin: 0 0 10px 0;
    color: #333;
}

.budget-info p {
    margin: 0;
    font-size: 1.2em;
    color: #2c3e50;
}
```